



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

ASSIGNMENT-1

The domain of the Project:
Physical Design (VLSI)

Team Mentors (and their designation):
Veeramani Sir (Staff Engineer -Synopsys)

Name: - SHAMS TANVEER

Period of the project
November 2025 to May 2026



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

Linux Command

chmod – Change File Permissions

The chmod command is used to change the access permissions of a file or directory in Linux. It controls who can read, write, or execute a file. The permission is represented by three digits, where the first digit is for the owner, the second for the group, and the third for others. Each digit is a combination of read (4), write (2), and execute (1).

- It defines access control for files and directories
- Uses numeric values (4 = read, 2 = write, 1 = execute)
- Three digits represent owner, group, and others

Example:

```
chmod 642 file.txt
```

Explanation:

- Owner → read + write (6)
- Group → read (4)
- Others → write (2)

grep – Search Pattern in File

The grep command is used to search for specific patterns or words inside a file. It processes the file line by line and prints the entire line wherever the pattern is found. It is very useful for filtering logs and large text files.

- Searches text inside files
- Works line by line
- Returns matching lines

Example:

```
grep -i "error" log.txt
```

Explanation:

- -i ignores case sensitivity
- It will match "Error", "ERROR", etc.
- Displays all lines containing the word "error"



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

awk – Text Processing Tool

The awk command is used for processing and analyzing structured text data. It reads files line by line, where each line is called a record, and each word is called a field. It is commonly used to extract specific columns from data files.

- Processes text data field-wise
- Each line = record
- Each word = field

Example:

```
awk '{print $1, $2}' data.txt
```

Explanation:

- \$1 → first field
- \$2 → second field
- Prints first and second columns from the file

vi – Text Editor

The vi command is used to open and edit text files in Linux. It operates in different modes such as command mode, insert mode, and last line mode. It is widely used for editing configuration files.

- Used to create and edit files
- Has different modes (command, insert, last line)
- Efficient for terminal-based editing

Example:

```
vi file.txt
```

Explanation:

- Opens the file in editor
- Press i to enter insert mode
- Press Esc then type :wq to save and exit



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

Basic Commands

- **ls – List Files**

The ls command is used to display files and directories in the current location.

- Shows directory contents
- Can show hidden files and details

Example:

```
ls -l
```

Explanation:

- -l shows detailed information like permissions, size, and date

- **cd – Change Directory**

The cd command is used to move between directories in the system.

- Helps in navigation
- Can move forward or backward

Example:

```
cd ..
```

Explanation:

- Moves one directory back

- **pwd – Print Working Directory**

The pwd command shows the current directory path where the user is working.

- Displays full path
- Useful for navigation

Example:

```
pwd
```

Explanation:

- Outputs current directory location



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

- **mkdir – Create Directory**

The mkdir command is used to create a new directory.

- Creates folders
- Helps organize files

Example:

```
mkdir project
```

Explanation:

- Creates a folder named "project"

- **rm – Remove Files**

The rm command is used to delete files or directories.

- Deletes files permanently
- Can remove folders using options

Example:

```
rm file.txt
```

Explanation:

- Deletes the file named "file.txt"



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

OpenLane Environment

OpenLane is an automated RTL-to-GDSII flow used for digital ASIC design. It works on top of tools like OpenROAD, Magic, and Netgen, and is typically run inside a Docker-based environment or local Linux setup. The environment includes all required EDA tools and configuration files needed for chip design.

- Provides complete ASIC design flow (synthesis → layout → verification)
- Uses pre-installed open-source tools
- Runs using Docker or local installation
- Requires PDK (Process Design Kit) like Sky130

Installation of Git

Git is a distributed version control system used to manage and track changes in source code. It is mainly used to clone repositories from platforms like GitHub, which is necessary for downloading the OpenLane project. Installing Git ensures that the user can access, update, and manage project files efficiently.

- Used for cloning repositories (e.g., OpenLane from GitHub)
- Helps in version control and tracking changes
- Lightweight and widely used in development environments

Example:

```
sudo apt install git -y
```

Explanation:

- `sudo apt install git -y` → Installs Git without manual confirmation

```
shams@VLSTI:~/Desktop$ sudo apt install git -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
git is already the newest version (1:2.43.0-1ubuntu7.3).
0 upgraded, 0 newly installed, 0 to remove and 8 not upgraded.
shams@VLSTI:~/Desktop$
```

Installation of Docker

Docker is a containerization platform that allows applications like OpenLane to run in an isolated environment with all dependencies included. It ensures consistency across different systems and avoids compatibility issues. Docker is essential for running OpenLane smoothly.



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

- Provides isolated environment using containers
- Avoids dependency conflicts
- Required to run OpenLane flow

Example:

```
sudo apt install docker.io -y
```

Explanation:

- `sudo apt install docker.io -y` → Installs Docker engine

```
shams@VLSI:~/Desktop$ sudo apt install docker.io -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
docker.io is already the newest version (29.1.3-0ubuntu3~24.04.1).
0 upgraded, 0 newly installed, 0 to remove and 8 not upgraded.
shams@VLSI:~/Desktop$
```

Post Installation Setup (Docker)

After installing Docker, some additional setup steps are required to start and configure the service.

Example:

```
sudo systemctl start docker
sudo systemctl enable docker
sudo usermod -aG docker $USER
```

Explanation:

- `systemctl start docker` → Starts Docker service
- `systemctl enable docker` → Enables auto-start at boot
- `usermod -aG docker $USER` → Allows user to run Docker without sudo



```
shams@VLSI:~/Desktop$ git clone https://github.com/The-OpenROAD-Project/OpenLane
git
fatal: destination path 'OpenLane' already exists and is not an empty directory.
shams@VLSI:~/Desktop$ cd OpenLane
shams@VLSI:~/Desktop/OpenLane$ ls
AUTHORS.md      designs      gui.py       Makefile     scripts
configuration   docker      install     README.md   tests
CONTRIBUTING.md docs        Jenkinsfile regression_results venv
default.cvcrc   env.py      klayoutrc   requirements.txt
dependencies     flow.tcl   LICENSE     run_designs.py
shams@VLSI:~/Desktop/OpenLane$
```

Terminal

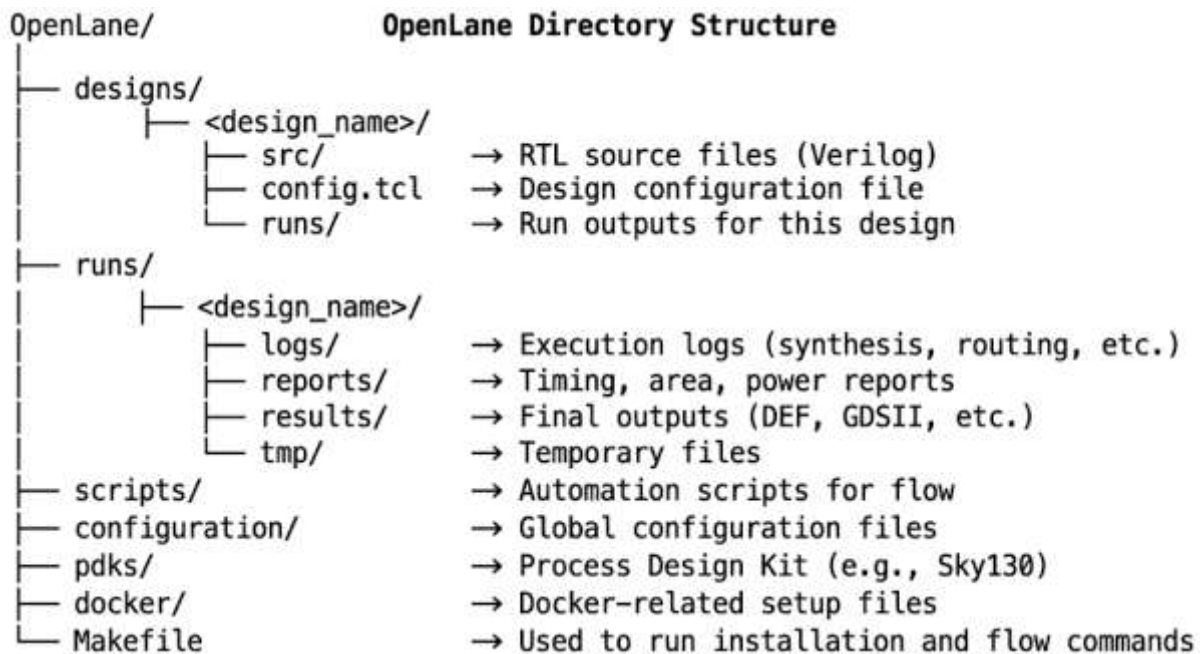


Fig: - Directory Structure of OpenLane

Reports in OpenLane

OpenLane generates multiple reports at different stages of the design flow. These reports help analyze performance, timing, area, and power of the design.



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

- Located inside runs/<design_name>/reports/
- Includes synthesis, floorplan, placement, routing reports
- Used for debugging and optimization

Example:

```
cd runs/spm/reports/synthesis  
ls
```

Explanation:

- Opens synthesis report folder
- Displays files like area report, timing report

Analysis of Logs

Logs are generated during each step of the OpenLane flow and are useful for debugging errors and understanding tool execution.

- Located inside runs/<design_name>/logs/
- Separate logs for synthesis, placement, routing
- Helps identify errors and warnings

Example:

```
cd runs/spm/logs/synthesis  
less synthesis.log
```

Explanation:

- Opens synthesis log file
- Shows step-by-step execution details

TCL Scripting

TCL (Tool Command Language) scripting is a simple and powerful scripting language widely used in VLSI design automation tools, including OpenLane, for controlling and automating the ASIC design flow. It is primarily used to define configurations, execute commands, and manage different stages such as synthesis, floorplanning, placement, and routing. In OpenLane, TCL scripts (like config.tcl) specify design parameters such as clock period, design name, input files, and constraints, allowing users to customize the flow without modifying the core tool. TCL scripts are easy to write and follow a command-based structure where each line represents an instruction executed sequentially. This scripting capability helps automate repetitive tasks, ensures consistency in design execution, and enables efficient control over the entire chip design process.



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

Example:

```
set ::env(DSIGN_NAME) "spm"
set ::env(CLOCK_PERIOD) 10
set ::env(VERILOG_FILES) "designs/spm/src/spm.v"
set ::env(SYNTH_STRATEGY) "AREA 0"
```

TCL scripting in OpenLane acts as the **backbone of automation**, enabling users to control the entire chip design flow efficiently through simple commands.

```
shams@VLSI:~/Desktop/OpenLane$ make mount
cd /home/shams/Desktop/OpenLane && \
    docker run --rm -v /home/shams/Desktop/OpenLane:/openlane -v /home/shams/
/home/shams/Desktop/OpenLane/designs:/openlane/install -v /home/shams:/home/shams -v /home/
shams/.ciel:/home/shams/.ciel -e PDK_ROOT=/home/shams/.ciel -e PDK=sky130A --us
er 1000:1000 -e DISPLAY=:0 -v /tmp/.X11-unix:/tmp/.X11-unix --network host --sec
urity-opt seccomp=unconfined -ti ghcr.io/the-openroad-project/openlane:ff5509f65
b17bfa4068d5336495ab1718987ff69-amd64
OpenLane Container (ff5509f):/openlane$
```

Fig - Moving into Docker using Command to run the design

```
OpenLane Container (ff5509f):/openlane$ ./flow.tcl -design spm
OpenLane v1.0.2 (ff5509f65b17bfa4068d5336495ab1718987ff69)
All rights reserved. (c) 2020-2025 Efabless Corporation and contributors.
Available under the Apache License, version 2.0. See the LICENSE file for more details.

[INFO]: Using configuration in 'designs/spm/config.json'...
[INFO]: PDK Root: /home/shams/.ciel
[INFO]: Process Design Kit: sky130A
[INFO]: Standard Cell Library: sky130_fd_sc_hd
[INFO]: Optimization Standard Cell Library: sky130_fd_sc_hd
[INFO]: Run Directory: /openlane/designs/spm/runs/RUN_2026.05.03_19.28.09
[INFO]: Saving runtime environment...
[INFO]: Preparing LEF files for the nom corner...
[INFO]: Preparing LEF files for the min corner...
[INFO]: Preparing LEF files for the max corner...
[INFO]: Running linter (Verilator) (log: designs/spm/runs/RUN_2026.05.03_19.28.09/logs/sy
[INFO]: 0 errors found by linter
[WARNING]: 1 warnings found by linter
[STEP 1]
[INFO]: Running Synthesis (log: designs/spm/runs/RUN_2026.05.03_19.28.09/logs/synthesis/1
[STEP 2]
[INFO]: Running Single-Corner Static Timing Analysis (log: designs/spm/runs/RUN_2026.05.0
.log)...
[STEP 3]
[INFO]: Running Initial Floorplanning (log: designs/spm/runs/RUN_2026.05.03_19.28.09/logs
[WARNING]: Current core area is too small for the power grid settings chosen. The power g
[INFO]: Floorplanned with width 90.62 and height 89.76.
[STEP 4]
[INFO]: Running IO Placement (log: designs/spm/runs/RUN_2026.05.03_19.28.09/logs/floorpla
[STEP 5]
[INFO]: Running Tap/Decap Insertion (log: designs/spm/runs/RUN_2026.05.03_19.28.09/logs/f
[INFO]: Power planning with power {VPWR} and ground {VGND}...
```



```
INFO]: Running LVS (log: designs/spm/runs/RUN_2026.05.03_19.28.09/logs/signoff/39-lvs.le
STEP 40]
INFO]: Running Magic DRC (log: designs/spm/runs/RUN_2026.05.03_19.28.09/logs/signoff/40-
INFO]: Converting Magic DRC database to various tool-readable formats...
INFO]: No DRC violations after GDS streaming out.
STEP 41]
INFO]: Running OpenROAD Antenna Rule Checker (log: designs/spm/runs/RUN_2026.05.03_19.28
STEP 42]
INFO]: Running Circuit Validity Checker ERC (log: designs/spm/runs/RUN_2026.05.03_19.28.
og)...
INFO]: Saving current set of views in 'designs/spm/runs/RUN_2026.05.03_19.28.09/results/
INFO]: Saving runtime environment...
INFO]: Generating final set of reports...
INFO]: Created manufacturability report at 'designs/spm/runs/RUN_2026.05.03_19.28.09/rep
Trash]: Created metrics report at 'designs/spm/runs/RUN_2026.05.03_19.28.09/reports/metri
WARNING]: There are max fanout violations in the design at the typical corner. Please re
26.05.03_19.28.09/reports/signoff/31-rcx_sta.checks.rpt'.
INFO]: There are no hold violations in the design at the typical corner.
INFO]: There are no setup violations in the design at the typical corner.
SUCCESS]: Flow complete.
INFO]: Note that the following warnings have been generated:
WARNING]: 1 warnings found by linter
WARNING]: Current core area is too small for the power grid settings chosen. The power g
WARNING]: Module sky130_fd_sc_hd__tapvpwrvgnd_1 blackboxed during sta
WARNING]: Module sky130_ef_sc_hd__decap_12 blackboxed during sta
WARNING]: Module sky130_fd_sc_hd__fill_1 blackboxed during sta
WARNING]: Module sky130_fd_sc_hd__fill_2 blackboxed during sta
WARNING]: VSRC_LOC_FILES is not defined. The IR drop analysis will run, but the values m
WARNING]: There are max fanout violations in the design at the typical corner. Please re
26.05.03_19.28.09/reports/signoff/31-rcx_sta.checks.rpt'.
```

Fig: - Showing a run of a sample design flow

```
OpenLane Container (ff5509f):/openlane$ cd designs
OpenLane Container (ff5509f):/openlane/designs$ ls
ci  spm
OpenLane Container (ff5509f):/openlane/designs$ cd spm
OpenLane Container (ff5509f):/openlane/designs/spm$ ls
config.json  pin_order.cfg  runs  src
OpenLane Container (ff5509f):/openlane/designs/spm$ cd runs
OpenLane Container (ff5509f):/openlane/designs/spm/runs$ ls
openlane_test          RUN_2026.04.30_16.18.57  RUN_2026.04.30_16.24.18
RUN_2026.04.30_16.17.31  RUN_2026.04.30_16.20.05  RUN_2026.05.03_19.28.09
OpenLane Container (ff5509f):/openlane/designs/spm/runs$
```

Fig: - Output Result in the RUN destination File

The successful completion of the **OpenLane** run indicates that the entire digital design flow—from synthesis through floorplanning, placement, routing, and signoff checks—has executed correctly, producing a valid GDSII layout from the RTL description. Although several warnings were reported, they are typical for a small test design like *spm* and do not indicate critical failures. The max fanout violation suggests that a signal is driving more gates than recommended, while the core area warning



reflects a compact layout that required automatic adjustment of the power grid. The presence of blackboxed standard cells simply means certain library elements were not fully expanded during timing analysis, which is normal in this context. Additionally, the absence of VSRC location files implies that IR drop analysis results may be approximate. Overall, despite these minor warnings, the flow completed successfully and the generated layout is valid for analysis and learning purposes.

TCL/AUTOMATION TASK

Wire Length of the design –

```
shans@VLSI: ~/Desktop/OpenLane/designs/spn/runs$ cd openlane_test
shans@VLSI: ~/Desktop/OpenLane/designs/spn/runs/openlane_test$ ls
cmds.log      logs          PDK_SOURCES  runtime.yanl
config_in.tcl OPENLANE_COMMIT reports      tnp
config.tcl    openlane.log  results      warnings.log
shans@VLSI: ~/Desktop/OpenLane/designs/spn/runs/openlane_test$ cd reports
shans@VLSI: ~/Desktop/OpenLane/designs/spn/runs/openlane_test/reports$ ls
cts      manufacturability.rpt placement signoff
Floorplan metrics.csv      routing  synthesis
shans@VLSI: ~/Desktop/OpenLane/designs/spn/runs/openlane_test/reports$ cd routing
shans@VLSI: ~/Desktop/OpenLane/designs/spn/runs/openlane_test/reports/routing$ ls
16-rsz_design_sta.checks.rpt  18-rsz_timing_sta.summary.rpt
16-rsz_design_sta.max.rpt     21-grt_sta.checks.rpt
16-rsz_design_sta.min.rpt     21-grt_sta.max.rpt
16-rsz_design_sta.power.rpt   21-grt_sta.min.rpt
16-rsz_design_sta.skew.rpt    21-grt_sta.power.rpt
16-rsz_design_sta.summary.rpt 21-grt_sta.skew.rpt
18-rsz_timing_sta.checks.rpt  21-grt_sta.summary.rpt
18-rsz_timing_sta.max.rpt     24-wire_lengths.csv
18-rsz_timing_sta.min.rpt     drc.drc
18-rsz_timing_sta.power.rpt   drc.klayout.xml
18-rsz_timing_sta.skew.rpt
shans@VLSI: ~/Desktop/OpenLane/designs/spn/runs/openlane_test/reports/routing$ awk -F',' '{sum += $2} END {print sum}' 24-wire_lengths.csv
6716.82
shans@VLSI: ~/Desktop/OpenLane/designs/spn/runs/openlane_test/reports/routing$ grep -i clk 24-wire_lengths.csv | awk -F',' '{sum += $2} END {print sum}'
784.48
shans@VLSI: ~/Desktop/OpenLane/designs/spn/runs/openlane_test/reports/routing$ grep -iv clk 24-wire_lengths.csv | awk -F',' '{sum += $2} END {print sum}'
5932.34
shans@VLSI: ~/Desktop/OpenLane/designs/spn/runs/openlane_test/reports/routing$
```

Total Wire Length: 6716.82 μm

`awk -F',' '{sum += $2} END {print sum}' 24-wire_lengths.csv`

Clock Wire Length: 784.45 μm

`grep -i clk 24-wire_lengths.csv | awk -F',' '{sum += $2} END {print sum}'`

Data Wire Length: 5932.34 μm

`grep -iv clk 24-wire_lengths.csv | awk -F',' '{sum += $2} END {print sum}'`



Worst IR drop and the % degradation –

```
toolplan_metrics.csv Routing Synthesis
OpenLane Container (ff5509f):/openlane/designs/spm/runs/RUN_2026.05.04_10.07.38/
reports$ cd signoff
OpenLane Container (ff5509f):/openlane/designs/spm/runs/RUN_2026.05.04_10.07.38/
reports/signoff$ ls
26-mca          32-irdrop-VGND.rpt      drc.rpt
28-mca          32-irdrop-VPWR.rpt     drc.tcl
30-mca          35-xor.rpt             drc.tr
31-rcx_sta.checks.rpt 35-xor.xml             spice.feedback.txt
31-rcx_sta.max.rpt   39-spm.lvs.rpt         spm.rpt
31-rcx_sta.min.rpt   41-antenna_violators_pins.txt spm.rpt.debug.gz
31-rcx_sta.power.rpt 41-antenna_violators.rpt spm.rpt.error.gz
31-rcx_sta.skew.rpt  drc.klayout.xml
31-rcx_sta.summary.rpt drc.rdb
OpenLane Container (ff5509f):/openlane/designs/spm/runs/RUN_2026.05.04_10.07.38/
reports/signoff$ less 32-irdrop-VPWR.rpt
OpenLane Container (ff5509f):/openlane/designs/spm/runs/RUN_2026.05.04_10.07.38/
reports/signoff$ grep -i drop 32-irdrop-VPWR.rpt
OpenLane Container (ff5509f):/openlane/designs/spm/runs/RUN_2026.05.04_10.07.38/
reports/signoff$ less 32-irdrop-VPWR.rpt
```

```
Instance name, X location, Y location, Voltage
_329_, 84.81, 100.64, 1.79975
input20, 84.81, 100.64, 1.79975
FILLER_0_32_165, 84.81, 100.64, 1.79975
_425_, 84.81, 95.2, 1.79972
_489_, 84.81, 95.2, 1.79972
input21, 84.81, 95.2, 1.79972
input22, 84.81, 95.2, 1.79972
input23, 84.81, 95.2, 1.79972
FILLER_0_31_165, 84.81, 95.2, 1.79972
_327_, 84.81, 89.76, 1.79971
_424_, 84.81, 89.76, 1.79971
_492_, 84.81, 89.76, 1.79971
FILLER_0_28_161, 84.81, 89.76, 1.79971
FILLER_0_29_181, 84.81, 89.76, 1.79971
FILLER_0_30_173, 84.81, 89.76, 1.79971
_427_, 84.81, 84.32, 1.79973
_491_, 84.81, 84.32, 1.79973
FILLER_0_27_169, 84.81, 84.32, 1.79973
_333_, 84.81, 78.88, 1.79974
_426_, 84.81, 78.88, 1.79974
FILLER_0_25_169, 84.81, 78.88, 1.79974
_337_, 84.81, 73.44, 1.79977
32-irdrop-VPWR.rpt
```



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

The last value is the voltage at the point.

IR drop = $1.8V - 1.79961V$

= $0.00039 V$

% Degradation = $(0.00039 / 1.8) \times 100$

$\approx 0.0217 \%$

Top Cells receiving less voltage -

```
OpenLane Container (ff5509f):/openlane/designs/spm/runs/RUN_2026.05.04_10.07.38/  
reports/signoff$ sort -t ',' -k4 -n 32-irdrop-VPWR.rpt | head -10
```

```
Instance name, X location, Y location, Voltage  
_229_, 5.52, 35.36, 1.79947  
_451_, 5.52, 35.36, 1.79947  
clkbuf_3_0__f_clk, 5.52, 35.36, 1.79947  
PHY_16, 5.52, 35.36, 1.79947  
PHY_18, 5.52, 35.36, 1.79947  
_225_, 28.172, 40.8, 1.79955  
_467_, 28.172, 40.8, 1.79955  
clkbuf_3_1__f_clk, 28.172, 40.8, 1.79955
```

```
OpenLane Container (ff5509f):/openlane/designs/spm/runs/RUN_2026.05.04_10.07.38/  
reports/signoff$ sort -t ',' -k4 -nr 32-irdrop-VGND.rpt | head -10
```

```
PHY_36, 5.52, 59.84, 0.00048724  
PHY_34, 5.52, 59.84, 0.00048724  
_453_, 5.52, 59.84, 0.00048724  
FILLER_0_18_23, 12.832, 59.84, 0.000474945  
clkbuf_3_2__f_clk, 12.832, 59.84, 0.000474945  
_385_, 12.832, 59.84, 0.000474945  
_235_, 12.832, 59.84, 0.000474945  
PHY_20, 5.52, 38.08, 0.000422366  
PHY_18, 5.52, 38.08, 0.000422366  
hold42, 5.52, 38.08, 0.000422366
```



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

Table-1: Cells with Lowest VDD Voltage (VPWR)

Rank	Cell Name	Voltage (V)
1	229	1.79947
2	451	1.79947
3	clkbuf_3_0__f_clk	1.79947
4	PHY_16	1.79947
5	PHY_18	1.79947
6	225	1.79955
7	467	1.79955
8	clkbuf_3_1__f_clk	1.79955

Table-2: Cells with Worst VSS Voltage (VGND)

Rank	Cell Name	Voltage (V)
1	PHY_36	0.00048724
2	PHY_34	0.00048724
3	453	0.00048724
4	FILLER_0_18_23	0.000474945
5	clkbuf_3_2__f_clk	0.000474945
6	385	0.000474945
7	235	0.000474945
8	PHY_20	0.000422366
9	PHY_18	0.000422366



LEVELS OF LOGIC

1. Firstly, looked for the STA report files inside the current directory using 'find . -name "*sta.rpt" '

```
shams@VLSI:~/Desktop/Openlane/designs/spm/runs/openlane_test$ find . -name "*sta*.rpt"
./reports/placement/8-gpl_sta.max.rpt
./reports/placement/11-dpl_sta.power.rpt
./reports/placement/11-dpl_sta.max.rpt
./reports/placement/8-gpl_sta.checks.rpt
./reports/placement/11-dpl_sta.checks.rpt
./reports/placement/8-gpl_sta.skew.rpt
./reports/placement/11-dpl_sta.min.rpt
./reports/placement/8-gpl_sta.summary.rpt
./reports/placement/8-gpl_sta.min.rpt
./reports/placement/11-dpl_sta.skew.rpt
./reports/placement/11-dpl_sta.summary.rpt
./reports/placement/8-gpl_sta.power.rpt
./reports/signoff/30-mca/rcx_nom_sta.max.rpt
./reports/signoff/30-mca/rcx_nom_sta.summary.rpt
./reports/signoff/30-mca/rcx_nom_sta.skew.rpt
./reports/signoff/30-mca/rcx_nom_sta.min.rpt
./reports/signoff/30-mca/rcx_nom_sta.power.rpt
./reports/signoff/30-mca/rcx_nom_sta.checks.rpt
./reports/signoff/31-rcx_sta.power.rpt
./reports/signoff/31-rcx_sta.skew.rpt
./reports/signoff/31-rcx_sta.summary.rpt
./reports/signoff/28-mca/rcx_max_sta.checks.rpt
./reports/signoff/28-mca/rcx_max_sta.summary.rpt
./reports/signoff/28-mca/rcx_max_sta.power.rpt
./reports/signoff/28-mca/rcx_max_sta.min.rpt
./reports/signoff/28-mca/rcx_max_sta.max.rpt
./reports/signoff/28-mca/rcx_max_sta.skew.rpt
./reports/signoff/31-rcx_sta.min.rpt
./reports/signoff/31-rcx_sta.checks.rpt
./reports/signoff/31-rcx_sta.max.rpt
./reports/signoff/26-mca/rcx_min_sta.power.rpt
./reports/signoff/26-mca/rcx_min_sta.skew.rpt
./reports/signoff/26-mca/rcx_min_sta.checks.rpt
./reports/signoff/26-mca/rcx_min_sta.min.rpt
./reports/signoff/26-mca/rcx_min_sta.max.rpt
./reports/signoff/26-mca/rcx_min_sta.summary.rpt
./reports/synthesis/1-synthesis.AREA_0.stat.rpt
./reports/synthesis/2-syn_sta.max.rpt
./reports/synthesis/2-syn_sta.min.rpt
./reports/synthesis/2-syn_sta.power.rpt
./reports/synthesis/2-syn_sta.summary.rpt
./reports/synthesis/2-syn_sta.checks.rpt
./reports/synthesis/2-syn_sta.skew.rpt
./reports/routing/21-grt_sta.min.rpt
./reports/routing/16-rsz_design_sta.power.rpt
./reports/routing/18-rsz_timing_sta.skew.rpt
./reports/routing/16-rsz_design_sta.checks.rpt
./reports/routing/16-rsz_design_sta.skew.rpt
./reports/routing/16-rsz_design_sta.summary.rpt
./reports/routing/18-rsz_timing_sta.max.rpt
./reports/routing/18-rsz_timing_sta.min.rpt
./reports/routing/21-grt_sta.skew.rpt
./reports/routing/21-grt_sta.max.rpt
./reports/routing/18-rsz_timing_sta.summary.rpt
./reports/routing/16-rsz_design_sta.min.rpt
./reports/routing/21-grt_sta.power.rpt
./reports/routing/21-grt_sta.checks.rpt
./reports/routing/21-grt_sta.summary.rpt
./reports/routing/18-rsz_timing_sta.checks.rpt
./reports/routing/16-rsz_design_sta.max.rpt
./reports/routing/18-rsz_timing_sta.power.rpt
./reports/cts/13-cts_sta.power.rpt
./reports/cts/13-cts_sta.min.rpt
```



```
./reports/cts/13-cts_sta.min.rpt
./reports/cts/13-cts_sta.max.rpt
./reports/cts/13-cts_sta.skew.rpt
./reports/cts/13-cts_sta.summary.rpt
./reports/cts/13-cts_sta.checks.rpt
shams@VLSI:~/Desktop/OpenLane/designs/spm/runs/openlane_test$
```

2. Then looking for the **31-rcx_sta.max.rpt** because it is the final signoff maximum timing analysis report. It gives the most accurate timing information of the design.
3. After that using “**less reports/signoff/31-rcx_sta.max.rpt**” in the shell to open and read the STA report file interactively inside the terminal.

```
=====
report_checks -path_delay max (Setup)
=====
===== Typical Corner =====

Startpoint: _444_ (rising edge-triggered flip-flop clocked by clk)
Endpoint: p (output port clocked by clk)
Path Group: clk
Path Type: max

Fanout    Cap    Slew    Delay    Time    Description
-----
          0.00    0.00    clock clk (rise edge)
          0.00    0.00    clock source latency
          0.07    0.05    0.05 ^ clk (in)
          1    0.01          0.07    0.00    0.05 ^ clkbuf_0_clk/A (sky130_fd_sc_hd__clkbuf_16)
          8    0.08          0.10    0.19    0.25 ^ clkbuf_0_clk/X (sky130_fd_sc_hd__clkbuf_16)
          9    0.03          0.10    0.00    0.25 ^ clkbuf_3_0__f_clk/A (sky130_fd_sc_hd__clkbuf_16)
          9    0.03          0.05    0.16    0.41 ^ clkbuf_3_0__f_clk/X (sky130_fd_sc_hd__clkbuf_16)
          1    0.00          0.05    0.00    0.41 ^ _444_/CLK (sky130_fd_sc_hd__dfrrp_1)
          1    0.00          0.04    0.37    0.78 v _444_/Q (sky130_fd_sc_hd__dfrrp_1)
          1    0.03          0.04    0.00    0.78 v output35/A (sky130_fd_sc_hd__clkbuf_4)
          1    0.03          0.08    0.19    0.96 v output35/X (sky130_fd_sc_hd__clkbuf_4)
          1    0.03          0.08    0.00    0.96 v p (net)
          1    0.03          0.08    0.00    0.96 v p (out)
          1    0.03          0.08    0.00    0.96 v data arrival time

reports/signoff/31-rcx_sta.max.rpt
```

4. Using ‘**/Startpoint**’ command finding the starting point of the timing path.



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

Startpoint: _444_ (rising edge-triggered flip-flop clocked by clk)
 Endpoint: p (output port clocked by clk)
 Path Group: clk
 Path Type: max

Fanout	Cap	Slew	Delay	Time	Description
			0.00	0.00	clock clk (rise edge)
			0.00	0.00	clock source latency
		0.07	0.05	0.05	^ clk (in)
1	0.01				clk (net)
		0.07	0.00	0.05	^ clkbuf_0_clk/A (sky130_fd_sc_hd__clkbuf_16)
		0.10	0.19	0.25	^ clkbuf_0_clk/X (sky130_fd_sc_hd__clkbuf_16)
8	0.08				clknet_0_clk (net)
		0.10	0.00	0.25	^ clkbuf_3_0_f_clk/A (sky130_fd_sc_hd__clkbuf_16)
		0.05	0.16	0.41	^ clkbuf_3_0_f_clk/X (sky130_fd_sc_hd__clkbuf_16)
9	0.03				clknet_3_0_leaf_clk (net)
		0.05	0.00	0.41	^ _444_/CLK (sky130_fd_sc_hd__dfrrtp_1)
		0.04	0.37	0.78	v _444_/Q (sky130_fd_sc_hd__dfrrtp_1)
1	0.00				net35 (net)
		0.04	0.00	0.78	v output35/A (sky130_fd_sc_hd__clkbuf_4)
		0.08	0.19	0.96	v output35/X (sky130_fd_sc_hd__clkbuf_4)
1	0.03				p (net)
		0.08	0.00	0.96	v p (out)
				0.96	data arrival time
			10.00	10.00	clock clk (rise edge)
			0.00	10.00	clock network delay (propagated)
			-0.25	9.75	clock uncertainty
			0.00	9.75	clock reconvergence pessimism
			-2.00	7.75	output external delay

Starting Point (SP): _444_ (rising edge-triggered flip-flop clocked by clk)

5. Using '/Endpoint' command finding the end point of the timing path.

Endpoint: p (output port clocked by clk)
 Path Group: clk
 Path Type: max

Fanout	Cap	Slew	Delay	Time	Description
			0.00	0.00	clock clk (rise edge)
			0.00	0.00	clock source latency
		0.07	0.05	0.05	^ clk (in)
1	0.01				clk (net)
		0.07	0.00	0.05	^ clkbuf_0_clk/A (sky130_fd_sc_hd__clkbuf_16)
		0.10	0.19	0.25	^ clkbuf_0_clk/X (sky130_fd_sc_hd__clkbuf_16)
8	0.08				clknet_0_clk (net)
		0.10	0.00	0.25	^ clkbuf_3_0_f_clk/A (sky130_fd_sc_hd__clkbuf_16)
		0.05	0.16	0.41	^ clkbuf_3_0_f_clk/X (sky130_fd_sc_hd__clkbuf_16)
9	0.03				clknet_3_0_leaf_clk (net)
		0.05	0.00	0.41	^ _444_/CLK (sky130_fd_sc_hd__dfrrtp_1)
		0.04	0.37	0.78	v _444_/Q (sky130_fd_sc_hd__dfrrtp_1)
1	0.00				net35 (net)
		0.04	0.00	0.78	v output35/A (sky130_fd_sc_hd__clkbuf_4)
		0.08	0.19	0.96	v output35/X (sky130_fd_sc_hd__clkbuf_4)
1	0.03				p (net)
		0.08	0.00	0.96	v p (out)
				0.96	data arrival time
			10.00	10.00	clock clk (rise edge)
			0.00	10.00	clock network delay (propagated)
			-0.25	9.75	clock uncertainty
			0.00	9.75	clock reconvergence pessimism
			-2.00	7.75	output external delay
				7.75	data required time

End Point (EP): p (output port clocked by clk)



- Using '/slack' command finding the slack value of the timing path.

```
6.79 slack (MET)
```

Startpoint: y (input port clocked by clk)
Endpoint: _503_ (rising edge-triggered flip-flop clocked by clk)
Path Group: clk
Path Type: max

Fanout	Cap	Slew	Delay	Time	Description
			0.00	0.00	clock clk (rise edge)
			0.00	0.00	clock network delay (propagated)
			2.00	2.00	^ input external delay
		0.02	0.01	2.01	^ y (in)
1	0.00				y (net)
		0.02	0.00	2.01	^ input34/A (sky130_fd_sc_hd__clkbuf_1)
		0.04	0.08	2.09	^ input34/X (sky130_fd_sc_hd__clkbuf_1)
1	0.00				net34 (net)
		0.04	0.00	2.09	^ _206_/A (sky130_fd_sc_hd__clkbuf_4)
		0.15	0.22	2.31	^ _206_/X (sky130_fd_sc_hd__clkbuf_4)
10	0.05				_097_ (net)
		0.15	0.00	2.31	^ _207_/A (sky130_fd_sc_hd__buf_2)
		0.20	0.27	2.58	^ _207_/X (sky130_fd_sc_hd__buf_2)
10	0.04				_098_ (net)
		0.20	0.00	2.58	^ _316_/A (sky130_fd_sc_hd__clkbuf_4)
		0.15	0.28	2.86	^ _316_/X (sky130_fd_sc_hd__clkbuf_4)
10	0.05				_165_ (net)
		0.15	0.00	2.86	^ _364_/A1 (sky130_fd_sc_hd__a31o_1)
		0.06	0.17	3.03	^ _364_/X (sky130_fd_sc_hd__a31o_1)
					021 (net)
1	0.00				_503_/D (sky130_fd_sc_hd__dftrtp_1)
		0.06	0.00	3.03	

;

Slack Value: 6.79 ns (MET)

- Finding the level of logic, using -- awk '/Startpoint/{flag=1} /slack \\\(MET\\)|slack \\\(VIOLATED\\)/{print; exit} flag' reports/signoff/31-rcx_sta.max.rpt | grep -c "sky130_fd_sc"

```
shams@VLSI:~/Desktop/OpenLane/designs/spm/runs/openlane_test$ awk '/Startpoint/{flag=1} /
)/{print; exit} flag' reports/signoff/31-rcx_sta.max.rpt | grep -c "sky130_fd_sc"
8
shams@VLSI:~/Desktop/OpenLane/designs/spm/runs/openlane_test$
```

Therefore, the total lol == 8



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

ASSIGNMENT-2

The domain of the Project:

Physical Design (VLSI)

Team Mentors (and their designation):

Veeramani Sir (Staff Engineer -Synopsys)

Name: - SHAMS TANVEER

Period of the project

November 2025 to May 2026



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

Implementing a 4-bit Up/Down Counter using Verilog HDL and perform the complete OpenLane RTL-to-GDSII flow. The objective is to analyze timing, area, power, utilization, routing, IR drop, and maximum operating frequency.

A 4-bit Up/Down Counter increments or decrements its count value depending on the control signal.

Verilog Code: -

```
module updown_counter (  
  
    input clk,  
  
    input rst,  
  
    input up_down,  
  
    output reg [3:0] count  
  
);  
  
  
always @(posedge clk or posedge rst)  
  
begin  
  
    if(rst)  
        count <= 4'b0000;  
  
    else if(up_down)  
        count <= count + 1;  
  
    else  
        count <= count - 1;  
  
end  
  
endmodule
```

Commands Used: -

```
cd ~/Desktop/OpenLane/designs
```

```
mkdir updown_counter
```

```
cd updown_counter
```

```
mkdir src
```



```
{
  "DESIGN_NAME": "updown_counter",
  "VERILOG_FILES": "dir::src/updown_counter.v",
  "CLOCK_PORT": "clk",
  "CLOCK_PERIOD": 10,
  "FP_CORE_UTIL": 20,
  "PL_TARGET_DENSITY": 0.30
}
```

```
shams@VL51:~/Desktop/OpenLane$ make mount
cd /home/shams/Desktop/OpenLane && \
    docker run --rm -v /home/shams/Desktop/OpenLane:/openlane -v /home/shams/Desktop/
all -v /home/shams:/home/shams -v /home/shams/.ciel:/home/shams/.ciel -e PDK_ROOT=/home/s
er 1000:1000 -e DISPLAY=:0 -v /tmp/.X11-unix:/tmp/.X11-unix --network host --security-opt
/the-openroad-project/openlane:ff5509f65b17bfa4068d5336495ab1718987ff69-amd64
OpenLane Container (ff5509f):/openlane$ ./flow.tcl -design updown_counter
OpenLane v1.0.2 (ff5509f65b17bfa4068d5336495ab1718987ff69)
All rights reserved. (c) 2020-2025 Efabless Corporation and contributors.
Available under the Apache License, version 2.0. See the LICENSE file for more details.

[INFO]: Using configuration in 'designs/updown_counter/config.json'...
[INFO]: PDK Root: /home/shams/.ciel
[INFO]: Process Design Kit: sky130A
[INFO]: Standard Cell Library: sky130_fd_sc_hd
[INFO]: Optimization Standard Cell Library: sky130_fd_sc_hd
[INFO]: Run Directory: /openlane/designs/updown_counter/runs/RUN_2026.06.06_09.22.40
[INFO]: Saving runtime environment...
[INFO]: Preparing LEF files for the nom corner...
[INFO]: Preparing LEF files for the min corner...
[INFO]: Preparing LEF files for the max corner...
[INFO]: Running linter (Verilator) (log: designs/updown_counter/runs/RUN_2026.06.06_09.22
..
[INFO]: 0 errors found by linter
[INFO]: 0 warnings found by linter
[STEP 1]
[INFO]: Running Synthesis (log: designs/updown_counter/runs/RUN_2026.06.06_09.22.40/logs/
[STEP 2]
[INFO]: Running Single-Corner Static Timing Analysis (log: designs/updown_counter/runs/RU
thesis/2-sta.log)...
[STEP 3]
[INFO]: Running Initial Floorplanning (log: designs/updown_counter/runs/RUN_2026.06.06_09
fp.log)...
[WARNING]: Current core area is too small for the power grid settings chosen. The power g
[INFO]: Floorplanned with width 35.88 and height 35.36.
[STEP 4]
[INFO]: Running IO Placement...
[STEP 5]
[INFO]: Running Tap/Decap Insertion (log: designs/updown_counter/runs/RUN_2026.06.06_09.2
..
[INFO]: Power planning with power {VPWR} and ground {VGND}...
[STEP 6]
[INFO]: Generating PDN (log: designs/updown_counter/runs/RUN_2026.06.06_09.22.40/logs/fle
[STEP 7]
[INFO]: Running Global Placement (log: designs/updown_counter/runs/RUN_2026.06.06_09.22.4
..
[STEP 8]
[INFO]: Running Single-Corner Static Timing Analysis (log: designs/updown_counter/runs/RU
ement/8-gpl_sta.log)...
[STEP 9]
[INFO]: Running Placement Resizer Design Optimizations (log: designs/updown_counter/runs/
acement/9-resizer.log)...
[STEP 10]
[INFO]: Running Detailed Placement (log: designs/updown_counter/runs/RUN_2026.06.06_09.22
log)...
[STEP 11]
[INFO]: Running Single-Corner Static Timing Analysis (log: designs/updown_counter/runs/RU
ement/11-dpl_sta.log)...
[STEP 12]
[INFO]: Running Clock Tree Synthesis (log: designs/updown_counter/runs/RUN_2026.06.06_09.
[STEP 13]
[INFO]: Running Single-Corner Static Timing Analysis (log: designs/updown_counter/runs/RU
13-cta_sta.log)...
[STEP 14]
[INFO]: Running Placement Resizer Design Optimizations (log: designs/updown_counter/runs/
```



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

```
[INFO]: Running Placement Resizer Timing Optimizations (log: designs/updown_counter/runs/s/14-resizer.log)...\n[STEP 15]\n[INFO]: Running Global Routing Resizer Design Optimizations (log: designs/updown_counter/gs/routing/15-resizer_design.log)...\n[STEP 16]\n[INFO]: Running Single-Corner Static Timing Analysis (log: designs/updown_counter/runs/RUing/16-rsz_design_sta.log)...\n[STEP 17]\n[INFO]: Running Global Routing Resizer Timing Optimizations (log: designs/updown_counter/gs/routing/17-resizer_timing.log)...\n[STEP 18]\n[INFO]: Running Single-Corner Static Timing Analysis (log: designs/updown_counter/runs/RUing/18-rsz_timing_sta.log)...\n[STEP 19]\n[INFO]: Running Global Routing (log: designs/updown_counter/runs/RUN_2026.06.06_09.22.40/\n[INFO]: Starting OpenROAD Antenna Repair Iterations...\n[STEP 20]\n[INFO]: Writing Verilog (log: designs/updown_counter/runs/RUN_2026.06.06_09.22.40/logs/roog)...\n[STEP 21]\n[INFO]: Running Single-Corner Static Timing Analysis (log: designs/updown_counter/runs/RUing/21-grt_sta.log)...\n[STEP 22]\n[INFO]: Running Fill Insertion (log: designs/updown_counter/runs/RUN_2026.06.06_09.22.40/\n[STEP 23]\n[INFO]: Running Detailed Routing (log: designs/updown_counter/runs/RUN_2026.06.06_09.22.4\n...\n[INFO]: No DRC violations after detailed routing.\n[STEP 24]\n[INFO]: Checking Wire Lengths (log: designs/updown_counter/runs/RUN_2026.06.06_09.22.40/1\n)...\n\n[STEP 25]\n[INFO]: Running SPEF Extraction at the min process corner (log: designs/updown_counter/ru/signoff/25-parasitics_extraction.min.log)...\n[STEP 26]\n[INFO]: Running Multi-Corner Static Timing Analysis at the min process corner (log: desig.06.06_09.22.40/logs/signoff/26-rcx_mcsta.min.log)...\n[STEP 27]\n[INFO]: Running SPEF Extraction at the max process corner (log: designs/updown_counter/ru/signoff/27-parasitics_extraction.max.log)...\n[STEP 28]\n[INFO]: Running Multi-Corner Static Timing Analysis at the max process corner (log: desig.06.06_09.22.40/logs/signoff/28-rcx_mcsta.max.log)...\n[STEP 29]\n[INFO]: Running SPEF Extraction at the nom process corner (log: designs/updown_counter/ru/signoff/29-parasitics_extraction.nom.log)...\n[STEP 30]\n[INFO]: Running Multi-Corner Static Timing Analysis at the nom process corner (log: desig.06.06_09.22.40/logs/signoff/30-rcx_mcsta.nom.log)...\n[STEP 31]\n[INFO]: Running Single-Corner Static Timing Analysis at the nom process corner (log: desi6.06.06_09.22.40/logs/signoff/31-rcx_sta.log)...\n[WARNING]: Module sky130_fd_sc_hd__tapvpwrvgn1 blackboxed during sta\n[WARNING]: Module sky130_ef_sc_hd__decap_12 blackboxed during sta\n[WARNING]: Module sky130_fd_sc_hd__fill_1 blackboxed during sta\n[WARNING]: Module sky130_fd_sc_hd__fill_2 blackboxed during sta\n[STEP 32]\n[INFO]: Creating IR Drop Report (log: designs/updown_counter/runs/RUN_2026.06.06_09.22.40\n[WARNING]: VSRC_LOC_FILES is not defined. The IR drop analysis will run, but the values m\n[STEP 33]\n[INFO]: Running Magic to generate various views...\n[INFO]: Streaming out GDSII with Magic (log: designs/updown_counter/runs/RUN_2026.06.06_0og)...
```



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

```
[INFO]: Generating MAGLEF views...
[INFO]: Generating lef with Magic (/openlane/designs/updown_counter/runs/RUN_2026.06.06_09.22.40)...
[STEP 34]
[INFO]: Streaming out GDSII with KLayout (log: designs/updown_counter/runs/RUN_2026.06.06_09.22.40/klayout.log)...
[STEP 35]
[INFO]: Running XOR on the layouts using KLayout (log: designs/updown_counter/runs/RUN_2026.06.06_09.22.40/xor.log)...
[INFO]: No XOR differences between KLayout and Magic gds.
[STEP 36]
[INFO]: Running Magic Spice Export from LEF (log: designs/updown_counter/runs/RUN_2026.06.06_09.22.40/spice.log)...
[STEP 37]
[INFO]: Writing Powered Verilog (logs: designs/updown_counter/runs/RUN_2026.06.06_09.22.40/logs/signoff/37-write_powered_verilog.log, designs/updown_counter/runs/RUN_2026.06.06_09.22.40/logs/signoff/37-write_powered_verilog.log)...
[STEP 38]
[INFO]: Writing Verilog (log: designs/updown_counter/runs/RUN_2026.06.06_09.22.40/logs/signoff/38-write_verilog.log)...
[STEP 39]
[INFO]: Running LVS (log: designs/updown_counter/runs/RUN_2026.06.06_09.22.40/logs/signoff/39-lvs.log)...
[STEP 40]
[INFO]: Running Magic DRC (log: designs/updown_counter/runs/RUN_2026.06.06_09.22.40/logs/signoff/40-drc.log)...
[INFO]: Converting Magic DRC database to various tool-readable formats...
[INFO]: No DRC violations after GDS streaming out.
[STEP 41]
[INFO]: Running OpenROAD Antenna Rule Checker (log: designs/updown_counter/runs/RUN_2026.06.06_09.22.40/logs/signoff/41-arc.log)...
[STEP 42]
[INFO]: Running Circuit Validity Checker ERC (log: designs/updown_counter/runs/RUN_2026.06.06_09.22.40/logs/signoff/42-erc_screen.log)...
[INFO]: Saving current set of views in 'designs/updown_counter/runs/RUN_2026.06.06_09.22.40/views'...
```

```
[INFO]: Saving current set of views in 'designs/updown_counter/runs/RUN_2026.06.06_09.22.40/views'...
[INFO]: Saving runtime environment...
[INFO]: Generating final set of reports...
[INFO]: Created manufacturability report at 'designs/updown_counter/runs/RUN_2026.06.06_09.22.40/manufacturability.rpt'.
[INFO]: Created metrics report at 'designs/updown_counter/runs/RUN_2026.06.06_09.22.40/reports/metrics.rpt'.
[INFO]: There are no max slew, max fanout or max capacitance violations in the design at the typical corner.
[INFO]: There are no hold violations in the design at the typical corner.
[INFO]: There are no setup violations in the design at the typical corner.
[SUCCESS]: Flow complete.
[INFO]: Note that the following warnings have been generated:
[WARNING]: Current core area is too small for the power grid settings chosen. The power grid will be generated with a minimum width of 0.5um.
[WARNING]: Module sky130_fd_sc_hd__tapvpwrvgn1 blackboxed during sta
[WARNING]: Module sky130_ef_sc_hd__decap_12 blackboxed during sta
[WARNING]: Module sky130_fd_sc_hd__fill_1 blackboxed during sta
[WARNING]: Module sky130_fd_sc_hd__fill_2 blackboxed during sta
[WARNING]: VSRC_LOC_FILES is not defined. The IR drop analysis will run, but the values may be inaccurate.
```

OpenLane Container (ff5509f):/openlane\$



Figuring out Maximum Frequency

```
OpenLane Container (ff5509f):/openlane/designs/updown_counter/runs/RUN_2026.06.06_09.22.40$ ls
cmds.log      config.tcl    OPENLANE_COMMIT  PDK_SOURCES  results      tmp
config_in.tcl logs          openlane.log     reports       runtime.yaml warnings.log
OpenLane Container (ff5509f):/openlane/designs/updown_counter/runs/RUN_2026.06.06_09.22.40$ ls reports/signoff
26-mca          31-rcx_sta.skew.rpt      41-antenna_violators_pins.txt  spice.feedback.txt
28-mca          31-rcx_sta.summary.rpt   41-antenna_violators.rpt       updown_counter.rpt
30-mca          32-irdrop-VGND.rpt      drc.klayout.xml                updown_counter.rpt.debug.gz
31-rcx_sta.checks.rpt  32-irdrop-VPWR.rpt      drc.rdb                        updown_counter.rpt.error.gz
31-rcx_sta.max.rpt    35-xor.rpt              drc.rpt
31-rcx_sta.min.rpt    35-xor.xml              drc.tcl
31-rcx_sta.power.rpt  39-updown_counter.lvs.rpt drc.tr
OpenLane Container (ff5509f):/openlane/designs/updown_counter/runs/RUN_2026.06.06_09.22.40$ grep "worst slack" reports/signoff/31-rcx_sta.summary.rpt
worst slack 6.83
worst slack 0.32
```

Using command “ grep "worst slack" reports/signoff/31-rcx_sta.summary.rpt “ I had found the **worst slack**.

worst slack- 6.83

report_worst_slack -max (Setup)

worst slack 6.83

report_worst_slack -min (Hold)

worst slack 0.32

Since maximum frequency is determined by setup timing, not hold timing.

Maximum frequency depends on the longest path (critical path):

Critical Path Delay = Clock Period - Setup Slack

Clock Period = 10 ns

Setup Slack = 6.83 ns

Critical Path Delay = 10 - 6.83

= 3.17 ns

Fmax = 1 / 3.17 ns

= 315.46 MHz



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

Timing Verification Table:

Clock Period	Frequency	Slack	Result
10 ns	100 MHz	+6.83 ns	Pass
6 ns	166.67 MHz	+2.83 ns	Pass
5 ns	200 MHz	+1.83 ns	Pass
4 ns	250 MHz	+0.83 ns	Pass
3 ns	333.33 MHz	-0.17 ns	Fail

Therefore, the design can operate successfully up to approximately 315 MHz, while timing violations begin to appear when the clock period is reduced below the critical path delay of 3.17 ns.

```
shams@VLSTI:~/Desktop/OpenLane/designs/updown_counter/runs/RUN_2026.06.06_09.22.40
$ grep -i uncertainty config.tcl
set ::env(SYNTH_CLOCK_UNCERTAINTY) "0.25"
```

Uncertainty: - 0.25ns

```
shams@VLSTI:~/Desktop/OpenLane/designs/updown_counter/runs/RUN_2026.06.06_09.22.40
$ grep -i derate config.tcl
set ::env(SYNTH_TIMING_DERATE) "0.05"
```

Derate Value: - 0.05%



Worst Slack Value at Each STA Stage

```
shams@VLSI: ~/Desktop/OpenLane/designs/updown_counter/runs/RUN_2026.06.06_09.22.40
$ grep -R "worst slack" logs reports/signoff
logs/placement/7-global.log:[INFO GPL-0100] worst slack 6.95e-09
logs/placement/7-global.log:[INFO GPL-0100] worst slack 6.93e-09
logs/placement/7-global.log:[INFO GPL-0100] worst slack 6.93e-09
logs/placement/7-global.log:[INFO GPL-0100] worst slack 6.91e-09
logs/placement/7-global.log:[INFO GPL-0100] worst slack 6.91e-09
logs/placement/8-gpl_sta.log:worst slack 6.88
logs/placement/8-gpl_sta.log:worst slack 0.33
logs/placement/11-dpl_sta.log:worst slack 6.86
logs/placement/11-dpl_sta.log:worst slack 0.21
logs/signoff/26-rcx_mcsta.min.log:worst slack 6.01
logs/signoff/26-rcx_mcsta.min.log:worst slack 0.11
logs/signoff/31-rcx_sta.log:worst slack 6.83
logs/signoff/31-rcx_sta.log:worst slack 0.32
logs/signoff/28-rcx_mcsta.max.log:worst slack 5.98
logs/signoff/28-rcx_mcsta.max.log:worst slack 0.12
logs/signoff/30-rcx_mcsta.nom.log:worst slack 5.99
logs/signoff/30-rcx_mcsta.nom.log:worst slack 0.11
logs/synthesis/2-sta.log:worst slack 6.91
logs/synthesis/2-sta.log:worst slack 0.32
logs/routing/21-grt_sta.log:worst slack 6.89
logs/routing/21-grt_sta.log:worst slack 0.27
logs/routing/18-rsz_timing_sta.log:worst slack 6.83
logs/routing/18-rsz_timing_sta.log:worst slack 0.32
logs/routing/16-rsz_design_sta.log:worst slack 6.83
logs/routing/16-rsz_design_sta.log:worst slack 0.32
logs/cts/13-cts_sta.log:worst slack 6.83
logs/cts/13-cts_sta.log:worst slack 0.20
reports/signoff/30-mca/rcx_nom_sta.summary.rpt:worst slack 5.99
reports/signoff/30-mca/rcx_nom_sta.summary.rpt:worst slack 0.11
reports/signoff/31-rcx_sta.summary.rpt:worst slack 6.83
reports/signoff/31-rcx_sta.summary.rpt:worst slack 0.32
reports/signoff/28-mca/rcx_max_sta.summary.rpt:worst slack 5.98
reports/signoff/28-mca/rcx_max_sta.summary.rpt:worst slack 0.12
reports/signoff/26-mca/rcx_min_sta.summary.rpt:worst slack 6.01
reports/signoff/26-mca/rcx_min_sta.summary.rpt:worst slack 0.11
```

STA Stage	Setup Worst Slack (ns)	Hold Worst Slack (ns)
Synthesis STA	6.91	0.32
Global Placement STA	6.88	0.33
Detailed Placement STA	6.86	0.21
CTS STA	6.83	0.20
Routing Design Optimization STA	6.83	0.32
Routing Timing Optimization STA	6.83	0.32
Global Routing STA	6.89	0.27
Signoff STA	6.83	0.32



Additional Table for Signoff Multi-Corner STA

Corner	Setup Slack (ns)	Hold Slack (ns)
Min Corner	6.01	0.11
Nominal Corner	5.99	0.11
Max Corner	5.98	0.12
Final Signoff STA	6.83	0.32

Power Analysis at Different Design Stages

```
shans@VLSI:~/Desktop/OpenLane/designs/updown_counter/runs/RUN_2026.06.06_09.22.40$ for f in reports/
do
echo "==== $f ====="
grep "Total" $f
done
==== reports/cts/13-cts_sta.power.rpt =====
Group      Internal Switching Leakage      Total
Total      8.11e-05  2.78e-05  2.50e-10  1.09e-04 100.0%
==== reports/placement/11-dpl_sta.power.rpt =====
Group      Internal Switching Leakage      Total
Total      3.03e-05  1.85e-05  2.13e-10  4.88e-05 100.0%
==== reports/placement/8-gpl_sta.power.rpt =====
Group      Internal Switching Leakage      Total
Total      2.86e-05  2.00e-05  2.05e-10  4.86e-05 100.0%
==== reports/routing/16-rsz_design_sta.power.rpt =====
Group      Internal Switching Leakage      Total
Total      8.42e-05  3.03e-05  2.57e-10  1.15e-04 100.0%
==== reports/routing/18-rsz_timing_sta.power.rpt =====
Group      Internal Switching Leakage      Total
Total      8.42e-05  3.03e-05  2.57e-10  1.15e-04 100.0%
==== reports/routing/21-grt_sta.power.rpt =====
Group      Internal Switching Leakage      Total
Total      8.41e-05  2.49e-05  2.57e-10  1.09e-04 100.0%
==== reports/signoff/31-rcx_sta.power.rpt =====
Group      Internal Switching Leakage      Total
Total      8.43e-05  3.06e-05  3.74e-10  1.15e-04 100.0%
==== reports/synthesis/2-syn_sta.power.rpt =====
Group      Internal Switching Leakage      Total
shans@VLSI:~/Desktop/OpenLane/designs/updown_counter/runs/RUN_2026.06.06_09.22.40$ grep "Total" reports/
yn_sta.power.rpt
Group      Internal Switching Leakage      Total
Total      2.87e-05  1.87e-05  1.21e-10  4.74e-05 100.0%
```



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

Stage	Internal Power (W)	Switching Power (W)	Leakage Power (W)	Total Power (W)
Synthesis STA	2.87E-05	1.87E-05	1.21E-10	4.74E-05
Global Placement STA	2.86E-05	2.00E-05	2.05E-10	4.86E-05
Detailed Placement STA	3.03E-05	1.85E-05	2.13E-10	4.88E-05
CTS STA	8.11E-05	2.78E-05	2.50E-10	1.09E-04
Routing Design Optimization STA	8.42E-05	3.03E-05	2.57E-10	1.15E-04
Routing Timing Optimization STA	8.42E-05	3.03E-05	2.57E-10	1.15E-04
Global Routing STA	8.41E-05	2.49E-05	2.57E-10	1.09E-04
Signoff STA	8.43E-05	3.06E-05	3.74E-10	1.15E-04

The power consumption of the 4-bit Up/Down Counter increases from $4.74 \times 10^{-5} \text{ W}$ at synthesis to $1.15 \times 10^{-4} \text{ W}$ at signoff. The major increase occurs after Clock Tree Synthesis due to clock buffer insertion. Leakage power remains negligible, while internal power is the dominant contributor. The final signoff analysis confirms that the design is low-power and timing-clean.

Area of Design at Each Stage

```
shams@VLST:~/Desktop/OpenLane/designs/updown_counter/runs/RUN_2026.06.06_09.22.48$ grep -R "Design area" logs reports
logs/placement/8-gpl_sta.log:Design area 277 u^2 22% utilization.
logs/placement/11-dpl_sta.log:Design area 259 u^2 20% utilization.
logs/placement/9-resizer.log:Design area 259 u^2 20% utilization.
logs/routing/21-grt_sta.log:Design area 353 u^2 28% utilization.
logs/routing/15-resizer_design.log:Design area 353 u^2 28% utilization.
logs/routing/18-rsz_timing_sta.log:Design area 353 u^2 28% utilization.
logs/routing/16-rsz_design_sta.log:Design area 353 u^2 28% utilization.
logs/routing/17-resizer_timing.log:Design area 353 u^2 28% utilization.
logs/cts/13-cts_sta.log:Design area 334 u^2 26% utilization.
logs/cts/14-resizer.log:Design area 354 u^2 28% utilization.
```



Stage	Design Area (μm^2)
Synthesis	257.75
Global Placement	277
Detailed Placement	259
CTS	334
CTS Optimization	354
Routing Design Optimization	353
Routing Timing Optimization	353
Global Routing	353
Final Signoff	353

The area of the Up/Down Counter design increases gradually throughout the OpenLane flow due to optimization and clock tree insertion. The synthesized design starts with an area of **257.75 μm^2** . During placement and CTS, additional cells such as buffers are introduced, causing the area to increase. After routing and optimization, the design area stabilizes at **353 μm^2** . The final area indicates that the design successfully meets physical design requirements while maintaining efficient utilization and sufficient routing space.

Signal Route Length

```
shansgVLSI:~/Desktop/OpenLane/designs/updown_counter/runs/RUN_2025-05-06_09:22:41$ grep -R "wire length" logs/routing
logs/routing/23-detailed.log:Total wire length = 482 um.
logs/routing/23-detailed.log:Total wire length on LAYER li1 = 0 um.
logs/routing/23-detailed.log:Total wire length on LAYER net1 = 250 um.
logs/routing/23-detailed.log:Total wire length on LAYER net2 = 217 um.
logs/routing/23-detailed.log:Total wire length on LAYER net3 = 14 um.
logs/routing/23-detailed.log:Total wire length on LAYER net4 = 0 um.
logs/routing/23-detailed.log:Total wire length on LAYER net5 = 0 um.
logs/routing/23-detailed.log:Total wire length = 483 um.
logs/routing/23-detailed.log:Total wire length on LAYER li1 = 0 um.
logs/routing/23-detailed.log:Total wire length on LAYER net1 = 254 um.
logs/routing/23-detailed.log:Total wire length on LAYER net2 = 214 um.
logs/routing/23-detailed.log:Total wire length on LAYER net3 = 14 um.
logs/routing/23-detailed.log:Total wire length on LAYER net4 = 0 um.
logs/routing/23-detailed.log:Total wire length on LAYER net5 = 0 um.
logs/routing/23-detailed.log:Total wire length = 483 um.
logs/routing/23-detailed.log:Total wire length on LAYER li1 = 0 um.
logs/routing/23-detailed.log:Total wire length on LAYER net1 = 254 um.
logs/routing/23-detailed.log:Total wire length on LAYER net2 = 214 um.
logs/routing/23-detailed.log:Total wire length on LAYER net3 = 14 um.
logs/routing/23-detailed.log:Total wire length on LAYER net4 = 0 um.
logs/routing/23-detailed.log:Total wire length on LAYER net5 = 0 um.
logs/routing/15-resizer.design.log:[INFO: RSZ-0058] Using max wire length 4508um.
```

Total route length = 483 μm

The total routed wire length of the design is **483 μm** , which is much smaller than the resizer threshold of **4508 μm** . Therefore, no long-wire optimization was required, indicating that the design is compact and efficiently routed.



```
shams@VLSI:~/Desktop/OpenLane/designs/updown_counter/runs/RUN_2026.06.06_09.22.40$ grep -i clk reports/routing/24-wire_lengths.csv
clknet_1_1__leaf_clk,27.72
clknet_1_0__leaf_clk,22.88
clk,22.03
clknet_0_clk,15.68
```

Total Clock Route Length

$$27.72 + 22.88 + 22.03 + 15.68 = 88.31 \mu m$$

Total Signal Route Length

$$483 - 88.31 = 394.69 \mu m$$

OpenLane generates a file named 24-wire_lengths.csv containing the routed length of each net. All nets containing the keyword clk were extracted and summed to obtain the total clock route length. The signal route length was then calculated by subtracting the clock route length from the total wire length reported by OpenLane.

The total routed wire length of the design is **483 μm** . Out of this, **88.31 μm** is used by clock distribution networks and **394.69 μm** is used by signal routing.

Design Utilization at Different Stages

```
shams@VLSI:~/Desktop/OpenLane/designs/updown_counter/runs/RUN_2026.06.06_09.22.40$ grep -R "Design area" logs
logs/placement/8-gpl_sta.log:Design area 277 u^2 22% utilization.
logs/placement/11-dpl_sta.log:Design area 259 u^2 20% utilization.
logs/placement/9-resizer.log:Design area 259 u^2 20% utilization.
logs/routing/21-grt_sta.log:Design area 353 u^2 28% utilization.
logs/routing/15-resizer_design.log:Design area 353 u^2 28% utilization.
logs/routing/18-rsz_timing_sta.log:Design area 353 u^2 28% utilization.
logs/routing/16-rsz_design_sta.log:Design area 353 u^2 28% utilization.
logs/routing/17-resizer_timing.log:Design area 353 u^2 28% utilization.
logs/cts/13-cts_sta.log:Design area 334 u^2 26% utilization.
logs/cts/14-resizer.log:Design area 354 u^2 28% utilization.
```



Stage	Utilization (%)
Global Placement (GPL)	22
Detailed Placement (DPL)	20
Clock Tree Synthesis (CTS)	26
Post-CTS Optimization	28
Routing	28
Signoff (Final)	28

Floorplan Analysis

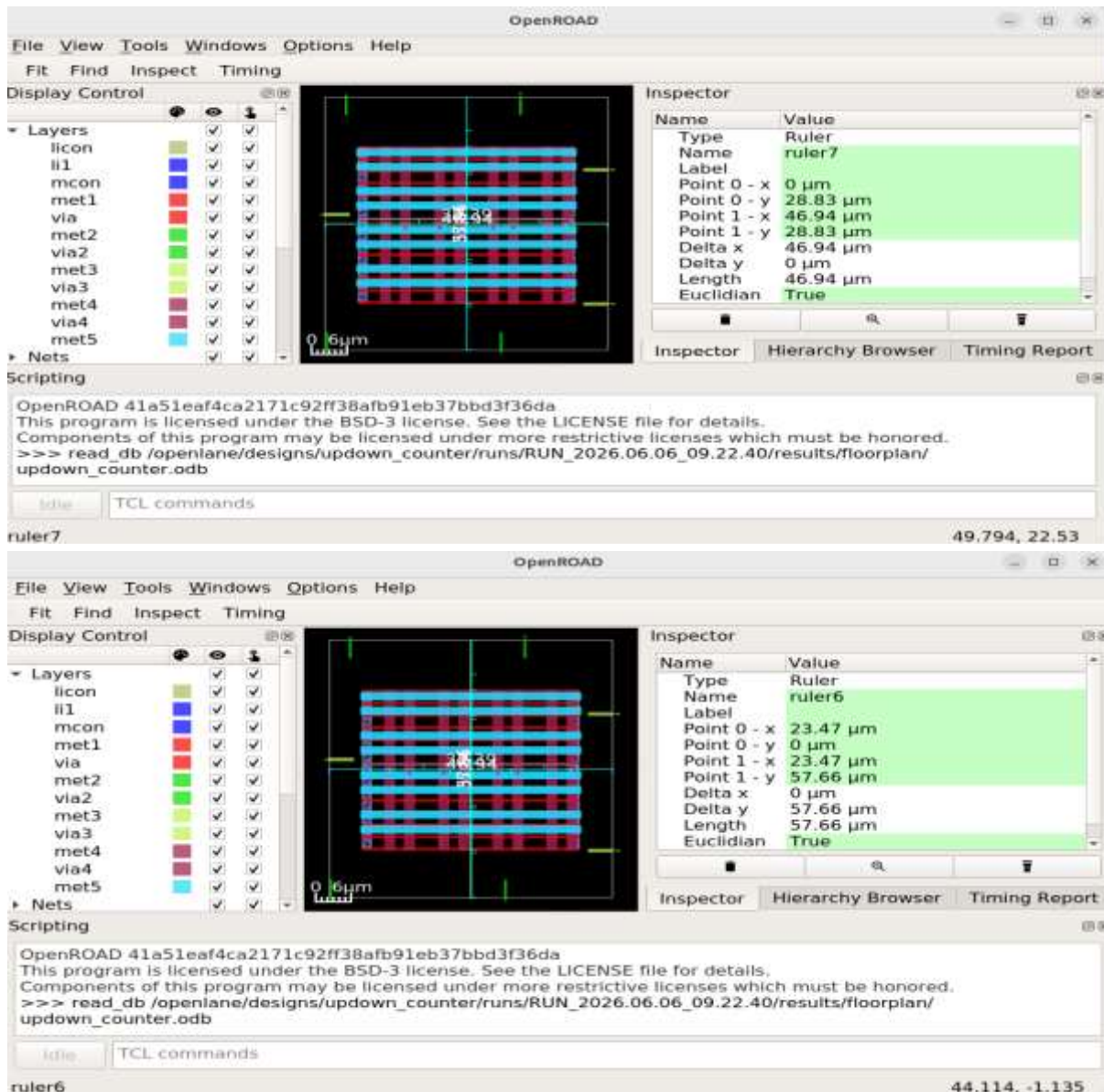
Floorplan Layout

The top screenshot shows the OpenROAD floorplan analysis tool. The main window displays a floorplan layout with various layers and nets. The 'Inspector' window on the right shows the details for the 'ruler3' measurement tool. The 'Scripting' window at the bottom shows the command 'read_db /openlane/designs/updown_counter/runs/RUN_2026.06.06_09.22.40/results/floorplan/updown_counter.oddb'.

Name	Value
Type	Ruler
Name	ruler3
Label	
Point 0 - x	5.605 μm
Point 0 - y	29.834 μm
Point 1 - x	40.995 μm
Point 1 - y	29.834 μm
Delta x	35.39 μm
Delta y	0 μm
Length	35.39 μm

The bottom screenshot shows the OpenROAD floorplan analysis tool. The main window displays a floorplan layout with various layers and nets. The 'Inspector' window on the right shows the details for the 'ruler4' measurement tool. The 'Scripting' window at the bottom shows the command 'read_db /openlane/designs/updown_counter/runs/RUN_2026.06.06_09.22.40/results/floorplan/updown_counter.oddb'.

Name	Value
Type	Ruler
Name	ruler4
Label	
Point 0 - x	23.46 μm
Point 0 - y	10.64 μm
Point 1 - x	23.46 μm
Point 1 - y	46.48 μm
Delta x	0 μm
Delta y	35.84 μm
Length	35.84 μm
Euclidian	True



Description: The floorplan of the up-down counter design was analyzed using the OpenROAD GUI. The floorplan stage defines the die area, core area, placement rows, power distribution network, and physical-only cells required for fabrication. The layout was inspected using the GUI, and measurements of the core and die dimensions were obtained using the Ruler tool.

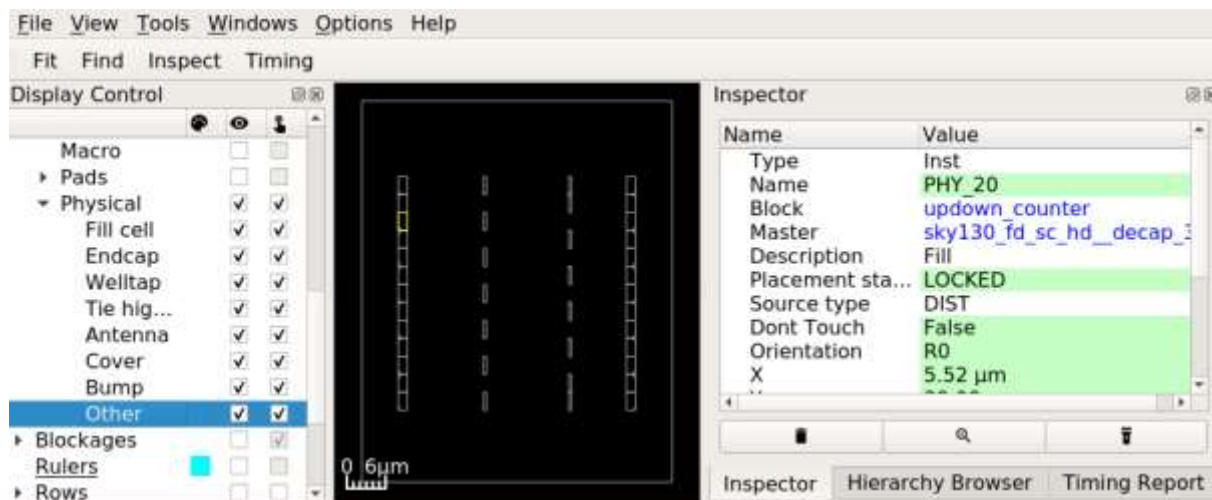
Parameter	Value (μm)
Core Width	35.39
Core Height	35.84
Die Width	46.94
Die Height	57.66



The measured core dimensions are $35.39\ \mu\text{m} \times 35.84\ \mu\text{m}$, providing the placement region for standard cells. The overall die dimensions are $46.94\ \mu\text{m} \times 57.66\ \mu\text{m}$, which include the core area, power network structures, and boundary spacing required for fabrication. The die area is larger than the core area to accommodate routing resources and physical-only cells.

Physical-Only Cells at Floorplan Stage

Physical-only cells were visualized using the OpenROAD GUI by enabling the Physical cell display option. These cells do not perform logical operations but are inserted to improve manufacturability, reliability, and power integrity.



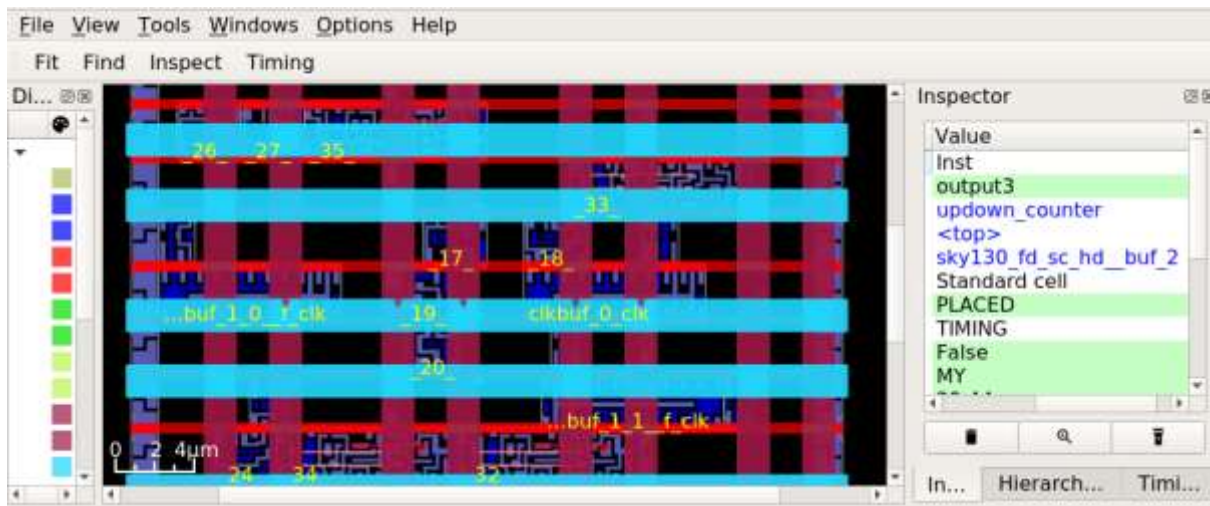
Physical Cell Type	Purpose
Fill Cell	Fills gaps between standard cells and maintains well continuity
Endcap Cell	Protects row boundaries and satisfies design rules
Well Tap Cell	Prevents latch-up by connecting wells to power rails
Decap Cell	Improves power integrity and reduces voltage fluctuations
Antenna Cell	Prevents antenna violations during fabrication

The OpenROAD GUI shows the presence of fill cells, endcap cells, well tap cells, and decap cells. These cells are inserted automatically during physical implementation to ensure reliable chip fabrication and robust power distribution.



Placement Analysis

Buffer: -

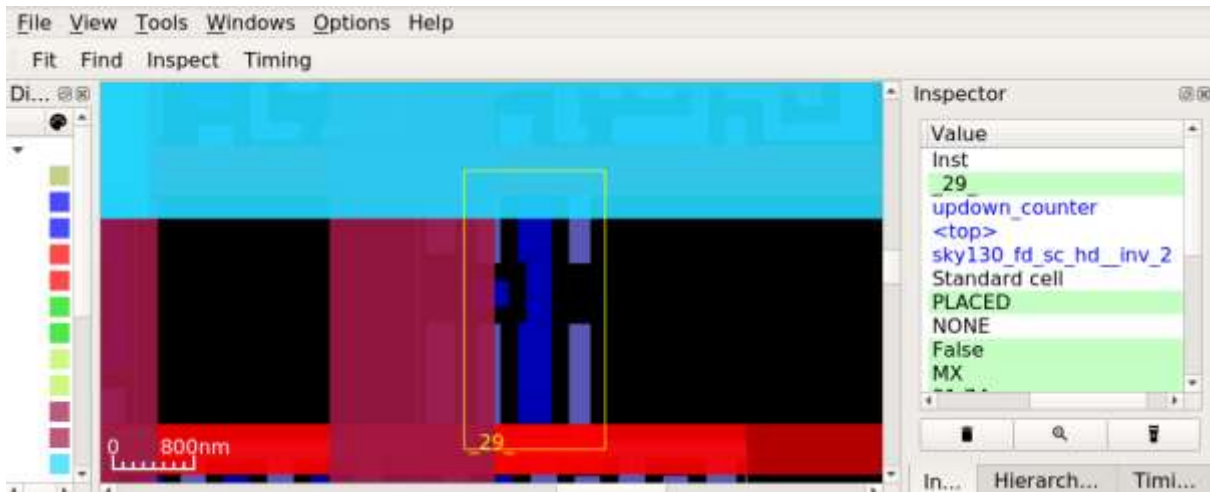


Buffer Cells Identified

Instance Name	Cell Type
hold1	sky130_fd_sc_hd__buf_1
clkbuf_0_clk	sky130_fd_sc_hd__clkbuf_16
clkbuf_1_0__f_clk	sky130_fd_sc_hd__clkbuf_16
clkbuf_1_1__f_clk	sky130_fd_sc_hd__clkbuf_16



Inverter: -



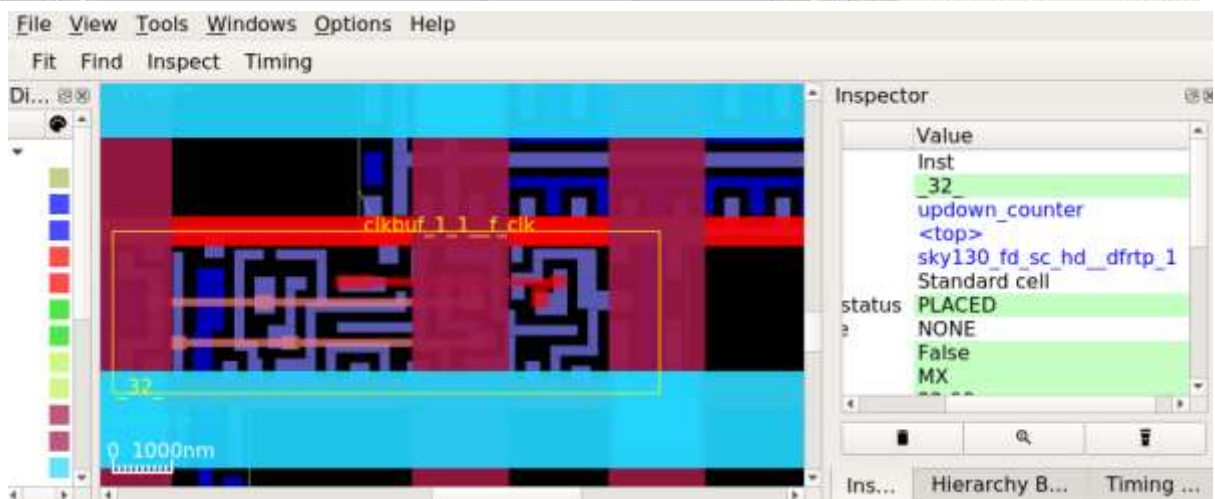
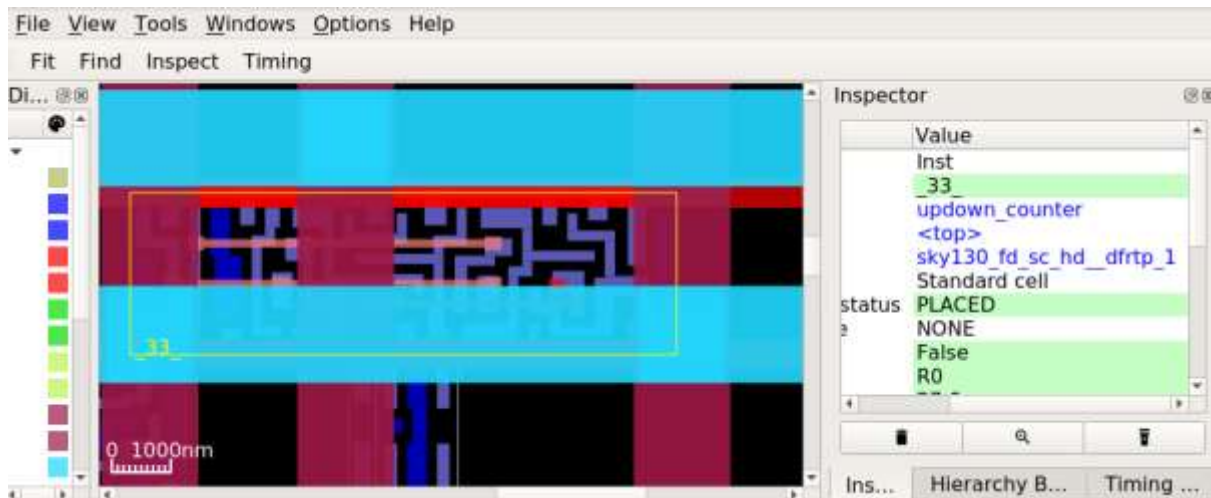
Inverter Cells Identified

Instance Name	Cell Type
16	sky130_fd_sc_hd__inv_2
28	sky130_fd_sc_hd__inv_2
29	sky130_fd_sc_hd__inv_2
30	sky130_fd_sc_hd__inv_2
31	sky130_fd_sc_hd__inv_2

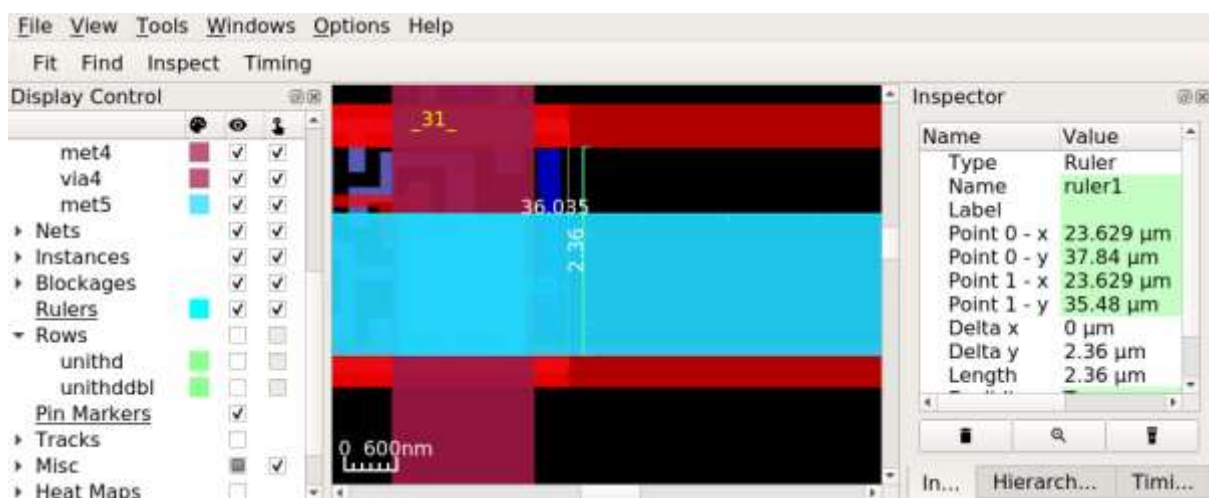


Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

Flops: -



Site Row: -





Site Row Measurement

The site row dimensions were measured using the OpenROAD GUI ruler tool during the placement stage.

Measurement	Value (μm)
Site Row Width	36.035
Site Row Height	2.36

Different library cell Height and Width

1. Xnor

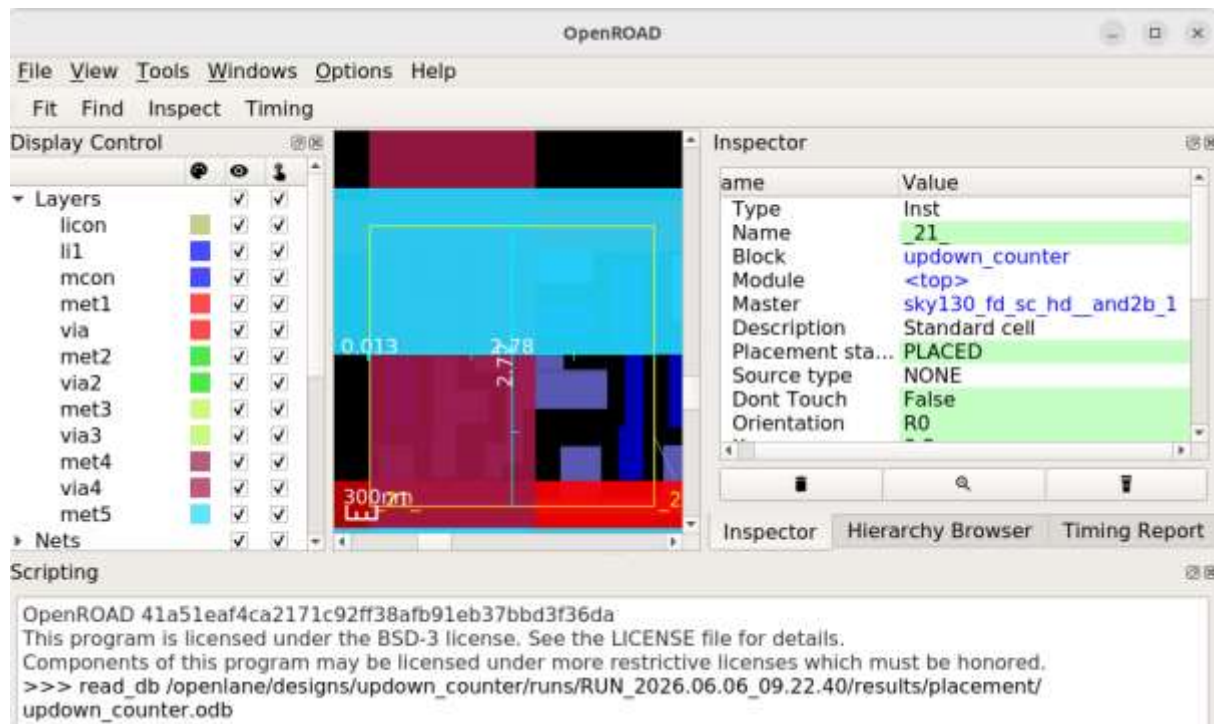


Height = 2.804 μm

Width = 3.022 μm



2. And



Height = 2.78 μm

Width = 2.72 μm

CTS Analysis: -

Synthesis Netlist

Contains only:

sky130_fd_sc_hd__inv_2_16_
sky130_fd_sc_hd__inv_2_28_
sky130_fd_sc_hd__inv_2_29_
sky130_fd_sc_hd__inv_2_30_
sky130_fd_sc_hd__inv_2_31_

```
shams@VLSI:~/Desktop/OpenLane/designs/updown_counter/runs/RUN_2026.06.06_09.22.40$ grep -i "buf\|inv" results/synthesis/*.v
sky130_fd_sc_hd__inv_2_16_ (
sky130_fd_sc_hd__inv_2_28_ (
sky130_fd_sc_hd__inv_2_29_ (
sky130_fd_sc_hd__inv_2_30_ (
sky130_fd_sc_hd__inv_2_31_ (
```



Final Netlist

Contains:

sky130_fd_sc_hd__inv_2_16_
sky130_fd_sc_hd__inv_2_28_
sky130_fd_sc_hd__inv_2_29_
sky130_fd_sc_hd__inv_2_30_
sky130_fd_sc_hd__inv_2_31_

sky130_fd_sc_hd__buf_1 hold1

sky130_fd_sc_hd__clkbuf_16 clkbuf_0_clk
sky130_fd_sc_hd__clkbuf_16 clkbuf_1_0__f_clk
sky130_fd_sc_hd__clkbuf_16 clkbuf_1_1__f_clk

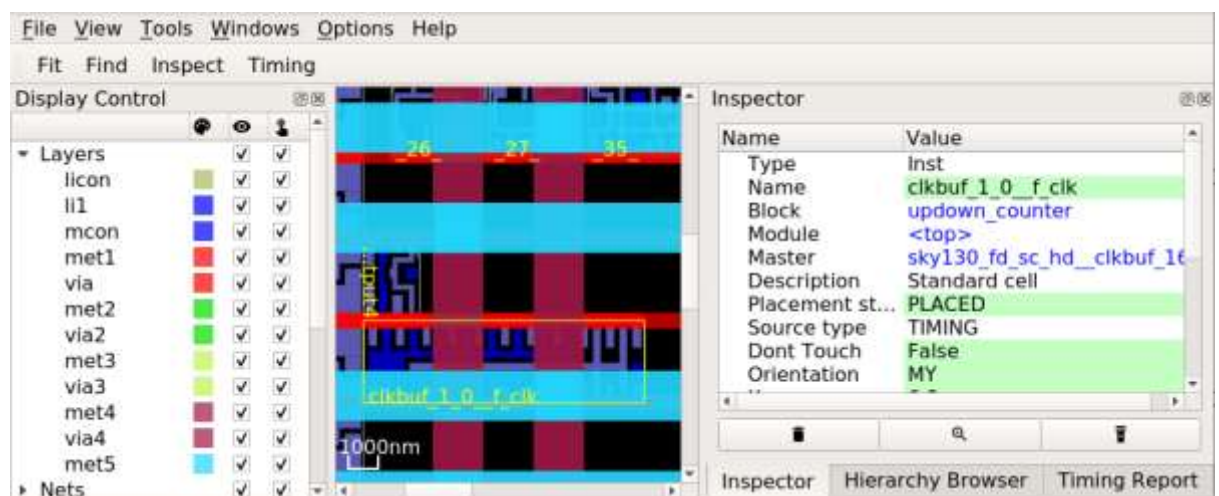
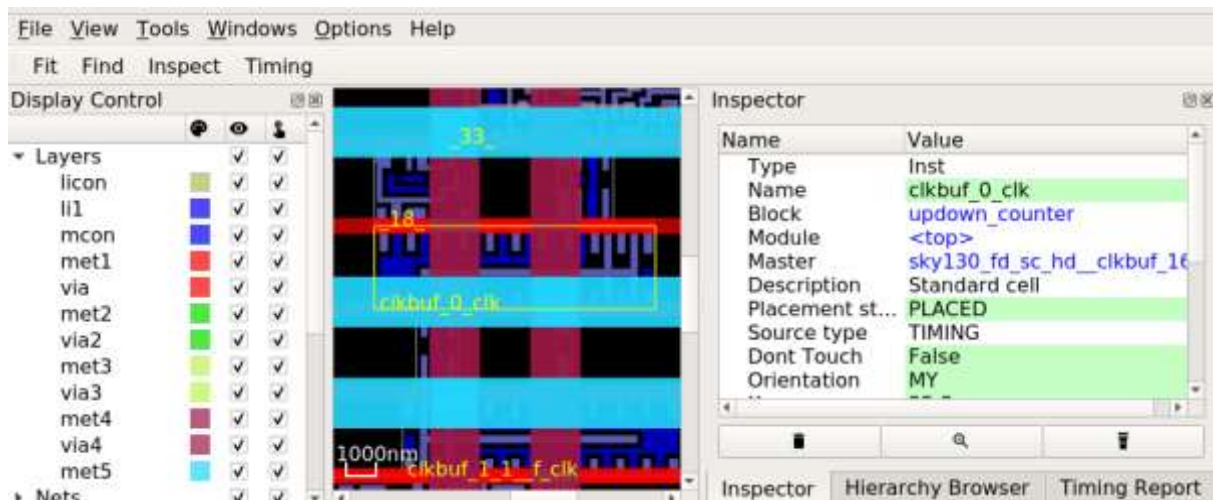
sky130_fd_sc_hd__clkbuf_2 input1
sky130_fd_sc_hd__clkbuf_2 input2

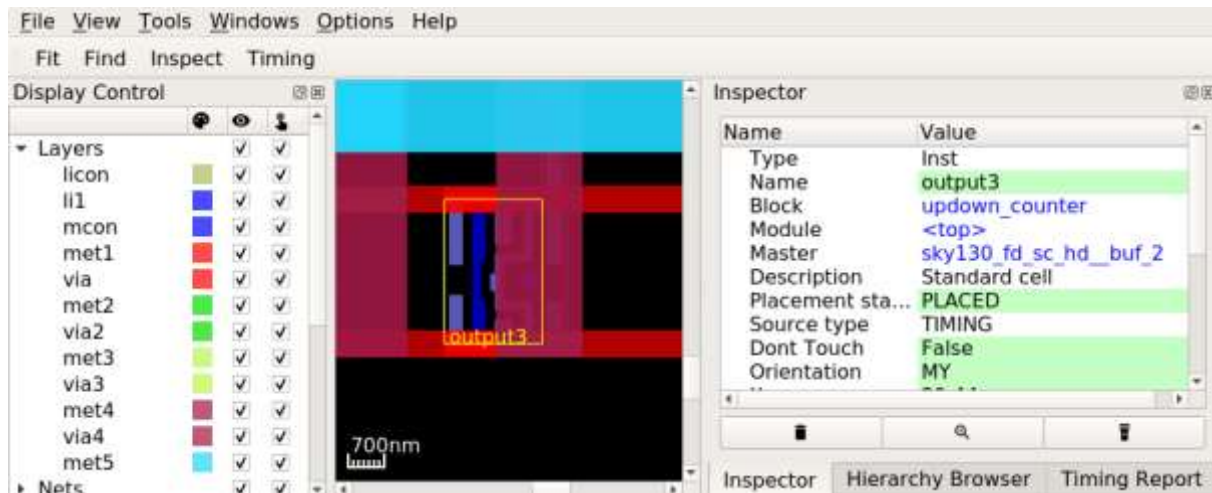
sky130_fd_sc_hd__clkbuf_4 output3
sky130_fd_sc_hd__clkbuf_4 output4
sky130_fd_sc_hd__clkbuf_4 output5
sky130_fd_sc_hd__clkbuf_4 output6

```
shams@VLSI: ~/Desktop/OpenLane/designs/updown_counter/runs/RUN_2026.06.06_09.22.4
0$ grep -i "buf\\|inv" results/final/verilog/gl/updown_counter.v
sky130_fd_sc_hd__inv_2_16_ (.A(net7),
sky130_fd_sc_hd__inv_2_28_ (.A(net1),
sky130_fd_sc_hd__inv_2_29_ (.A(net1),
sky130_fd_sc_hd__inv_2_30_ (.A(net1),
sky130_fd_sc_hd__inv_2_31_ (.A(net1),
sky130_fd_sc_hd__clkbuf_16 clkbuf_0_clk (.A(clk),
sky130_fd_sc_hd__clkbuf_16 clkbuf_1_0__f_clk (.A(clknet_0_clk),
sky130_fd_sc_hd__clkbuf_16 clkbuf_1_1__f_clk (.A(clknet_0_clk),
sky130_fd_sc_hd__buf_1 hold1 (.A(net3),
sky130_fd_sc_hd__clkbuf_2 input1 (.A(rst),
sky130_fd_sc_hd__clkbuf_2 input2 (.A(up_down),
sky130_fd_sc_hd__clkbuf_4 output3 (.A(net3),
sky130_fd_sc_hd__clkbuf_4 output4 (.A(net4),
sky130_fd_sc_hd__clkbuf_4 output5 (.A(net5),
sky130_fd_sc_hd__clkbuf_4 output6 (.A(net6),
```

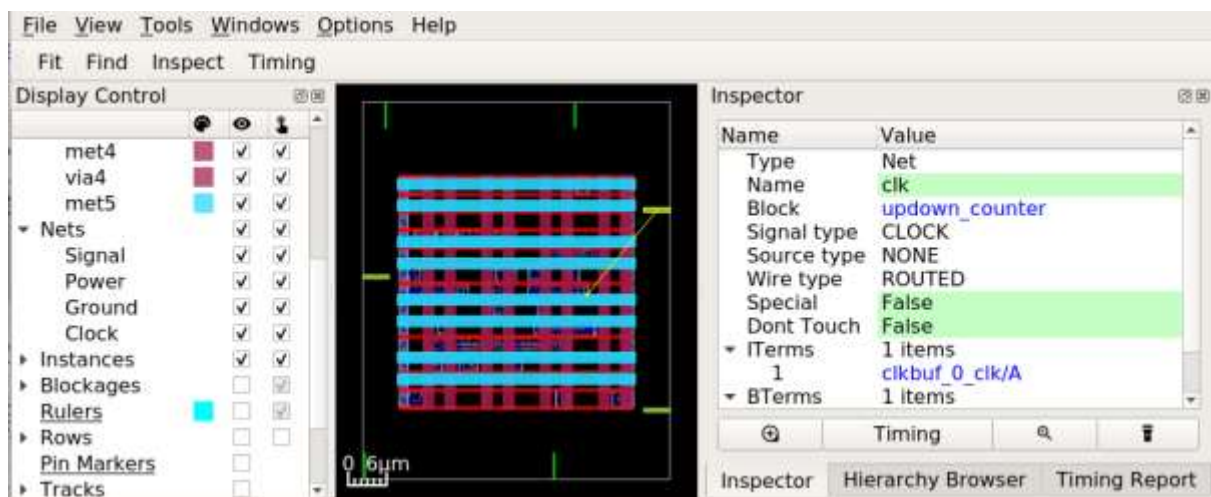


Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

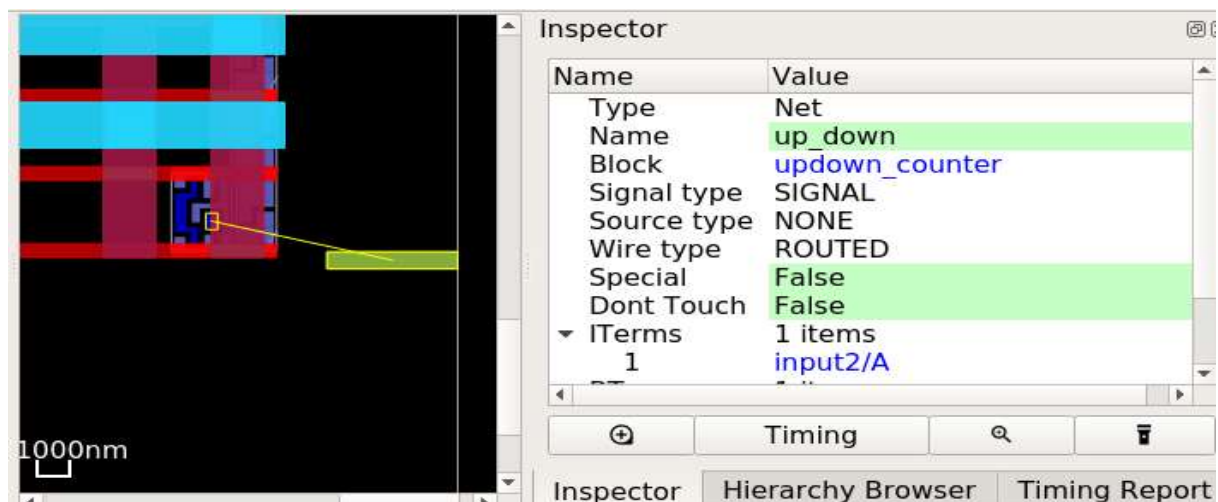


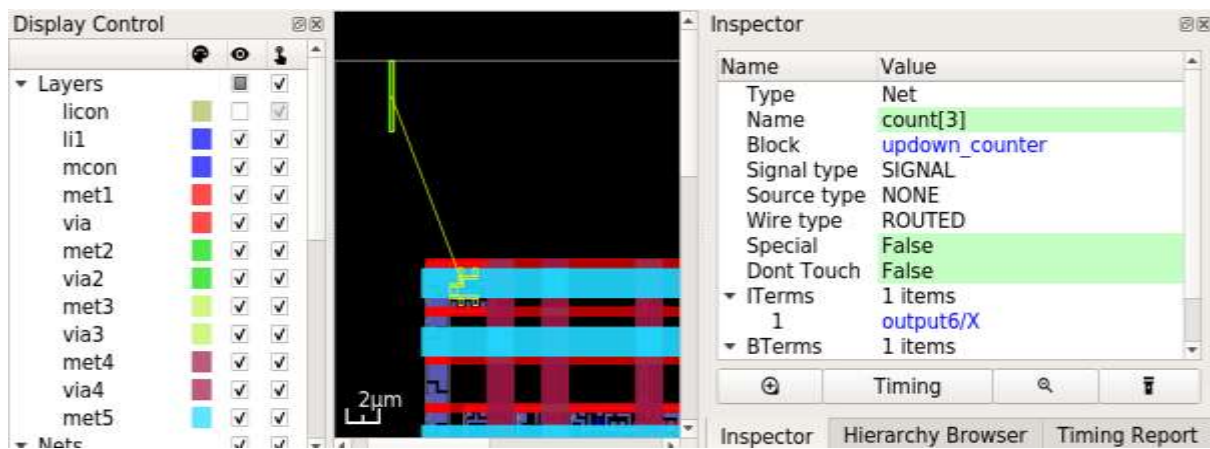


Clock Routing and its metal layer: -



Signal Routing and its metal layer: -





Newly added buffer/inverter during routing stage: -

Earlier,

```
shams@VLSI:~/Desktop/OpenLane/designs/updown_counter/runs/RUN_2026.06.06_09.22.4
0$ grep -i "buf\\|inv" results/synthesis/*.v
sky130_fd_sc_hd__inv_2_16_ (
sky130_fd_sc_hd__inv_2_28_ (
sky130_fd_sc_hd__inv_2_29_ (
sky130_fd_sc_hd__inv_2_30_ (
sky130_fd_sc_hd__inv_2_31_ (
```

Only 5 inverters are present during synthesis stage.

After synthesis,

```
shams@VLSI:~/Desktop/OpenLane/designs/updown_counter/runs/RUN_2026.06.06_09.22.4
0$ grep -i "buf\\|inv" results/final/verilog/gl/updown_counter.v
sky130_fd_sc_hd__inv_2_16_ (.A(net7),
sky130_fd_sc_hd__inv_2_28_ (.A(net1),
sky130_fd_sc_hd__inv_2_29_ (.A(net1),
sky130_fd_sc_hd__inv_2_30_ (.A(net1),
sky130_fd_sc_hd__inv_2_31_ (.A(net1),
sky130_fd_sc_hd__clkbuf_16 clkbuf_0_clk (.A(clk),
sky130_fd_sc_hd__clkbuf_16 clkbuf_1_0__f_clk (.A(clknet_0_clk),
sky130_fd_sc_hd__clkbuf_16 clkbuf_1_1__f_clk (.A(clknet_0_clk),
sky130_fd_sc_hd__buf_1 hold1 (.A(net3),
sky130_fd_sc_hd__clkbuf_2 input1 (.A(rst),
sky130_fd_sc_hd__clkbuf_2 input2 (.A(up_down),
sky130_fd_sc_hd__clkbuf_4 output3 (.A(net3),
sky130_fd_sc_hd__clkbuf_4 output4 (.A(net4),
sky130_fd_sc_hd__clkbuf_4 output5 (.A(net5),
sky130_fd_sc_hd__clkbuf_4 output6 (.A(net6),
```

Routing Stage inserted 7 new buffers and 0 inverters.



32-bit Up-Down Counter

```
shams@VLSI:~/Desktop$ cd OpenLane/designs
shams@VLSI:~/Desktop/OpenLane/designs$ mkdir updown_counter_32
shams@VLSI:~/Desktop/OpenLane/designs$ cd updown_counter_32
shams@VLSI:~/Desktop/OpenLane/designs/updown_counter_32$ mkdir src
shams@VLSI:~/Desktop/OpenLane/designs/updown_counter_32$ ls
src
shams@VLSI:~/Desktop/OpenLane/designs/updown_counter_32$ nano src/updown_counter_32.v
```

```
GNU nano 7.2 src/updown_counter_32.v *
module updown_counter_32 (
    input clk,
    input rst,
    input up_down,
    output reg [31:0] count
);

always @(posedge clk or posedge rst)
begin
    if (rst)
        count <= 32'd0;
    else if (up_down)
        count <= count + 1'b1;
    else
        count <= count - 1'b1;
end

endmodule

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```

```
shams@VLSI:~/Desktop$ cd OpenLane/designs
shams@VLSI:~/Desktop/OpenLane/designs$ mkdir updown_counter_32
shams@VLSI:~/Desktop/OpenLane/designs$ cd updown_counter_32
shams@VLSI:~/Desktop/OpenLane/designs/updown_counter_32$ mkdir src
shams@VLSI:~/Desktop/OpenLane/designs/updown_counter_32$ ls
src
shams@VLSI:~/Desktop/OpenLane/designs/updown_counter_32$ nano src/updown_counter_32.v
shams@VLSI:~/Desktop/OpenLane/designs/updown_counter_32$ nano src/updown_counter_32.v
shams@VLSI:~/Desktop/OpenLane/designs/updown_counter_32$ nano config.json
```



```
shams@VLSI: ~/Desktop/OpenLane/designs/updown_counter_32
GNU nano 7.2 config.json

"DESIGN_NAME": "updown_counter_32",
"VERILOG_FILES": "dir::src/updown_counter_32.v",
"CLOCK_PORT": "clk",
"CLOCK_PERIOD": 6,
"FP_CORE_UTIL": 70,
"FP_PDN_CORE_RING": true,
"CTS_TARGET_SKEW": 0.3,
"FP_TAPCELL_DIST": 14

[ Read 10 lines ]
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line

shams@VLSI:~/Desktop/OpenLane/designs/updown_counter_32$ nano config.json
shams@VLSI:~/Desktop/OpenLane/designs/updown_counter_32$ nano config.json
shams@VLSI:~/Desktop/OpenLane/designs/updown_counter_32$ nano src/base.sdc
```

```
shams@VLSI: ~/Desktop/OpenLane/designs/updown_counter_32
GNU nano 7.2 src/base.sdc *
create_clock -name clk -period 6 [get_ports clk]

set_clock_uncertainty 0.6 [get_clocks clk]

set_timing_derate -early 0.97
set_timing_derate -late 1.03

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line
```



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

```
shams@VLSI:~/Desktop/OpenLane$ make mount
cd /home/shams/Desktop/OpenLane && \
    docker run --rm -v /home/shams/Desktop/OpenLane:/openlane -v /home/shams/
/Desktop/OpenLane/designs:/openlane/install -v /home/shams:/home/shams -v /home/
shams/.ciel:/home/shams/.ciel -e PDK_ROOT=/home/shams/.ciel -e PDK=sky130A --us
er 1000:1000 -e DISPLAY=:0 -v /tmp/.X11-unix:/tmp/.X11-unix --network host --sec
urity-opt seccomp=unconfined -ti ghcr.io/the-openroad-project/openlane:ff5509f65
b17bfa4068d5336495ab1718987ff69-amd64
OpenLane Container (ff5509f):/openlane$ ./flow.tcl -design updown_counter_32
OpenLane v1.0.2 (ff5509f65b17bfa4068d5336495ab1718987ff69)
All rights reserved. (c) 2020-2025 Efabless Corporation and contributors.
Available under the Apache License, version 2.0. See the LICENSE file for more d
etails.

[INFO]: Using configuration in 'designs/updown_counter_32/config.json'...
[INFO]: PDK Root: /home/shams/.ciel
[INFO]: Process Design Kit: sky130A
[INFO]: Standard Cell Library: sky130_fd_sc_hd
[INFO]: Optimization Standard Cell Library: sky130_fd_sc_hd
[INFO]: Run Directory: /openlane/designs/updown_counter_32/runs/RUN_2026.06.07_0
7.23.41
[INFO]: Saving runtime environment...
[INFO]: Preparing LEF files for the nom corner...
[INFO]: Preparing LEF files for the min corner...
```

```
shams@VLSI: ~/Desktop/OpenLane
[INFO]: Saving current set of views in 'designs/updown_counter_32/runs/RUN_2026.
06.07_07.23.41/results/final'...
[INFO]: Saving runtime environment...
[INFO]: Generating final set of reports...
[INFO]: Created manufacturability report at 'designs/updown_counter_32/runs/RUN_
2026.06.07_07.23.41/reports/manufacturability.rpt'.
[INFO]: Created metrics report at 'designs/updown_counter_32/runs/RUN_2026.06.07
_07.23.41/reports/metrics.csv'.
[INFO]: There are no max slew, max fanout or max capacitance violations in the d
esign at the typical corner.
[INFO]: There are no hold violations in the design at the typical corner.
[INFO]: There are no setup violations in the design at the typical corner.
[SUCCESS]: Flow complete.
[INFO]: Note that the following warnings have been generated:
[WARNING]: Current core area is too small for the power grid settings chosen. Th
e power grid will be scaled down.
[WARNING]: Module sky130_fd_sc_hd__tapvpwrvgn1 blackboxed during sta
[WARNING]: Module sky130_fd_sc_hd__fill_1 blackboxed during sta
[WARNING]: Module sky130_fd_sc_hd__fill_2 blackboxed during sta
[WARNING]: Module sky130_ef_sc_hd__decap_12 blackboxed during sta
[WARNING]: VSRC_LOC_FILES is not defined. The IR drop analysis will run, but the
values may be inaccurate.

OpenLane Container (ff5509f):/openlane$
```

Logs Reports: -

1. Clock Report

```
shams@VLSI:~/Desktop/OpenLane/designs/updown_counter_32/runs/RUN_2026.06.07_07.2
3.41$ grep -R "clock clk" logs reports | head -1
logs/placement/8-gpl_sta.log:          0.00    0.00    0.00    clock clk
(rise edge)
shams@VLSI:~/Desktop/OpenLane/designs/updown_counter_32/runs/RUN_2026.06.07_07.2
3.41$ grep -R "clock clk" logs reports | tail -1
reports/cts/13-cts_sta.checks.rpt:          6.00    6.00    clock
clk (rise edge)
```

2. Utilization Report

```
shams@VLSI:~/Desktop/OpenLane/designs/updown_counter_32/runs/RUN_2026.06.07_07.23.41$ grep -R "utilization" logs
logs/placement/8-gpl_sta.log:Design area 2725 u^2 74% utilization.
logs/placement/11-dpl_sta.log:Design area 2301 u^2 62% utilization.
logs/placement/9-resizer.log:Design area 2301 u^2 62% utilization.
logs/routing/21-grt_sta.log:Design area 3059 u^2 83% utilization.
logs/routing/15-resizer_design.log:Design area 2975 u^2 81% utilization.
logs/routing/18-rsz_timing_sta.log:Design area 3059 u^2 83% utilization.
logs/routing/16-rsz_design_sta.log:Design area 2975 u^2 81% utilization.
logs/routing/17-resizer_timing.log:Design area 3059 u^2 83% utilization.
logs/cts/13-cts_sta.log:Design area 2426 u^2 66% utilization.
logs/cts/14-resizer.log:Design area 3024 u^2 82% utilization.
```

3. Uncertainty Report

[illegible]



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

6. Core Ring



7. CTS target skew 5% of the clock Period

```
shams@VLSI:~/Desktop/OpenLane/designs/updown_counter_32$ grep CTS_TARGET_SKEW config.json
  "CTS_TARGET_SKEW": 0.3,
shams@VLSI:~/Desktop/OpenLane/designs/updown_counter_32$
```

Clock period = 6 ns

5% of 6 ns:

$$6 \times 0.05 = 0.3 \text{ ns}$$



ASSIGNMENT-3

The domain of the Project:
Physical Design (VLSI)

Team Mentors (and their designation):
Veeramani Sir (Staff Engineer -Synopsys)

Name: - SHAMS TANVEER

Period of the project
November 2025 to May 2026



PicoRV32

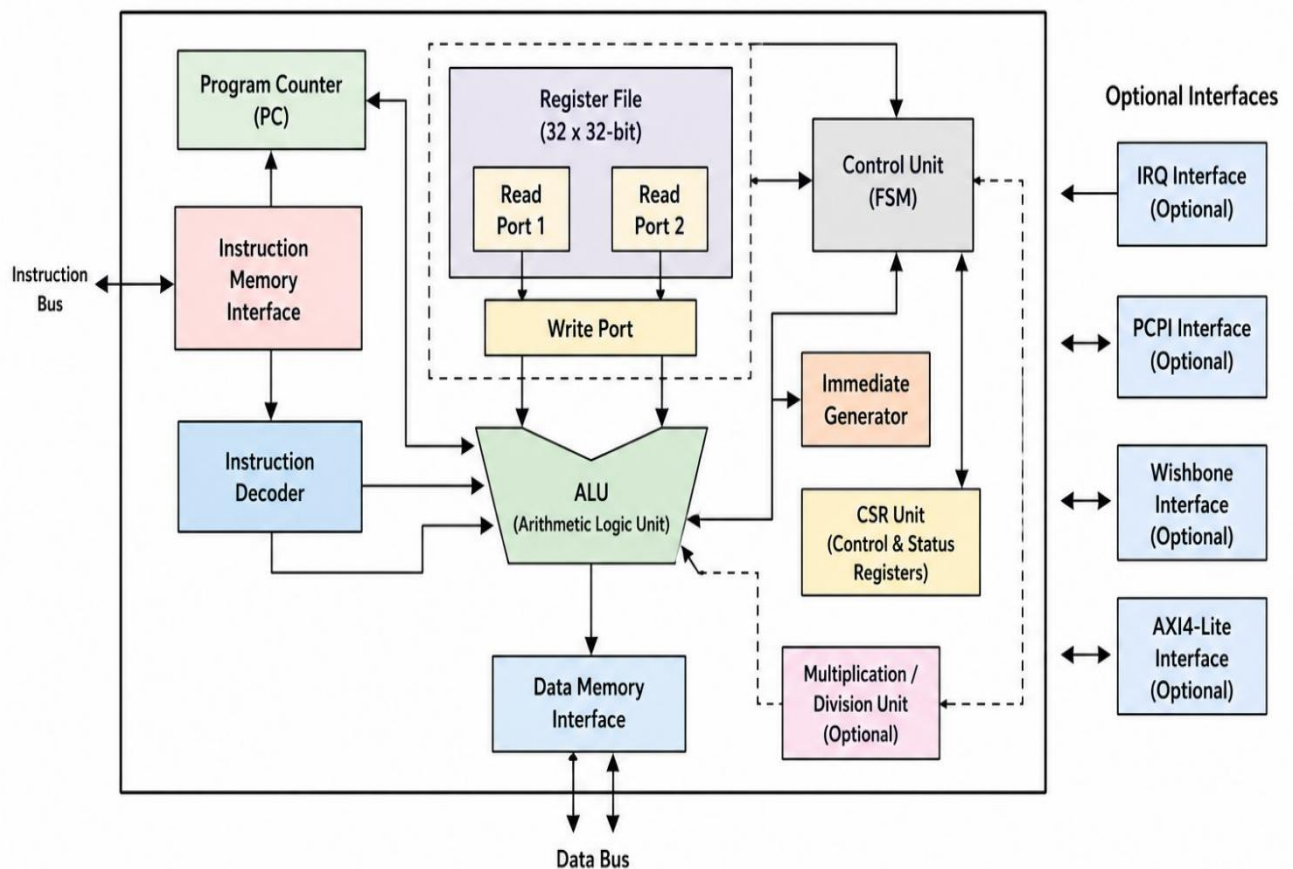
Introduction:

The rapid growth of embedded systems, Internet of Things (IoT) devices, and edge computing has increased the demand for processor cores that are compact, energy-efficient, and highly customizable. Open-source hardware has emerged as a key solution to address these requirements by providing freely accessible processor designs that can be modified, verified, and integrated into a wide range of applications. Among the available open-source instruction set architectures, the RISC-V architecture has gained significant popularity due to its simplicity, modularity, scalability, and royalty-free nature. PicoRV32 is a lightweight, high-performance 32-bit RISC-V processor core developed to provide an efficient implementation of the RV32I base integer instruction set. It is written in Verilog Hardware Description Language (HDL) and is optimized for minimal resource utilization while maintaining competitive operating frequencies. Owing to its small area footprint and configurable architecture, PicoRV32 is widely used in FPGA prototyping, ASIC development, embedded controllers, educational platforms, and research projects. The processor core offers several optional features, including hardware multiplication and division support, interrupt handling, compressed instruction support, and various memory interfaces, allowing designers to tailor the processor according to application requirements. Its modular architecture and clean RTL implementation also make it an excellent choice for learning digital design and VLSI implementation techniques. In modern semiconductor design, implementing a processor extends beyond writing RTL code. The design must undergo a complete physical design flow to transform the hardware description into a manufacturable integrated circuit. This process includes logic synthesis, floorplanning, placement, clock tree synthesis (CTS), routing, timing analysis, and physical verification. Open-source Electronic Design Automation (EDA) tools have made it possible to perform this complete RTL-to-GDSII flow without relying on commercial software. In this project, the PicoRV32 processor core is implemented using the OpenLane automated ASIC flow based on the SkyWater SKY130 Process Design Kit (PDK). The RTL design is synthesized into a gate-level netlist, followed by floorplanning, standard cell placement, clock tree synthesis, global and detailed routing, and signoff verification. Throughout the implementation, design reports such as area utilization, timing analysis, cell utilization, routing congestion, and design rule checking (DRC) are generated to evaluate the quality of the final design. The primary objective of this work is to study the architecture of the PicoRV32 processor and demonstrate its complete ASIC implementation using an open-source VLSI design flow. The project provides practical insight into processor architecture, RTL design, digital synthesis, physical implementation, and chip layout generation, thereby bridging the gap between digital logic design and modern integrated circuit fabrication. This



documentation presents the complete design methodology, beginning with an overview of the RISC-V architecture and PicoRV32 processor, followed by a detailed explanation of the RTL implementation. It then describes each stage of the OpenLane physical design flow and analyzes the generated reports and layout results, providing a comprehensive understanding of the complete ASIC implementation process.

PicoRV32 – 32-bit RISC-V Processor Core



PicoRV32 Core – Key Features

- 32-bit RISC-V (RV32I)
- Compact & Area Efficient
- High Performance (1.7+ DMIPS/MHz)
- Optional Hardware MUL/DIV
- Optional IRQ, PCPI, Wishbone, AXI Interfaces
- Written in Verilog HDL

Fig-1. Architecture of RISC-V



Design Flow:

ASIC IMPLEMENTATION FLOW (RTL to GDSII)

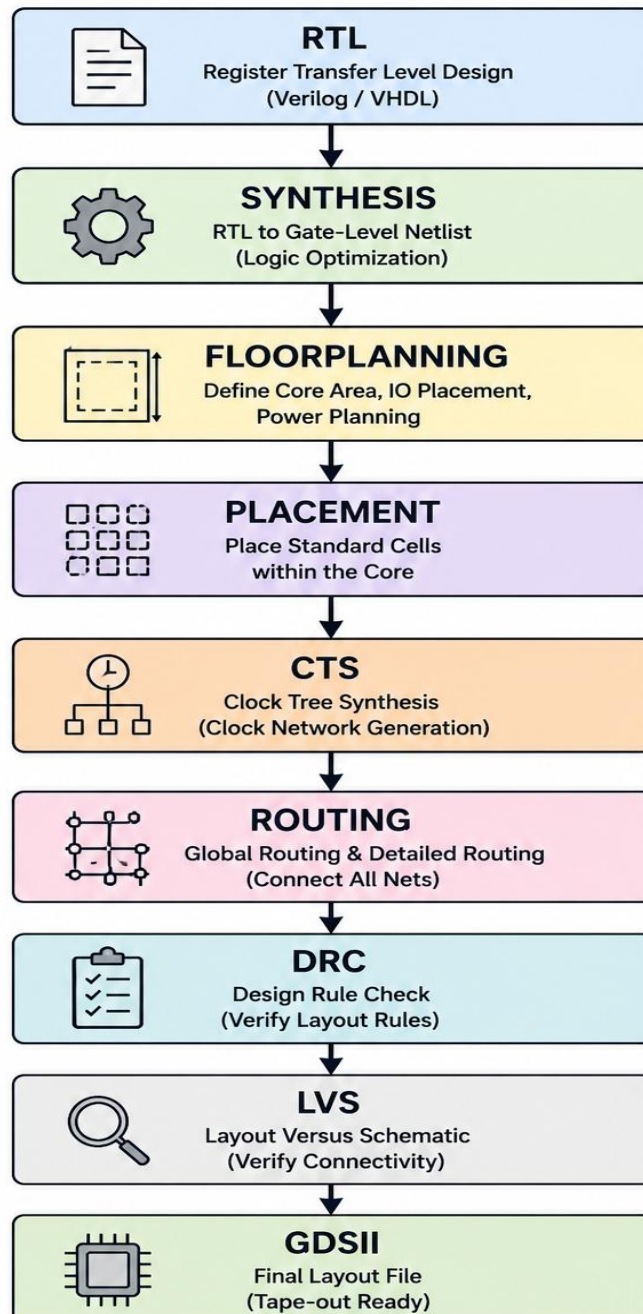


Fig-2. Design Flow



OpenLane:

OpenLane is an open-source automated digital ASIC implementation flow developed to convert a Register Transfer Level (RTL) design into a manufacturable GDSII layout. It integrates several open-source Electronic Design Automation (EDA) tools into a single automated flow, enabling the complete physical design process without requiring commercial software. OpenLane primarily targets the SkyWater SKY130 Process Design Kit (PDK) and supports the complete RTL-to-GDSII implementation required for ASIC fabrication. The OpenLane flow begins with the RTL description of the digital circuit written in Verilog. The RTL is synthesized into a gate-level netlist using logic synthesis, after which floorplanning is performed to define the chip dimensions, core area, input/output pin placement, and power distribution network. Standard cells are then placed within the core area while optimizing timing and minimizing congestion. Clock Tree Synthesis (CTS) is subsequently carried out to distribute the clock signal uniformly across the design with reduced clock skew. After CTS, the routing stage establishes physical metal interconnections between all standard cells using multiple metal layers available in the technology library. Once routing is completed, several verification stages are performed, including Design Rule Checking (DRC), Layout Versus Schematic (LVS), and timing analysis to ensure that the layout satisfies fabrication requirements and matches the original RTL functionality. The final output generated by OpenLane is a GDSII file, which represents the complete integrated circuit layout ready for manufacturing. In this project, OpenLane is used to implement the PicoRV32 processor core through the complete ASIC design flow. The Verilog RTL of the PicoRV32 processor is synthesized and optimized using the SKY130 standard cell library. The automated flow performs floorplanning, standard cell placement, clock tree synthesis, global and detailed routing, and signoff verification. Throughout the implementation, various reports such as synthesis statistics, area utilization, timing analysis, routing congestion, DRC, and LVS reports are generated to evaluate the quality of the design. The successful completion of the OpenLane flow demonstrates the transformation of the PicoRV32 processor from RTL code into a manufacturable ASIC layout.

Key Features of OpenLane

- Fully open-source RTL-to-GDSII ASIC implementation flow.
- Supports the SkyWater SKY130 Process Design Kit (PDK).
- Integrates multiple open-source EDA tools into a unified framework.
- Automates synthesis, floorplanning, placement, CTS, routing, and signoff verification.



- Generates timing, area, power, DRC, and LVS reports.
- Produces a fabrication-ready GDSII layout.
- Suitable for academic research, education, and open-source ASIC development.

Directory Structure:

OpenLane Directory Structure

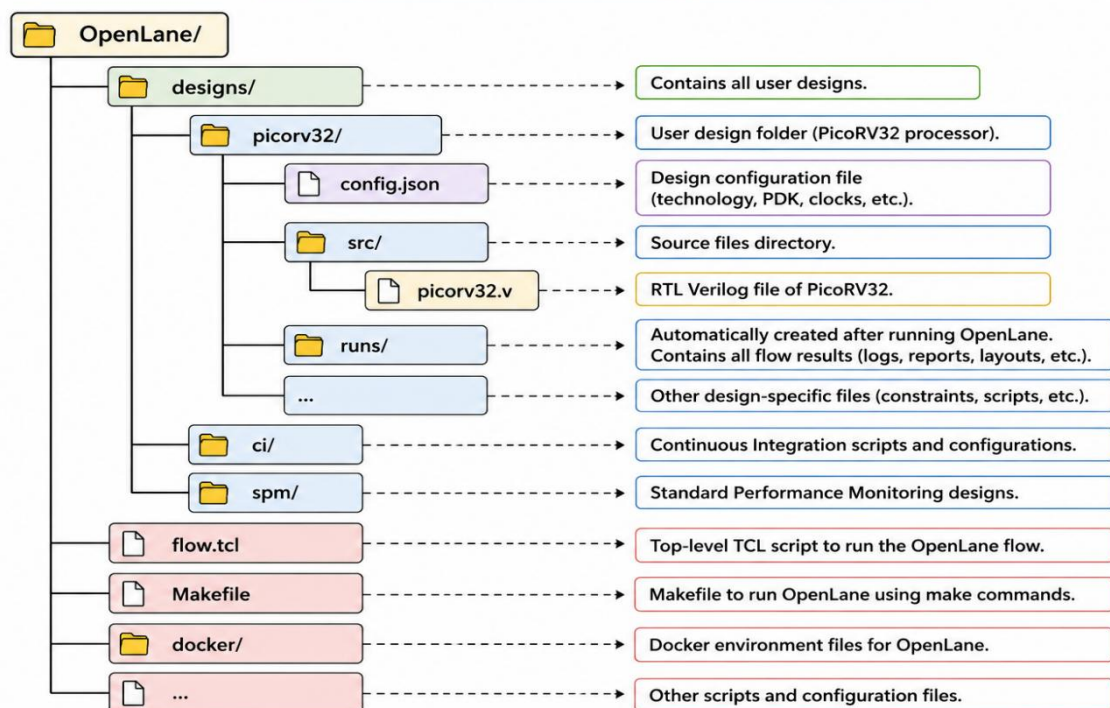


Fig-3. Directory Structure of the project.

Design Setup and Execution in OpenLane

Simple execution of the design without any parameter just to check the outcomes by loading the netlist file and setting the basics criteria to overview of the design.

```
shams_vlsi@VLSI:~/Desktop$ cd VLSI
shams_vlsi@VLSI:~/Desktop/VLSI$ cd OpenLane
shams_vlsi@VLSI:~/Desktop/VLSI/OpenLane$ ls
AUTHORS.md  default.cvcrc  docker  flow.tcl  Jenkinsfile  Makefile  requirements.txt  scripts
configuration  dependencies  docs  gui.py  klayoutrc  README.md  run_designs.py  tests
CONTRIBUTING.md  designs  env.py  install  LICENSE  regression_results  runs  venv
shams_vlsi@VLSI:~/Desktop/VLSI/OpenLane$ cd designs
shams_vlsi@VLSI:~/Desktop/VLSI/OpenLane/designs$ ls
ci  spm
shams_vlsi@VLSI:~/Desktop/VLSI/OpenLane/designs$ mkdir -p picorv32/src
shams_vlsi@VLSI:~/Desktop/VLSI/OpenLane/designs$ ls
ci  picorv32  spm
shams_vlsi@VLSI:~/Desktop/VLSI/OpenLane/designs$ cd picorv32
```



```
shams_vlsi@VLSI:~/Desktop/VLSI/OpenLane/designs/picorv32$ ls
src
shams_vlsi@VLSI:~/Desktop/VLSI/OpenLane/designs/picorv32$ cd src
shams_vlsi@VLSI:~/Desktop/VLSI/OpenLane/designs/picorv32/src$ ls
'picorv32 .v'
shams_vlsi@VLSI:~/Desktop/VLSI/OpenLane/designs/picorv32/src$

shams_vlsi@VLSI:~/Desktop/VLSI/OpenLane/designs/picorv32$ nano config.json
shams_vlsi@VLSI:~/Desktop/VLSI/OpenLane/designs/picorv32$
```

```
GNU nano 7.2 config.json
{
  "DESIGN_NAME": "picorv32",
  "VERILOG_FILES": "dir::src/picorv32.v",
  "CLOCK_PORT": "clk",
  "CLOCK_PERIOD": 10
}

[ Read 6 lines ]
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line M-E Redo      M-6 Copy
```

```
shams_vlsi@VLSI:~/Desktop/VLSI/OpenLane$ make mount
cd /home/shams_vlsi/Desktop/VLSI/OpenLane && \
  docker run --rm -v /home/shams_vlsi/Desktop/VLSI/OpenLane:/openlane -v /home/shams_vlsi/Desktop/VLSI/OpenLane/designs:/openlane/install -v /home/shams_vlsi:/home/shams_vlsi -v /home/shams_vlsi/.ciel:/home/shams_vlsi/.ciel -e PDK_ROOT=/home/shams_vlsi/.ciel -e PDK=sky130A --user 1000:1000 -e DISPLAY=:0 -v /tmp/.X11-unix:/tmp/.X11-unix --network host --security-opt seccomp=unconfined -ti ghcr.io/the-openroad-project/openlane:ff5509f65b17bfa4068d5336495ab1718987ff69-and64
OpenLane Container (ff5509f):/openlane$ flow.tcl -design picorv32
OpenLane v1.0.2 (ff5509f65b17bfa4068d5336495ab1718987ff69)
All rights reserved. (c) 2020-2025 Efabless Corporation and contributors.
Available under the Apache License, version 2.0. See the LICENSE file for more details.

[INFO]: Using configuration in 'designs/picorv32/config.json'...
[INFO]: PDK Root: /home/shams_vlsi/.ciel
[INFO]: Process Design Kit: sky130A
[INFO]: Standard Cell Library: sky130_fd_sc_hd
[INFO]: Optimization Standard Cell Library: sky130_fd_sc_hd
[INFO]: Run Directory: /openlane/designs/picorv32/runs/RUN_2026.06.29_16.49.26
[INFO]: Saving runtime environment...
[INFO]: Preparing LEF files for the nom corner...
[INFO]: Preparing LEF files for the min corner...
[INFO]: Preparing LEF files for the max corner...
[INFO]: Running linter (Verilator) (log: designs/picorv32/runs/RUN_2026.06.29_16.49.26/logs/synthesis/linter.log)...
[INFO]: 0 errors found by linter
[WARNING]: 26 warnings found by linter
[STEP 1]
[INFO]: Running Synthesis (log: designs/picorv32/runs/RUN_2026.06.29_16.49.26/logs/synthesis/1-synthesis.log)...
[STEP 2]
[INFO]: Running Single-Corner Static Timing Analysis (log: designs/picorv32/runs/RUN_2026.06.29_16.49.26/logs/synthesis/2-sta.log)...
[STEP 3]
[INFO]: Running Initial Floorplanning (log: designs/picorv32/runs/RUN_2026.06.29_16.49.26/logs/floorplan/3-initial_fp.log)...
```



```
[STEP 41]
[INFO]: Running OpenROAD Antenna Rule Checker (log: designs/picorv32/runs/RUN_2026.06.29_16.49.26/logs/signoff/41-arc.log)...
[STEP 42]
[INFO]: Running Circuit Validity Checker ERC (log: designs/picorv32/runs/RUN_2026.06.29_16.49.26/logs/signoff/42-erc_screen.log)...
[INFO]: Saving current set of views in 'designs/picorv32/runs/RUN_2026.06.29_16.49.26/results/final'...
[INFO]: Saving runtime environment...
[INFO]: Generating final set of reports...
[INFO]: Created manufacturability report at 'designs/picorv32/runs/RUN_2026.06.29_16.49.26/reports/manufacturability.rpt'.
[INFO]: Created metrics report at 'designs/picorv32/runs/RUN_2026.06.29_16.49.26/reports/metrics.csv'.
[WARNING]: There are max slew violations in the design at the typical corner. Please refer to 'designs/picorv32/runs/RUN_2026.06.29_16.49.26/reports/signoff/31-rcx_sta.checks.rpt'.
[WARNING]: There are max fanout violations in the design at the typical corner. Please refer to 'designs/picorv32/runs/RUN_2026.06.29_16.49.26/reports/signoff/31-rcx_sta.checks.rpt'.
[INFO]: There are no hold violations in the design at the typical corner.
[INFO]: There are no setup violations in the design at the typical corner.
[SUCCESS]: Flow complete.
[INFO]: Note that the following warnings have been generated:
[WARNING]: 26 warnings found by linter
[WARNING]: Module sky130_fd_sc_hd__tapvpwrvrgnd_1 blackboxed during sta
[WARNING]: Module sky130_fd_sc_hd__fill_1 blackboxed during sta
[WARNING]: Module sky130_fd_sc_hd__fill_2 blackboxed during sta
[WARNING]: Module sky130_ef_sc_hd__decap_12 blackboxed during sta
[WARNING]: VSRC_LOC_FILES is not defined. The IR drop analysis will run, but the values may be inaccurate.
[WARNING]: There are max slew violations in the design at the typical corner. Please refer to 'designs/picorv32/runs/RUN_2026.06.29_16.49.26/reports/signoff/31-rcx_sta.checks.rpt'.
[WARNING]: There are max fanout violations in the design at the typical corner. Please refer to 'designs/picorv32/runs/RUN_2026.06.29_16.49.26/reports/signoff/31-rcx_sta.checks.rpt'.
```

Design Configuration:

For the implementation of the PicoRV32 processor, only the essential design parameters were specified in the config.json file. These parameters include the design name, RTL source file, clock port, and target clock period.

```
{
  "DESIGN_NAME": "picorv32",
  "VERILOG_FILES": "dir::src/picorv32.v",
  "CLOCK_PORT": "clk",
  "CLOCK_PERIOD": 10
}
```

No additional OpenLane implementation parameters were explicitly defined. Consequently, OpenLane automatically used its default settings for synthesis, floorplanning, placement, clock tree synthesis, routing, and signoff. This demonstrates that the PicoRV32 processor can be successfully implemented using the standard OpenLane flow without requiring manual tuning of physical design parameters.

The implementation was executed using the command:

```
flow.tcl -design picorv32
```



OpenLane successfully completed all stages of the ASIC flow, including synthesis, floorplanning, placement, clock tree synthesis (CTS), routing, timing analysis, design rule checking (DRC), layout versus schematic (LVS), and GDSII generation. The final execution concluded with the message "[SUCCESS]: Flow complete.", indicating that the PicoRV32 processor was successfully transformed from RTL into a fabrication-ready ASIC layout using the default OpenLane configuration.

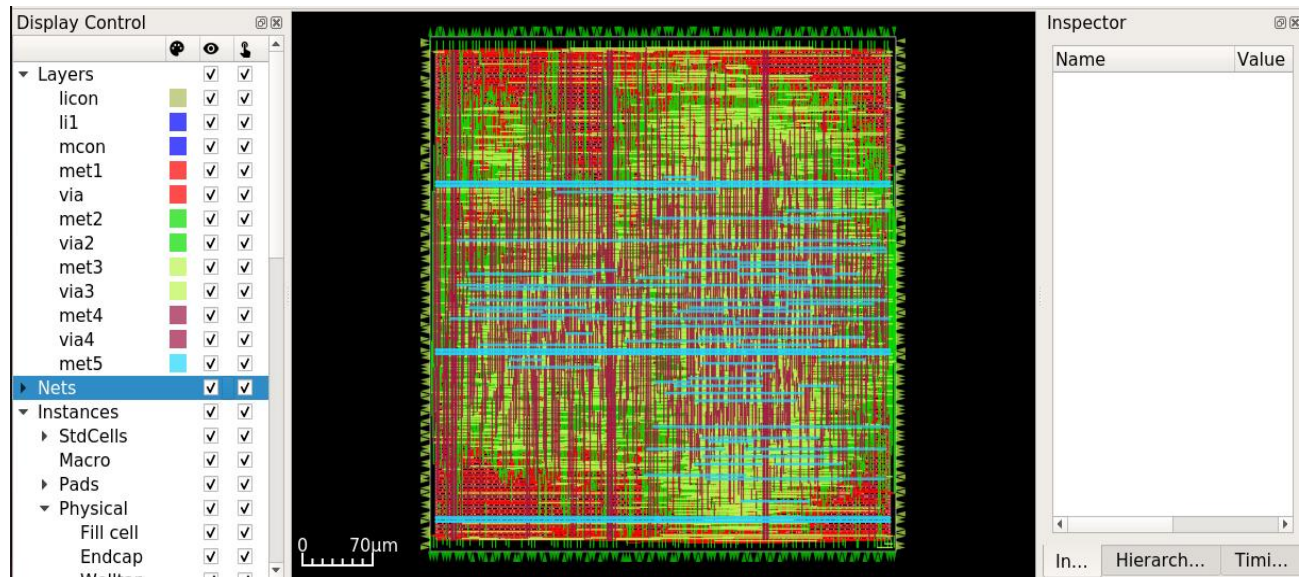


Fig-4. Visualization of Physical Implementation of ASIC

This implementation was carried out using the default OpenLane flow without applying any additional design constraints or parameters provided by the mentor. The objective of this initial implementation was to verify that the RTL design could successfully complete the RTL-to-GDSII flow and to identify any issues or violations that might occur during synthesis, floorplanning, placement, clock tree synthesis, routing, and signoff. This baseline run also serves to evaluate the overall quality of the design, including timing, area, power, and design rule compliance, before applying the assignment-specific constraints and optimization parameters.

Analysing Maximum Frequency for the Design:

The maximum operating frequency of the picorv32 processor is determined by evaluating the design under different clock period constraints. The physical design flow is executed multiple times, each with a different clock period. After every implementation, the timing reports are analyzed to verify whether setup timing is met. The clock period is gradually reduced until timing violations occur. The minimum clock period for which the design satisfies all timing requirements is considered the critical operating point. The corresponding maximum operating frequency is then calculated using:



$$F_{\max} = \frac{1}{T_{\min}}$$

where $T_{\min}$ is the minimum clock period (in seconds) that meets timing constraints.

Using command: -

cat /openlane/designs/picorv32/runs/RUN_2026.06.30_10.59.10/reports/signoff/31-rx_sta.summary.rpt

Visualizing different clock period, we have got the highest maximum frequency in which the design can operate.

Table-1. Analysing different clock period

w	Frequency (MHz)	WNS (ns)	TNS (ns)	Worst Setup Slack (ns)	Worst Hold Slack (ns)	Result
10.0	100.00	0.00	0.00	+3.15	Positive	☑ Pass
8.0	125.00	0.00	0.00	+1.19	Positive	☑ Pass
7.0	142.86	0.00	0.00	+0.19	+0.12	☑ Pass
6.8	147.06	0.00	0.00	+0.01	+0.12	☑ Pass
6.7	149.25	-0.10	-0.39	-0.10	+0.11	✗ Fail
6.5	153.85	-0.23	-1.19	-0.23	+0.05	✗ Fail
6.0	166.67	-0.79	-32.80	-0.79	+0.10	✗ Fail

The smallest clock period at which the design satisfies all setup and hold timing constraints is:

$$T_{\min} = 6.8 \text{ ns}$$

Hence,

$$F_{\max} = \frac{1}{6.8 \times 10^{-9}} = 147.06 \text{ MHz}$$



Design Setup and Execution with given Parameters:

The design was originally implemented and signed off at a clock period of 20 ns (50 MHz), where static timing analysis reported a worst setup slack of +10.71 ns, indicating a comfortable margin with an actual critical path delay of approximately 9.3 ns. To further characterize the design's performance limits, a clock period sweep was carried out, tightening the period from 10.0 ns down to 6.0 ns while monitoring setup and hold slack at each step. At 10.0 ns (100 MHz), the design passed with a worst setup slack of +3.15 ns, and at 8.0 ns (125 MHz) it still passed comfortably with +1.19 ns of margin. At 7.0 ns (142.86 MHz), the margin narrowed to +0.19 ns, and at 6.8 ns (147.06 MHz) the design still met timing, though with only +0.01 ns of setup slack remaining. Beyond this point, timing failed: at 6.7 ns (149.25 MHz), worst setup slack dropped to -0.10 ns with a total negative slack of -0.39 ns, and performance degraded further at 6.5 ns and 6.0 ns, where total negative slack reached -32.80 ns. Hold timing remained positive across every period tested, so no hold violations were observed at any point in the sweep. These results establish 6.8 ns (147.06 MHz) as the true minimum clock period the design can achieve, since 6.7 ns and below consistently fail. However, because the margin at 6.8 ns is negligible, it should be treated only as the theoretical performance limit rather than a safe operating frequency. For practical use, the original 20 ns signoff period remains the most reliable choice given its large margin, while 8.0 ns (125 MHz) offers a strong middle ground a substantial frequency improvement over 20 ns while still retaining healthy margin against process, voltage, and temperature variation.

Table-2. Assignment Parameters

S.No	Requirement	Value at 20 ns	What to Set
1	Synthesis uncertainty (15%)	3.0 ns	SYNTH_CLOCK_UNCERTAINTY = 3.0
2	CTS uncertainty (5%)	1.0 ns	Clock uncertainty setting = 1.0
3	Derate (3%)	±3%	Add derate command in SDC file
4	I/O delay (50%)	10.0 ns	Input/output delay setting = 10.0
5	Core utilization	60%	FP_CORE_UTIL = 60
6	Aspect ratio	0.7	FP_ASPECT_RATIO = 0.7
7	Pin placement	Inputs one side, outputs opposite, on Metal4	Custom pin order file + FP_IO_HLAYER = met4 / FP_IO_VLAYER = met4



S.No	Requirement	Value at 20 ns	What to Set
8	Core ring	Enabled	FP_PDN_CORE_RING = true
9	CTS target skew (5%)	1.0 ns	CTS_TARGET_SKEW = 1.0
10	Clock network delay after CTS	Enabled	CTS_CLK_NETWORK_LATENCY = true

Extraction of all inputs and outputs ports.

```
shams_vlsi@VLSI:~/Desktop/VLSI/OpenLane/designs/picorv32/src$ grep -n "input\|output" picorv32.v
90:    input clk, resetn,
91:    output reg trap,
93:    output reg      mem_valid,
94:    output reg      mem_instr,
95:    input           mem_ready,
97:    output reg [31:0] mem_addr,
98:    output reg [31:0] mem_wdata,
99:    output reg [ 3:0] mem_wstrb,
100:   input           [31:0] mem_rdata,
103:   output           mem_la_read,
104:   output           mem_la_write,
105:   output [31:0] mem_la_addr,
106:   output reg [31:0] mem_la_wdata,
107:   output reg [ 3:0] mem_la_wstrb,
110:   output reg      pcpi_valid,
111:   output reg [31:0] pcpi_insn,
112:   output [31:0] pcpi_rs1,
113:   output [31:0] pcpi_rs2,
114:   input           pcpi_wr,
115:   input [31:0] pcpi_rd,
116:   input           pcpi_wait,
117:   input           pcpi_ready,
120:   input [31:0] irq,
121:   output reg [31:0] eoi,
124:   output reg      rvfi_valid,
125:   output reg [63:0] rvfi_order,
126:   output reg [31:0] rvfi_insn,
127:   output reg      rvfi_trap,
128:   output reg      rvfi halt,
```



Port placement – Inputs on one side, Outputs on the other side, using Metal4.

```
shams_vlsi@VLSI:~/Desktop/OpenLane/designs/picorv32$ cat pin_order.cfg
#N
clk
resetn
mem_ready
mem_rdata.*
pcpi_wr
pcpi_rd.*
pcpi_wait
pcpi_ready
irq.*

#S
trap
mem_valid
mem_instr
mem_addr.*
mem_wdata.*
mem_wstrb.*
mem_la.*
pcpi_valid
pcpi_insn.*
pcpi_rs1.*
pcpi_rs2.*
eoi.*
trace_valid
trace_data.*
shams_vlsi@VLSI:~/Desktop/OpenLane/designs/picorv32$
```

Setting up the configurations according to the need.

```
shams_vlsi@VLSI: ~/Desktop/OpenLane/designs... x shams_vlsi@VLSI: ~/Desktop/OpenLane x shams_vlsi@VLSI: ~/Desktop/OpenLane/designs... x
{
  "DESIGN_NAME": "picorv32",
  "VERILOG_FILES": "dir::src/*.v",
  "CLOCK_PORT": "clk",
  "CLOCK_PERIOD": 20,
  "SYNTH_CLOCK_UNCERTAINTY": 3.0,
  "SYNTH_TIMING_DERATE": 0.03,
  "FP_SIZING": "relative",
  "FP_CORE_UTIL": 55,
  "FP_ASPECT_RATIO": 0.7,
  "FP_IO_HLAYER": "met4",
  "FP_IO_VLAYER": "met4",
  "FP_IO_MIN_DISTANCE": 0.5,
  "FP_PIN_ORDER_CFG": "dir::pin_order.cfg",
  "FP_PDN_CORE_RING": true,
  "FP_PDN_CORE_RING_VOFFSET": 0,
  "FP_PDN_CORE_RING_HOFFSET": 0,
  "FP_PDN_ENABLE_RAILS": true,
  "FP_PDN_ENABLE_GLOBAL_CONNECTIONS": true,
  "FP_TAPCELL_DIST": 14,
  "GRT_REPAIR_ANTENNAS": true,
  "CTS_TARGET_SKEW": 1.0,
  "CTS_CLK_NETWORK_LATENCY": true,
  "PL_RESIZER_DESIGN_OPTIMIZATIONS": true,
  "PL_RESIZER_TIMING_OPTIMIZATIONS": false,
  "PL_RESIZER_BUFFER_INPUT_PORTS": true,
  "PL_RESIZER_BUFFER_OUTPUT_PORTS": true,
  "GLB_RESIZER_TIMING_OPTIMIZATIONS": false,
  "SYNTH_SDC_FILE": "dir::pre_cts.sdc",
  "PNR_SDC_FILE": "dir::pre_cts.sdc",
  "SIGNOFF_SDC_FILE": "dir::post_cts.sdc"
}
```



Setting up the pre-CTS requirement.

```
shams_vlsi@VLSI:~/Desktop/OpenLane/designs/picorv32$ cat pre_cts.sdc
#=====
# PicoRV32 Timing Constraints - PRE-CTS
# Clock Period = 20 ns
#=====

create_clock -name clk -period 20 [get_ports clk]

# Synthesis / pre-CTS clock uncertainty = 15% of 20ns = 3.0ns
set_clock_uncertainty 3.0 [get_clocks clk]

# Input Delay = 50% of clock period (exclude clk pin itself)
set_input_delay 10 -clock clk [remove_from_collection [all_inputs] [get_ports clk]]

# Output Delay = 50% of clock period
set_output_delay 10 -clock clk [all_outputs]

# Timing Derate = 3%
set_timing_derate -early 0.97
set_timing_derate -late 1.03

# Clock tree does not exist yet - keep ideal (do NOT set_propagated_clock here)
shams_vlsi@VLSI:~/Desktop/OpenLane/designs/picorv32$
```

Setting up the post-CTS requirement.

```
shams_vlsi@VLSI:~/Desktop/OpenLane/designs/picorv32$ cat post_cts.sdc
#=====
# PicoRV32 Timing Constraints - POST-CTS
# Clock Period = 20 ns
#=====

create_clock -name clk -period 20 [get_ports clk]

# Post-CTS clock uncertainty = 5% of 20ns = 1.0ns
set_clock_uncertainty 1.0 [get_clocks clk]

# Input Delay = 50% of clock period (exclude clk pin itself)
set_input_delay 10 -clock clk [remove_from_collection [all_inputs] [get_ports clk]]

# Output Delay = 50% of clock period
set_output_delay 10 -clock clk [all_outputs]

# Timing Derate = 3%
set_timing_derate -early 0.97
set_timing_derate -late 1.03

# Clock tree now exists - use propagated (real insertion delay/skew)
set_propagated_clock [get_clocks clk]
shams_vlsi@VLSI:~/Desktop/OpenLane/designs/picorv32$
```

Execution of the Flow.

```
shams_vlsi@VLSI:~/Desktop/OpenLane$ make mount
cd /home/shams_vlsi/Desktop/OpenLane && \
  docker run --rm -v /home/shams_vlsi/Desktop/OpenLane:/openlane -v /home/shams_vlsi/Desktop/OpenLane/designs:/openlane/install -v /home/shams_vlsi:/home/shams_vlsi -v /home/shams_vlsi/.ciel:/home/shams_vlsi/.ciel -e PDK_ROOT=/home/shams_vlsi/.ciel -e PDK=sky130A --user 1000:1000 -e DISPLAY=:0 -v /tmp/.X11-unix:/tmp/.X11-unix --network host --security opt seccomp=unconfined -ti ghcr.io/the-openroad-project/openlane:ff5509f65b17bfa4068d5336495ab1718987ff69-amd64
openlane Container (ff5509f):/openlane$ flow.tcl -design picorv32
openlane v1.0.2 (ff5509f65b17bfa4068d5336495ab1718987ff69)
All rights reserved. (c) 2020-2025 Efabless Corporation and contributors.
Available under the Apache License, version 2.0. See the LICENSE file for more details.

INFO]: Using configuration in 'designs/picorv32/config.json'...
INFO]: PDK Root: /home/shams_vlsi/.ciel
INFO]: Process Design Kit: sky130A
INFO]: Standard Cell Library: sky130_fd_sc_hd
INFO]: Optimization Standard Cell Library: sky130_fd_sc_hd
INFO]: Run Directory: /openlane/designs/picorv32/runs/RUN_2026.07.04_18.52.21
INFO]: Saving runtime environment...
INFO]: Preparing LEF files for the nom corner...
INFO]: Preparing LEF files for the min corner...
INFO]: Preparing LEF files for the max corner...
INFO]: Running linter (Verilator) (log: designs/picorv32/runs/RUN_2026.07.04_18.52.21/logs/synthesis/linter.log)...
INFO]: 0 errors found by linter
WARNING]: 26 warnings found by linter
STEP 1]
INFO]: Running Synthesis (log: designs/picorv32/runs/RUN_2026.07.04_18.52.21/logs/synthesis/1-synthesis.log)...
STEP 2]
INFO]: Running Single-Corner Static Timing Analysis (log: designs/picorv32/runs/RUN_2026.07.04_18.52.21/logs/synthesis/2-sta.log)...
STEP 3]
INFO]: Running Initial Floorplanning (log: designs/picorv32/runs/RUN_2026.07.04_18.52.21/logs/floorplan/3-initial_floorplan.log)...
```



```
[STEP 9]
[INFO]: Running Placement Resizer Design Optimizations (log: designs/picorv32/runs/RUN_2026.07.04_18.52.21/logs/placement/9-resizer.log)...
[STEP 10]
[INFO]: Running Detailed Placement (log: designs/picorv32/runs/RUN_2026.07.04_18.52.21/logs/placement/10-detailed.log)...
[STEP 11]
[INFO]: Running Single-Corner Static Timing Analysis (log: designs/picorv32/runs/RUN_2026.07.04_18.52.21/logs/placement/11-dpl_sta.log)...
[STEP 12]
[INFO]: Running Clock Tree Synthesis (log: designs/picorv32/runs/RUN_2026.07.04_18.52.21/logs/cts/12-cts.log)...
[STEP 13]
[INFO]: Running Single-Corner Static Timing Analysis (log: designs/picorv32/runs/RUN_2026.07.04_18.52.21/logs/cts/13-cts_sta.log)...
[STEP 14]
[INFO]: Running Placement Resizer Timing Optimizations (log: designs/picorv32/runs/RUN_2026.07.04_18.52.21/logs/cts/14-resizer.log)...
[STEP 15]
[INFO]: Running Global Routing Resizer Design Optimizations (log: designs/picorv32/runs/RUN_2026.07.04_18.52.21/logs/routing/15-resizer_design.log)...
[STEP 16]
[INFO]: Running Single-Corner Static Timing Analysis (log: designs/picorv32/runs/RUN_2026.07.04_18.52.21/logs/routing/16-rsz_design_sta.log)...
[STEP 17]
[INFO]: Running Global Routing Resizer Timing Optimizations (log: designs/picorv32/runs/RUN_2026.07.04_18.52.21/logs/routing/17-resizer_timing.log)...
[STEP 18]
[INFO]: Running Single-Corner Static Timing Analysis (log: designs/picorv32/runs/RUN_2026.07.04_18.52.21/logs/routing/18-rsz_timing_sta.log)...
[STEP 19]
[INFO]: Running Global Routing (log: designs/picorv32/runs/RUN_2026.07.04_18.52.21/logs/routing/19-global.log)...
[INFO]: Converting Magic DRC database to various tool-readable formats...
[INFO]: No DRC violations after GDS streaming out.
[STEP 41]
[INFO]: Running OpenROAD Antenna Rule Checker (log: designs/picorv32/runs/RUN_2026.07.04_18.52.21/logs/signoff/41-arc.log)...
[STEP 42]
[INFO]: Running Circuit Validity Checker ERC (log: designs/picorv32/runs/RUN_2026.07.04_18.52.21/logs/signoff/42-erc_screen.log)...
[INFO]: Saving current set of views in 'designs/picorv32/runs/RUN_2026.07.04_18.52.21/results/final'...
[INFO]: Saving runtime environment...
[INFO]: Generating final set of reports...
[INFO]: Created manufacturability report at 'designs/picorv32/runs/RUN_2026.07.04_18.52.21/reports/manufacturability.rpt'
[INFO]: Created metrics report at 'designs/picorv32/runs/RUN_2026.07.04_18.52.21/reports/metrics.csv'.
[WARNING]: There are max fanout violations in the design at the typical corner. Please refer to 'designs/picorv32/runs/RUN_2026.07.04_18.52.21/reports/signoff/31-rcx_sta.checks.rpt'.
[INFO]: There are no hold violations in the design at the typical corner.
[INFO]: There are no setup violations in the design at the typical corner.
[SUCCESS]: Flow complete.
[INFO]: Note that the following warnings have been generated:
[WARNING]: 26 warnings found by linter
[WARNING]: Module sky130_fd_sc_hd__tapvpwrvrnd_1 blackboxed during sta
[WARNING]: Module sky130_ef_sc_hd__decap_12 blackboxed during sta
[WARNING]: Module sky130_fd_sc_hd__fill_1 blackboxed during sta
[WARNING]: Module sky130_fd_sc_hd__fill_2 blackboxed during sta
[WARNING]: VSRC_LOC_FILES is not defined. The IR drop analysis will run, but the values may be inaccurate.
[WARNING]: There are max fanout violations in the design at the typical corner. Please refer to 'designs/picorv32/runs/RUN_2026.07.04_18.52.21/reports/signoff/31-rcx_sta.checks.rpt'.

OpenLane Container (ff5509f):/openlane$
```

The complete OpenLane RTL-to-GDSII flow for the picorv32 design was executed successfully at the target clock period of 20 ns (50 MHz), progressing through synthesis, floorplanning, placement, clock tree synthesis (CTS), detailed routing, and signoff without any fatal errors. Static timing analysis confirmed the design meets timing at this period, with a worst setup slack of +10.71 ns, corresponding to an actual critical path delay of approximately 9.3 ns — providing a comfortable margin well within the 20 ns budget. Hold timing also passed with positive slack throughout. While a max fanout violation count of 108 was flagged in the STA report, it did not result in any timing failures at this period. LVS and DRC signoff checks were run to completion, confirming



physical layout integrity. Overall, the design flow completed cleanly at 20 ns, demonstrating a robust and manufacturable implementation with substantial timing margin for reliable operation. As the design flow completed successfully, now looking into parameters through the reports and logs.

Verification of Design Constraints

1. Clock uncertainty (15% synthesis, 5% CTS)

```
shams_vlsi@VLSI:~/Desktop/OpenLane/designs/picorv32/runs/RUN_2026.07.07_13.19.19$ grep -n "clock uncertainty" reports/synthesis/2-syn_sta.min.rpt
27:          3.00    3.00    clock uncertainty
59:          3.00    3.00    clock uncertainty
94:          3.00    3.00    clock uncertainty
126:         3.00    3.00    clock uncertainty
158:         3.00    3.00    clock uncertainty
190:         3.00    3.00    clock uncertainty
222:         3.00    3.00    clock uncertainty
254:         3.00    3.00    clock uncertainty
286:         3.00    3.00    clock uncertainty
318:         3.00    3.00    clock uncertainty
350:         3.00    3.00    clock uncertainty
382:         3.00    3.00    clock uncertainty
```

```
shams_vlsi@VLSI:~/Desktop/OpenLane/designs/picorv32/runs/RUN_2026.07.07_13.19.19$ cat reports/cts/13-cts_sta.skew.rpt
=====
report_clock_skew
=====
Clock clk
Latency    CRPR      Skew
_16904_/CLK ^
0.95
_16113_/CLK ^
0.83    -0.02    0.10
```

- “grep -n "clock uncertainty" reports/synthesis/2-syn_sta.min.rpt”
- “cat reports/cts/13-cts_sta.skew.rpt”

The clock skew after Clock Tree Synthesis (CTS) is **0.10 ns**. For a design clock period of **20 ns**, the target skew specified in the assignment is **5% (1.0 ns)**. Since the measured skew is significantly lower than the target value, the CTS implementation successfully satisfies the skew requirement.

2. Timing Derate = 3%

- “grep -Rin "timing_derate"”

```
shams_vlsi@VLSI:~/Desktop/OpenLane/designs/picorv32/runs/RUN_2026.07.07_13.19.19$ grep -Rin "timing_derate" .
./config.tcl:368:set ::env(SYNTH_TIMING_DERATE) "0.03"
./config_in.tcl:11:set ::env(SYNTH_TIMING_DERATE) "0.03"
./results/final/sdc/picorv32.sdc:833:set_timing_derate -early 0.9700
./results/final/sdc/picorv32.sdc:834:set_timing_derate -late 1.0300
./results/cts/picorv32.sdc:833:set_timing_derate -early 0.9700
./results/cts/picorv32.sdc:834:set_timing_derate -late 1.0300
./tmp/14-picorv32.sdc:833:set_timing_derate -early 0.9700
./tmp/14-picorv32.sdc:834:set_timing_derate -late 1.0300
./tmp/placement/9-resizer.sdc:832:set_timing_derate -early 0.9700
./tmp/placement/9-resizer.sdc:833:set_timing_derate -late 1.0300
./tmp/floorplan/3-initial_fp.sdc:832:set_timing_derate -early 0.9700
./tmp/floorplan/3-initial_fp.sdc:833:set_timing_derate -late 1.0300
```



3. Input and Output Delay 50% of Clock Period

- “grep "set_input_delay"
~/Desktop/OpenLane/designs/picorv32/runs/RUN_2026.07.07_16.55.47/results/final/sdc/picorv32.sdc | head”

```
shams_vlsi@VLSI:~/Desktop/OpenLane/designs/picorv32/runs/RUN_2026.07.07_16.55.47$ grep "set_input_delay" ~/Desktop/OpenLane/designs/picorv32/runs/RUN_2026.07.07_16.55.47/results/final/sdc/picorv32.sdc | head
set_input_delay 10.0000 -clock [get_clocks {clk}] -add_delay [get_ports {irq[0]}]
set_input_delay 10.0000 -clock [get_clocks {clk}] -add_delay [get_ports {irq[10]}]
set_input_delay 10.0000 -clock [get_clocks {clk}] -add_delay [get_ports {irq[11]}]
set_input_delay 10.0000 -clock [get_clocks {clk}] -add_delay [get_ports {irq[12]}]
set_input_delay 10.0000 -clock [get_clocks {clk}] -add_delay [get_ports {irq[13]}]
set_input_delay 10.0000 -clock [get_clocks {clk}] -add_delay [get_ports {irq[14]}]
set_input_delay 10.0000 -clock [get_clocks {clk}] -add_delay [get_ports {irq[15]}]
set_input_delay 10.0000 -clock [get_clocks {clk}] -add_delay [get_ports {irq[16]}]
set_input_delay 10.0000 -clock [get_clocks {clk}] -add_delay [get_ports {irq[17]}]
set_input_delay 10.0000 -clock [get_clocks {clk}] -add_delay [get_ports {irq[18]}]
```

- “grep "set_output_delay"
~/Desktop/OpenLane/designs/picorv32/runs/RUN_2026.07.07_16.55.47/results/final/sdc/picorv32.sdc | head”

```
shams_vlsi@VLSI:~/Desktop/OpenLane/designs/picorv32/runs/RUN_2026.07.07_16.55.47$ grep "set_output_delay" ~/Desktop/OpenLane/designs/picorv32/runs/RUN_2026.07.07_16.55.47/results/final/sdc/picorv32.sdc | head
set_output_delay 10.0000 -clock [get_clocks {clk}] -add_delay [get_ports {eoi[0]}]
set_output_delay 10.0000 -clock [get_clocks {clk}] -add_delay [get_ports {eoi[10]}]
set_output_delay 10.0000 -clock [get_clocks {clk}] -add_delay [get_ports {eoi[11]}]
set_output_delay 10.0000 -clock [get_clocks {clk}] -add_delay [get_ports {eoi[12]}]
set_output_delay 10.0000 -clock [get_clocks {clk}] -add_delay [get_ports {eoi[13]}]
set_output_delay 10.0000 -clock [get_clocks {clk}] -add_delay [get_ports {eoi[14]}]
set_output_delay 10.0000 -clock [get_clocks {clk}] -add_delay [get_ports {eoi[15]}]
set_output_delay 10.0000 -clock [get_clocks {clk}] -add_delay [get_ports {eoi[16]}]
set_output_delay 10.0000 -clock [get_clocks {clk}] -add_delay [get_ports {eoi[17]}]
```

4. Utilization at Floorplan

- “grep -Ei "util|core area|die area" logs/floorplan/*.log”
- “grep -i "area" reports/synthesis/*.stat.rpt”

```
shams_vlsi@VLSI:~/Desktop/OpenLane/designs/picorv32/runs/RUN_2026.07.07_16.55.47$ grep -Ei "util|core area|die area" logs/floorplan/*.log
logs/floorplan/3-initial_fp.log:[INFO] Floorplanned on a die area of 0.0 0.0 546.55 396.615 (microns). Saving to /openlane/designs/picorv32/runs/RUN_2026.07.07_16.55.47/reports/floorplan/3-initial_fp_die_area.rpt.
logs/floorplan/3-initial_fp.log:[INFO] Floorplanned on a core area of 5.52 10.88 540.96 383.52 (microns). Saving to /openlane/designs/picorv32/runs/RUN_2026.07.07_16.55.47/reports/floorplan/3-initial_fp_core_area.rpt.
logs/floorplan/6-pdn.log:[WARNING PSM-0016] Voltage pad location (VSRC) file not specified, defaulting pad location to checkerboard pattern on core area.
logs/floorplan/6-pdn.log:[WARNING PSM-0016] Voltage pad location (VSRC) file not specified, defaulting pad location to checkerboard pattern on core area.
shams_vlsi@VLSI:~/Desktop/OpenLane/designs/picorv32/runs/RUN_2026.07.07_16.55.47$ grep -i "area" reports/synthesis/*.stat.rpt
Chip area for module '\picorv32': 100370.012800
```

Standard Cell (Chip) Area

From synthesis: Chip area = 100370.0128 μm^2

Core Area

From the floorplan:

Lower Left = (5.52, 10.88)

Upper Right = (540.96, 383.52)



Core width:

$$540.96 - 5.52 = 535.44 \mu m$$

Core height:

$$383.52 - 10.88 = 372.64 \mu m$$

Core area: $535.44 \times 372.64 = 199531.16 \mu m^2$

Actual Utilization

$$\begin{aligned} \text{Utilization} &= \frac{100370.0128}{199531.16} \times 100 \\ &= 50.30\% \end{aligned}$$

5. Floorplan Ratio 0.7

Core Area:

LL = (5.52, 10.88)

UR = (540.96, 383.52)

Calculate:

- **Core Width**

$$540.96 - 5.52 = 535.44 \mu m$$

- **Core Height**

$$383.52 - 10.88 = 372.64 \mu m$$

Calculate Aspect Ratio

OpenLane defines:

$$\boxed{\text{Aspect Ratio} = \frac{\text{Height}}{\text{Width}}}$$

So,

$$\frac{372.64}{535.44} = 0.696$$

$$\approx 0.70$$



6. Port Placement — Inputs One Side, Outputs Other Side on Metal 4

“awk ’

/^PINS/ {inside=1}

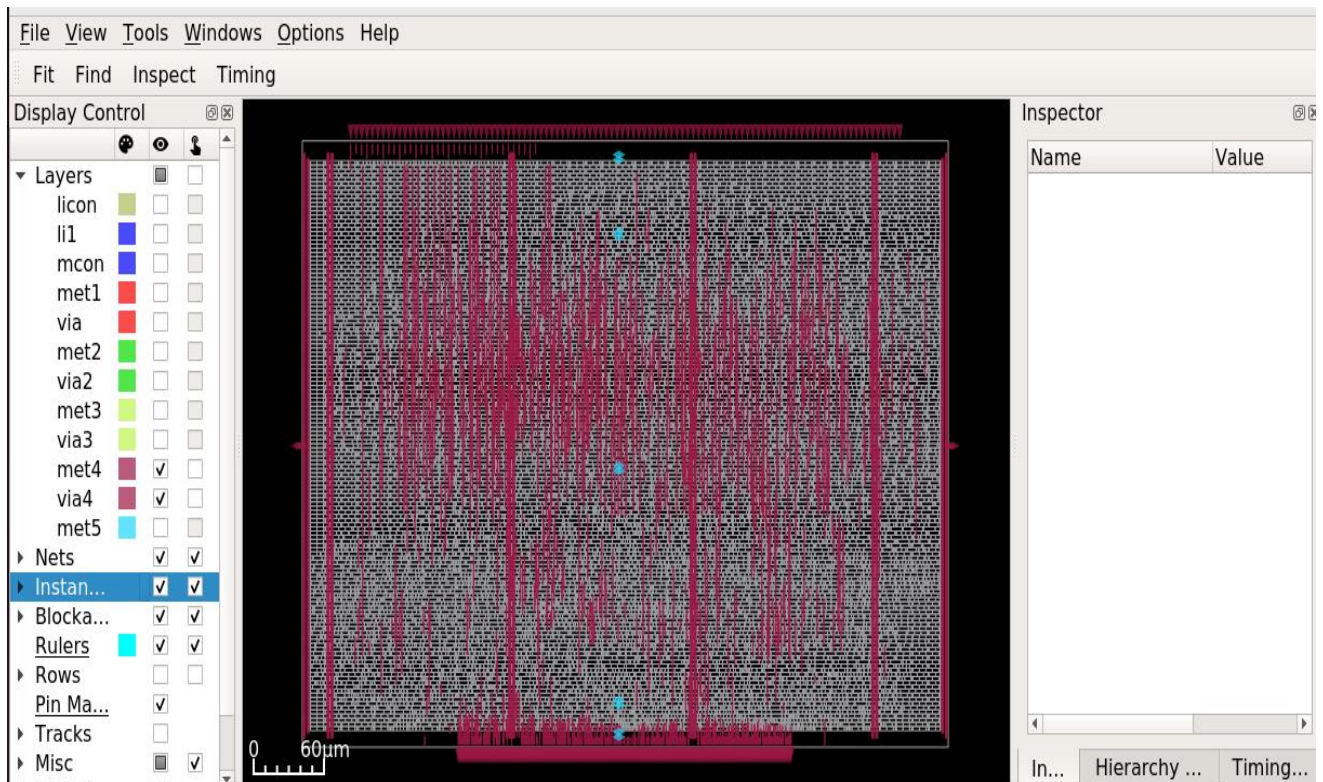
/^END PINS/ {inside=0}

inside && /^ *-/ {pin=\$2}

inside && /LAYER met4/ {print pin, "-> met4"}

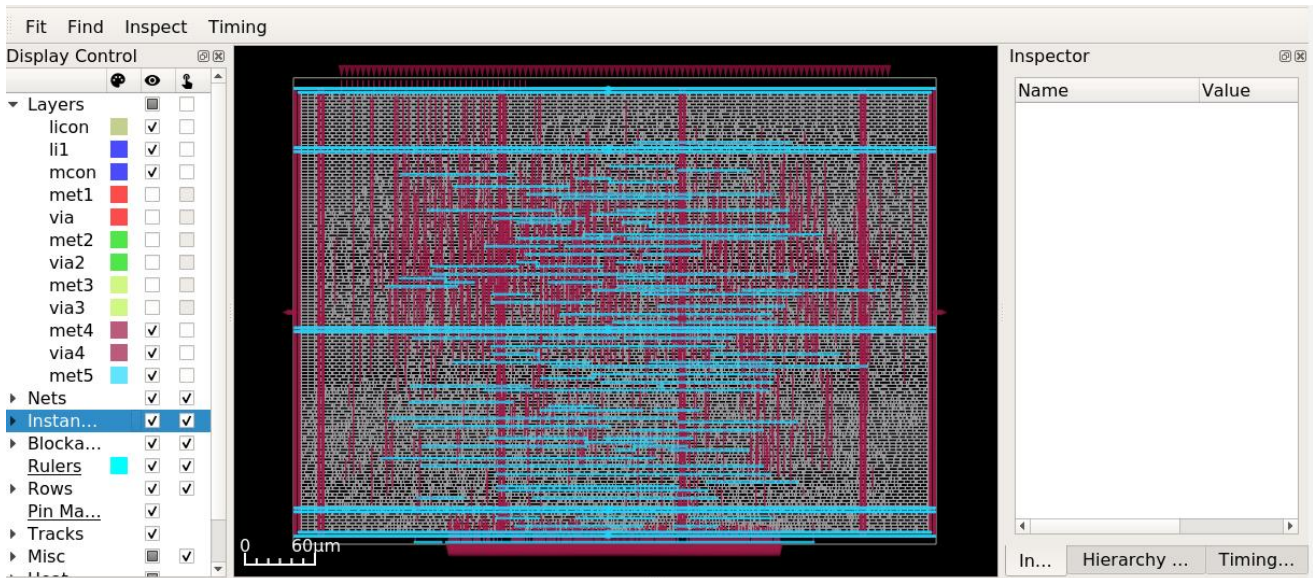
' results/final/def/picorv32.def | head -30”

```
VGND -> met4
VGND -> met4
VGND -> met4
VGND -> met4
VGND -> met4
VGND -> met4
VPWR -> met4
VPWR -> met4
VPWR -> met4
VPWR -> met4
VPWR -> met4
clk -> met4
eoi[0] -> met4
eoi[10] -> met4
eoi[11] -> met4
eoi[12] -> met4
eoi[13] -> met4
eoi[14] -> met4
eoi[15] -> met4
eoi[16] -> met4
eoi[17] -> met4
eoi[18] -> met4
eoi[19] -> met4
eoi[1] -> met4
eoi[20] -> met4
eoi[21] -> met4
eoi[22] -> met4
eoi[23] -> met4
eoi[24] -> met4
```





7. Enable Core Ring in the Design



8. CTS Target Skew 5% of Clock Period

- “cat reports/signoff/28-rcx_sta.skew.rpt”

```
shams_vlsi@VLSI:~/Desktop/OpenLane/designs/picorv32/runs/RUN_2026.07.07_16.55.47$ cat reports/signoff/28-rcx_sta.skew.rpt
t
=====
report_clock_skew
=====
Clock clk
Latency    CRPR    Skew
_15584_/CLK ^
1.21
_16131_/CLK ^
1.07    -0.03    0.11
```

9. Enable Clock Network Delay Post CTS Stage

- “grep -A15 "clock clk" reports/signoff/28-rcx_sta.max.rpt | head -20”

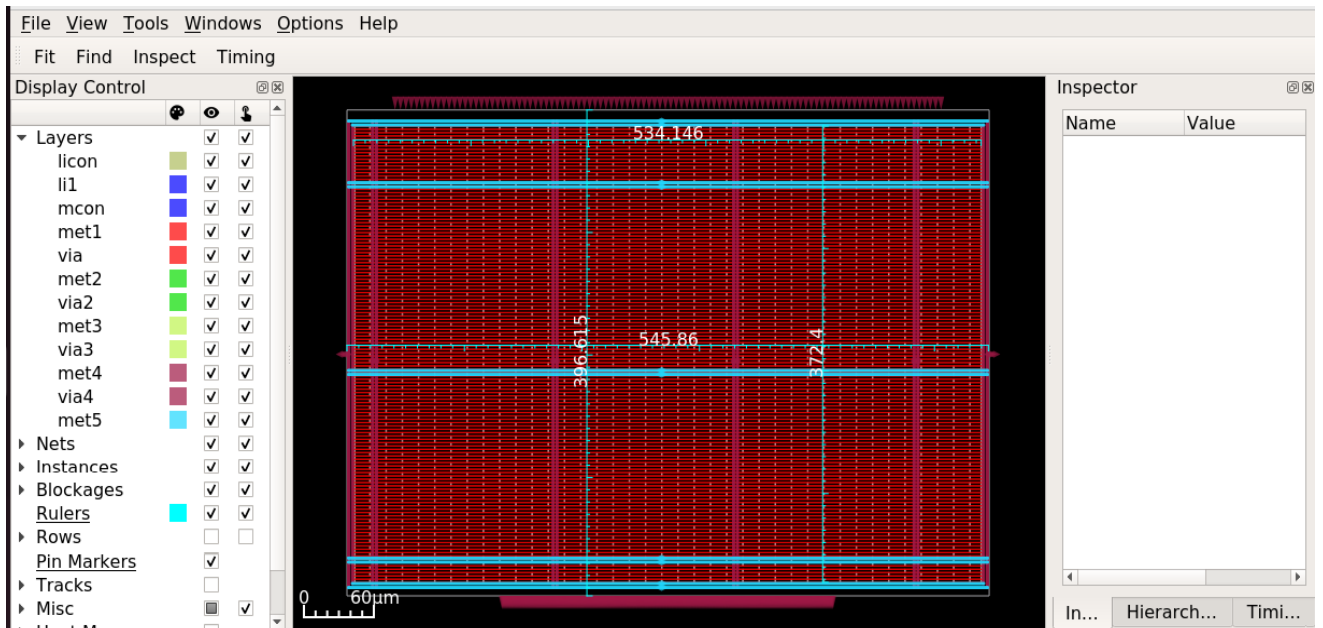
```
shams_vlsi@VLSI:~/Desktop/OpenLane/designs/picorv32/runs/RUN_2026.07.07_16.55.47$ grep -A15 "clock clk" reports/signoff/28-rcx_sta.max.rpt | head -20
0.00    0.00    clock clk (rise edge)
0.00    0.00    clock network delay (propagated)
10.00   10.00 ^ input external delay
10.02   10.02 ^ resetn (in)
1    0.01    0.04    0.02    10.02 ^ resetn (net)
          0.04    0.00    10.02 ^ input34/A (sky130_fd_sc_hd__buf_6)
9    0.13    0.25    0.24    10.27 ^ input34/X (sky130_fd_sc_hd__buf_6)
          0.25    0.01    10.28 ^ _07619_/A (sky130_fd_sc_hd__inv_2)
          0.05    0.06    10.34 v _07619_/Y (sky130_fd_sc_hd__inv_2)
2    0.01    0.05    0.00    10.34 v _07651_/A (sky130_fd_sc_hd__or2_2)
          0.09    0.36    10.70 v _07651_/X (sky130_fd_sc_hd__or2_2)
4    0.02    0.09    0.00    10.70 v _07652_/A (sky130_fd_sc_hd__inv_2)
          0.09    0.11    10.81 ^ _07652_/Y (sky130_fd_sc_hd__inv_2)
--
20.00   20.00    clock clk (rise edge)
0.00    20.00    clock network delay (propagated)
-3.00   17.00    clock uncertainty
```



Exploration of the Design in the Gui.

1. Floorplan

Opening floorplan database in the gui using command – “read_db /openlane/designs/picorv32/runs/RUN_2026.07.04_18.52.21/results/floorplan/picorv32.odt”



Parameter

Dimension (µm)

Core Width

535.44

Core Height

372.64

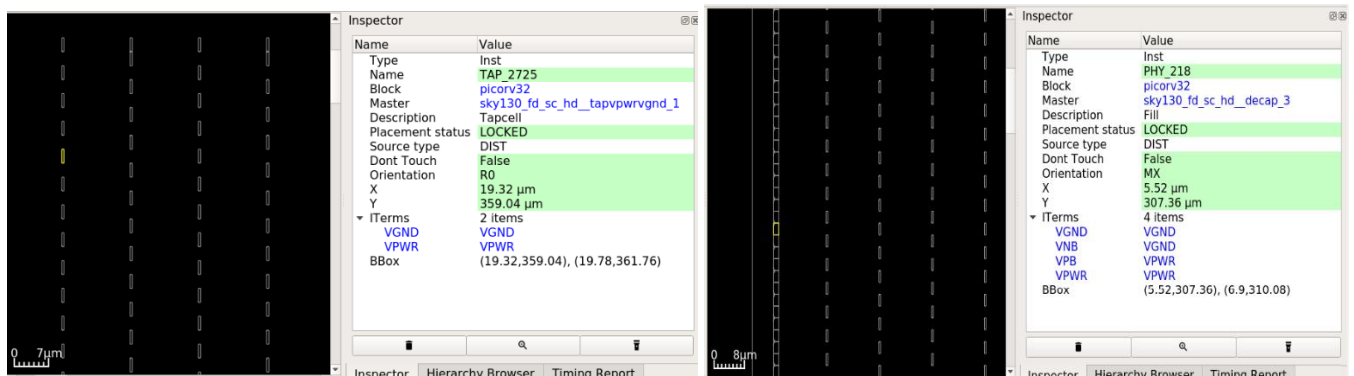
Die Width

546.550

Die Height

396.615

Physical only Cells



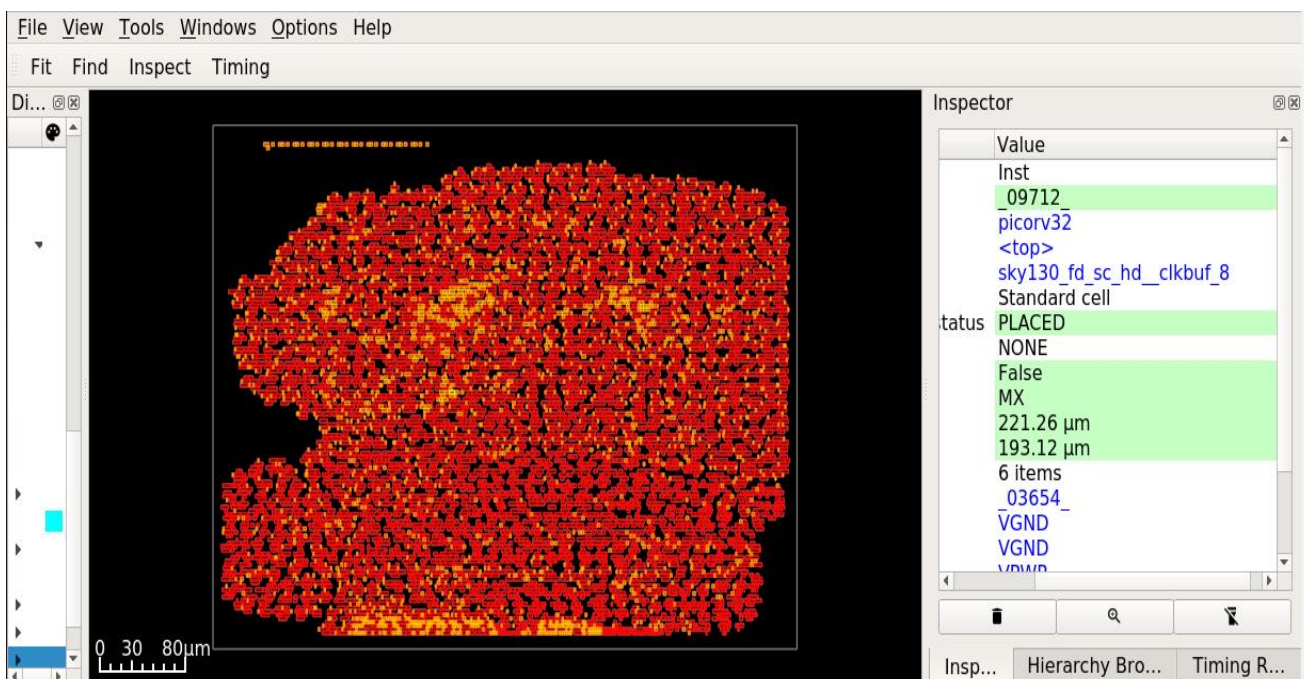


2. Placement

Opening placement database in the gui using command – “read_db /openlane/designs/picorv32/runs/RUN_2026.07.04_18.52.21/results/placement/picorv32.oddb”

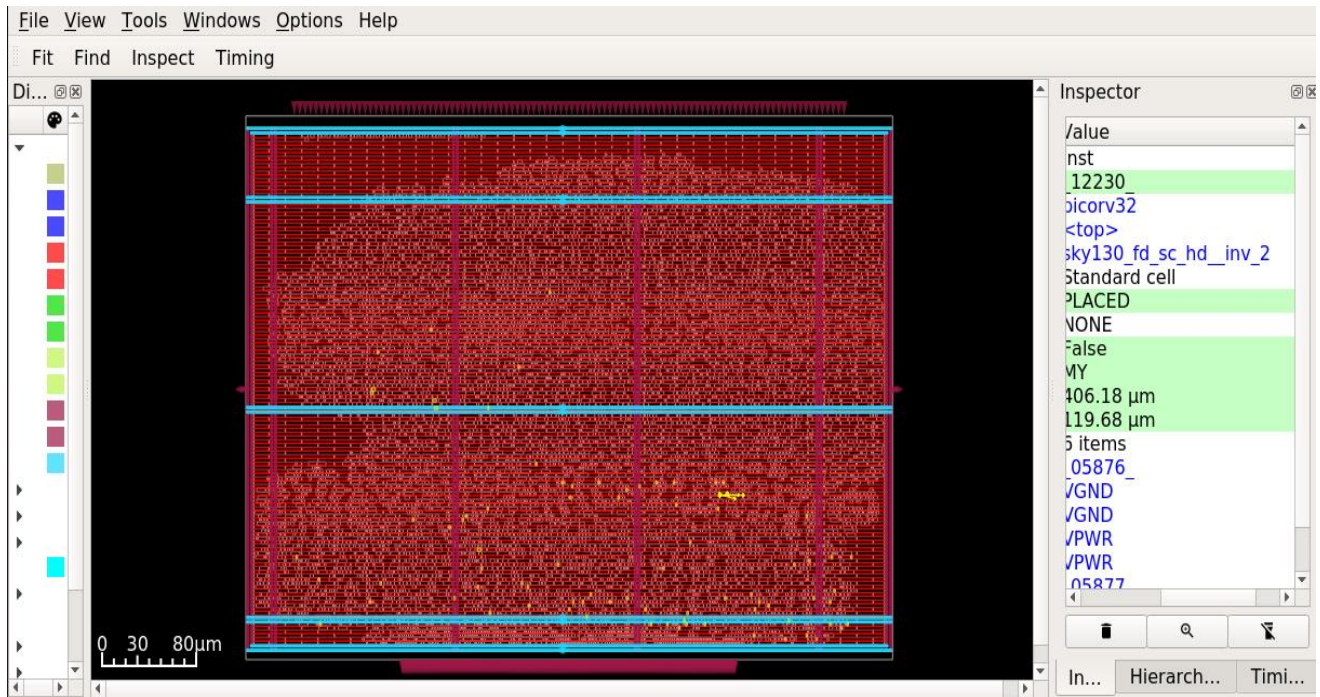
```
Buffers: - set block [ord::get_db_block]

    foreach inst [$block getInsts] {
        set name [[$inst getMaster] getName]
        if {[regexp -nocase "buf" $name]} {
            gui::highlight_inst [$inst getName] 1
        }
    }
}
```



```
Inverters: - set block [ord::get_db_block]
```

```
    foreach inst [$block getInsts] {
        set name [[$inst getMaster] getName]
        if {[regexp -nocase "inv" $name]} {
            gui::highlight_inst [$inst getName] 2
        }
    }
}
```

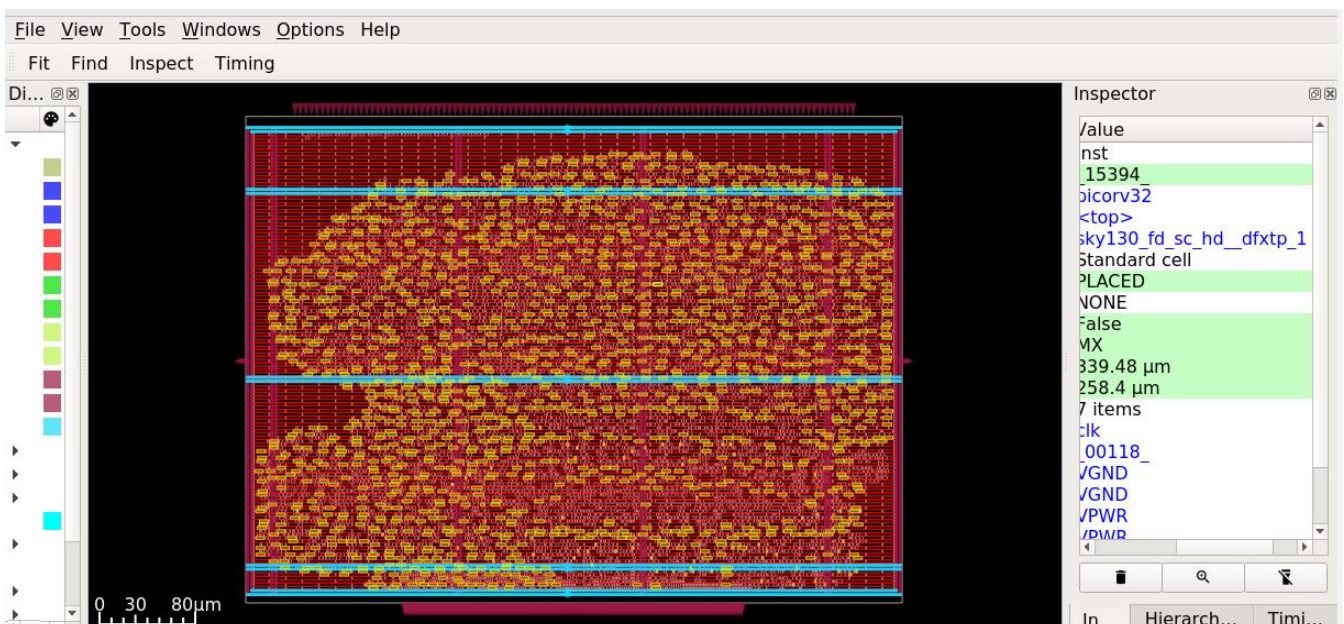


Flipflops: - set block [ord::get_db_block]

```

foreach inst [$block getInsts] {
  set name [[$inst getMaster] getName]
  if {[regexp -nocase "df" $name]} {
    gui::highlight_inst [$inst getName] 3
  }
}

```





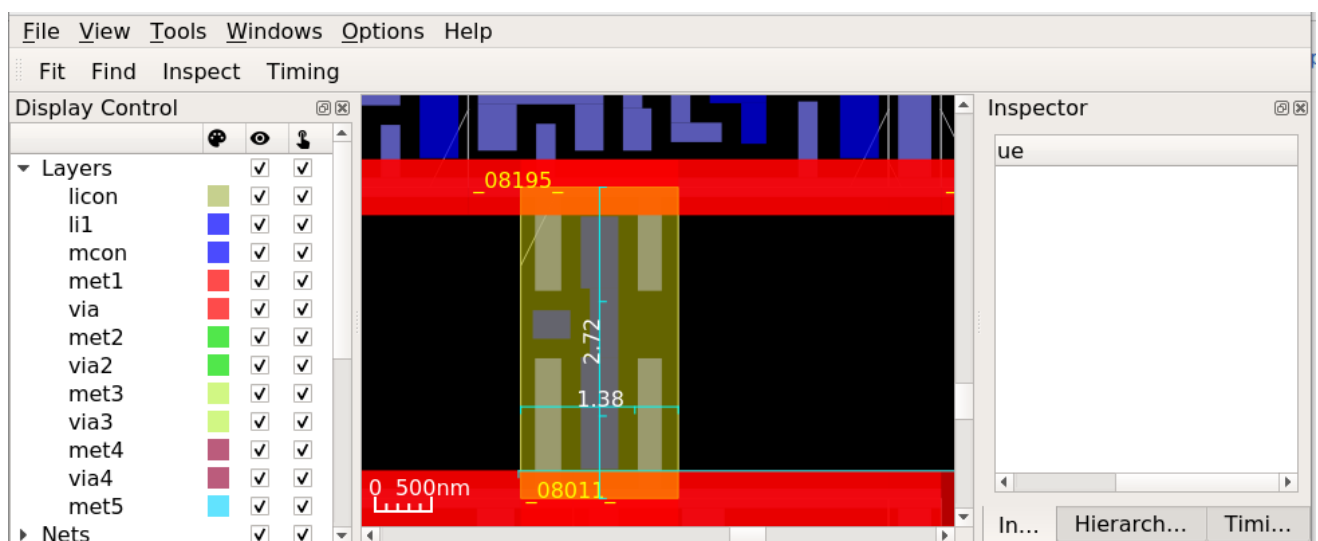
Site Row: -



Parameter	Dimension
Site Row Width	535.44 μm
Site Row Height	2.72 μm

Different Library Cells: -

- **Inverter**



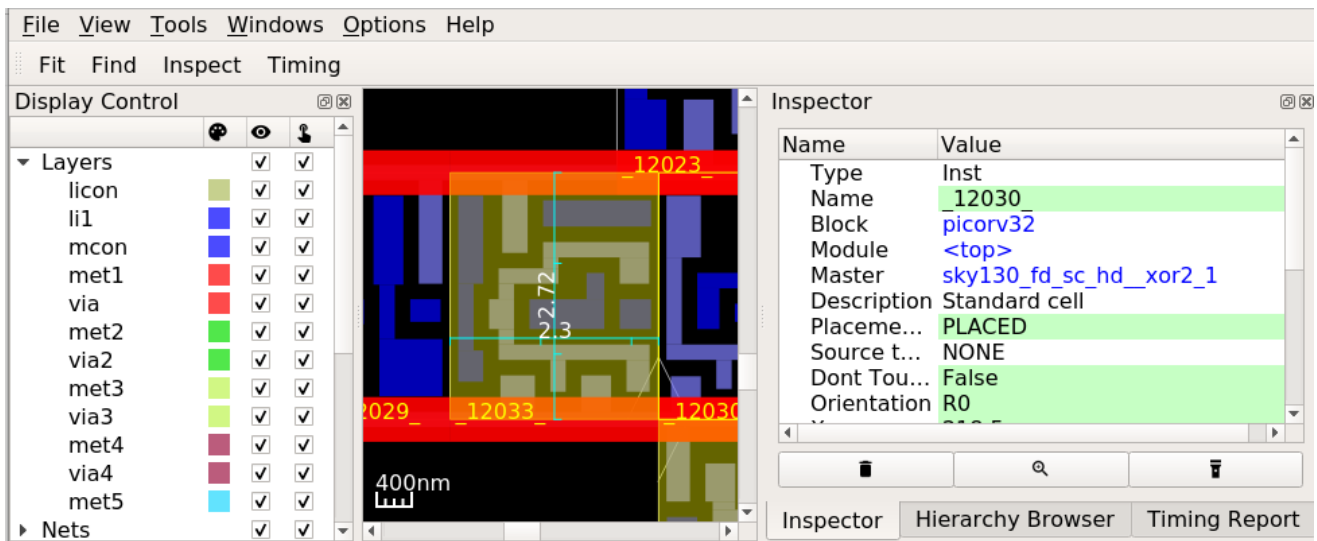
Width = 1.38 μm

Height = 2.72 μm

Height = **2.72 μm**



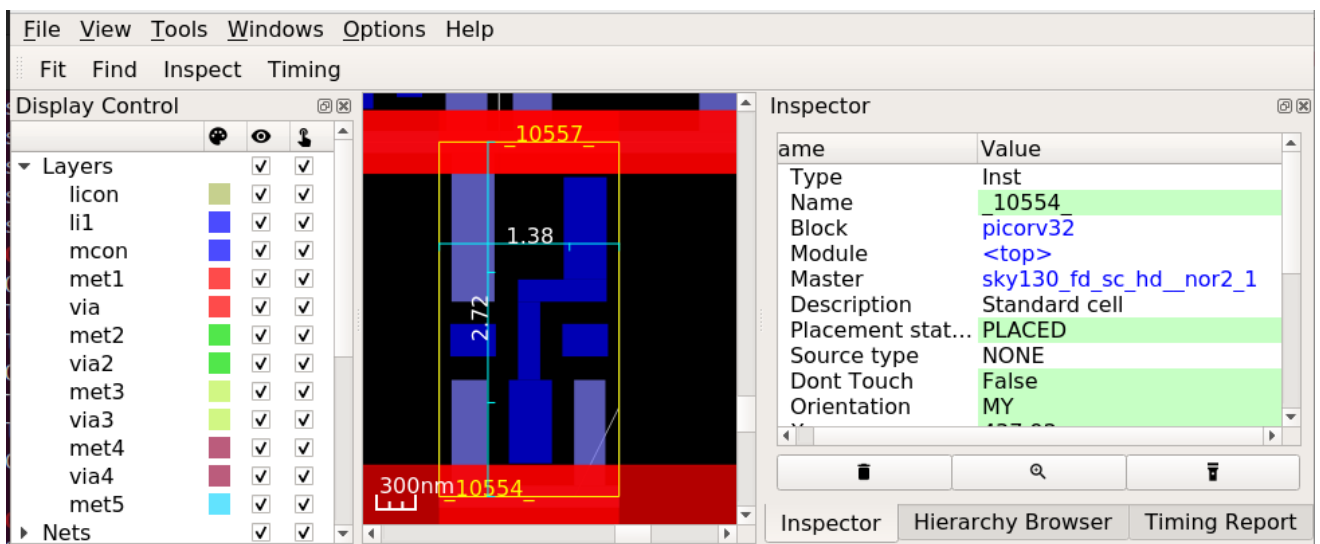
- **OR**



Width = 2.3 μm

Height = 2.72 μm

- **NOR**

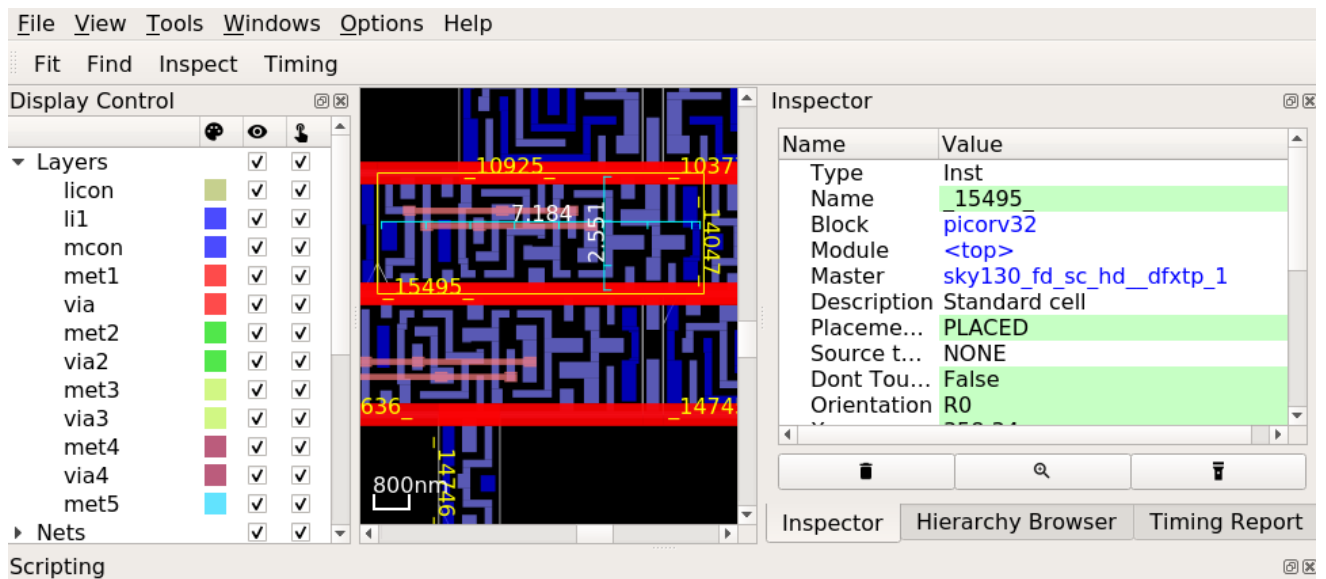


Width = 1.38 μm

Height = 2.72 μm



- **Flipflop**

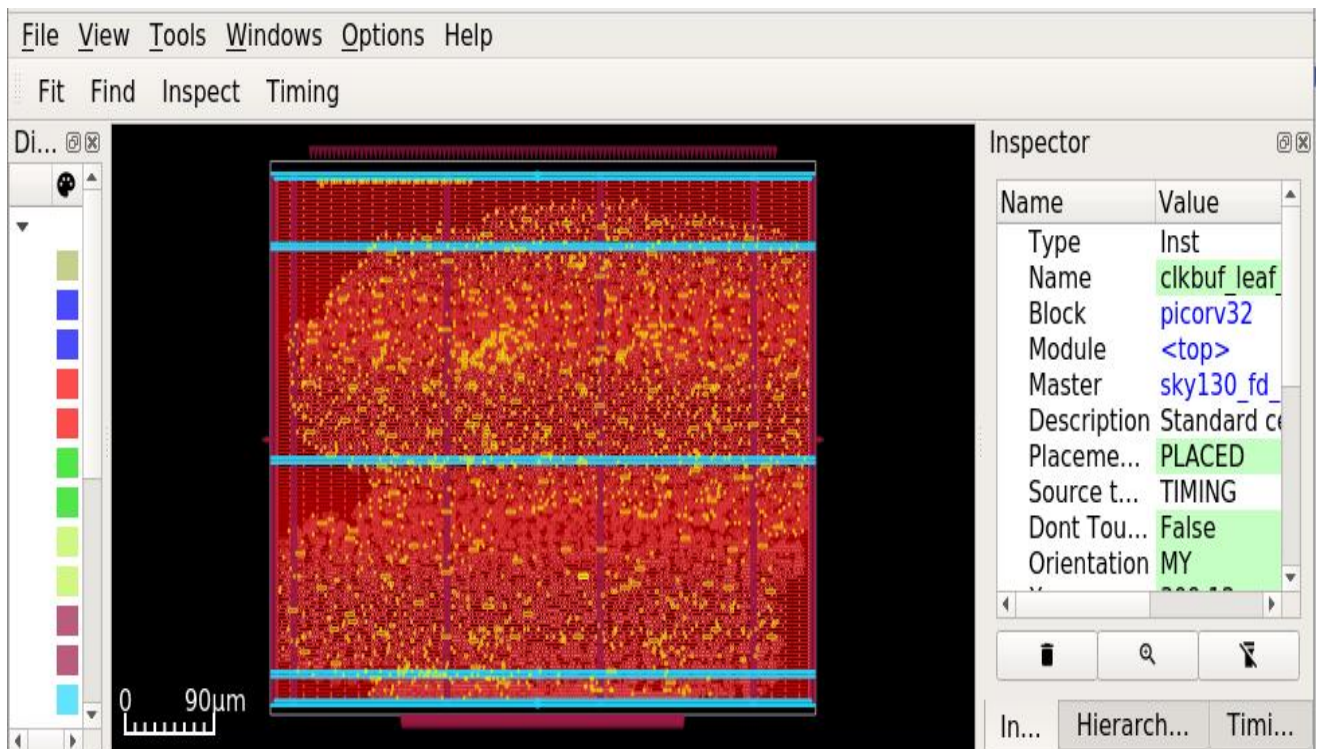


- Width = 7.184 μm
- Height = 2.551 μm

3. CTS

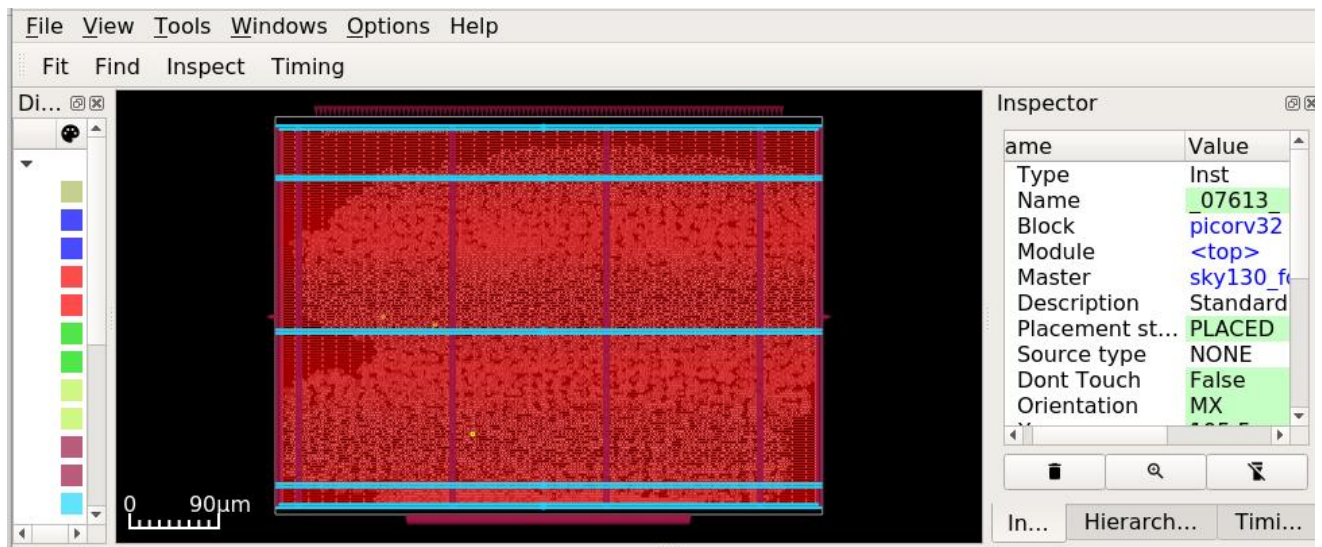
Opening CTS database using command “read_db /openlane/designs/picorv32/runs/RUN_2026.07.04_18.52.21/results/cts/picorv32.odb”.

Clock Buffer: -



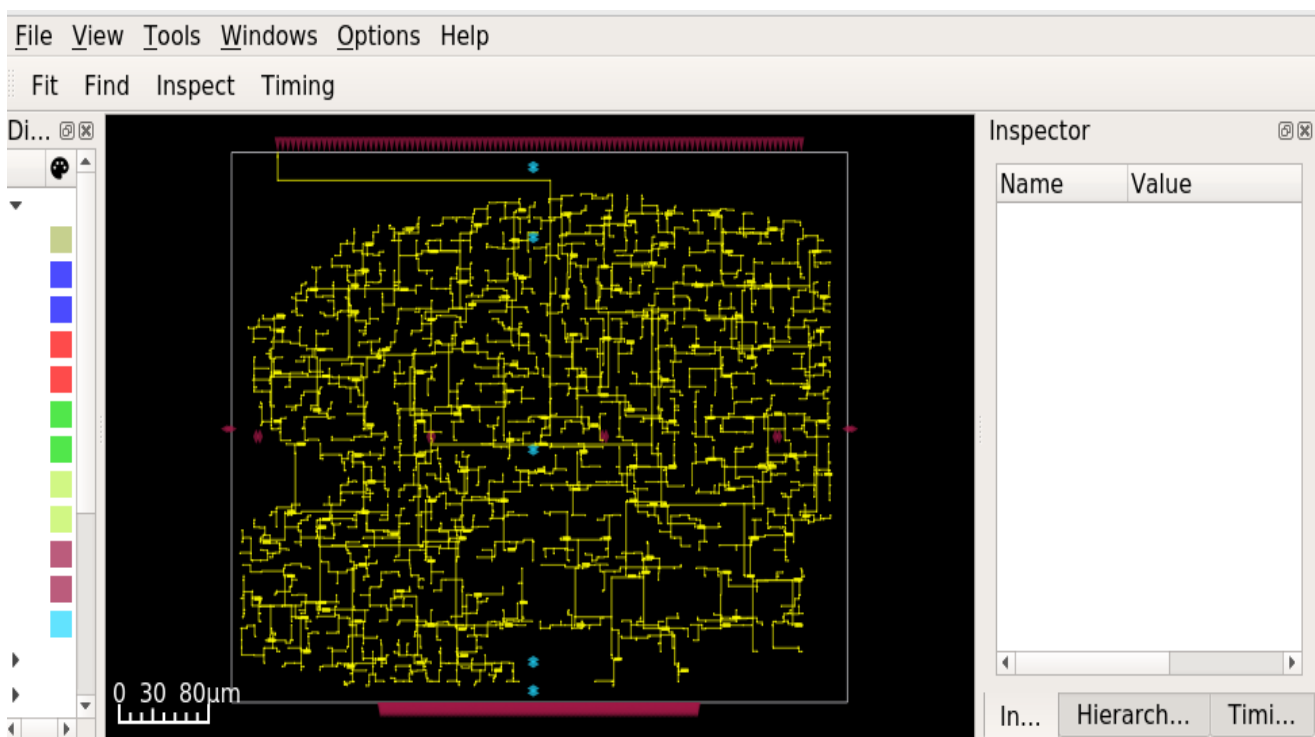


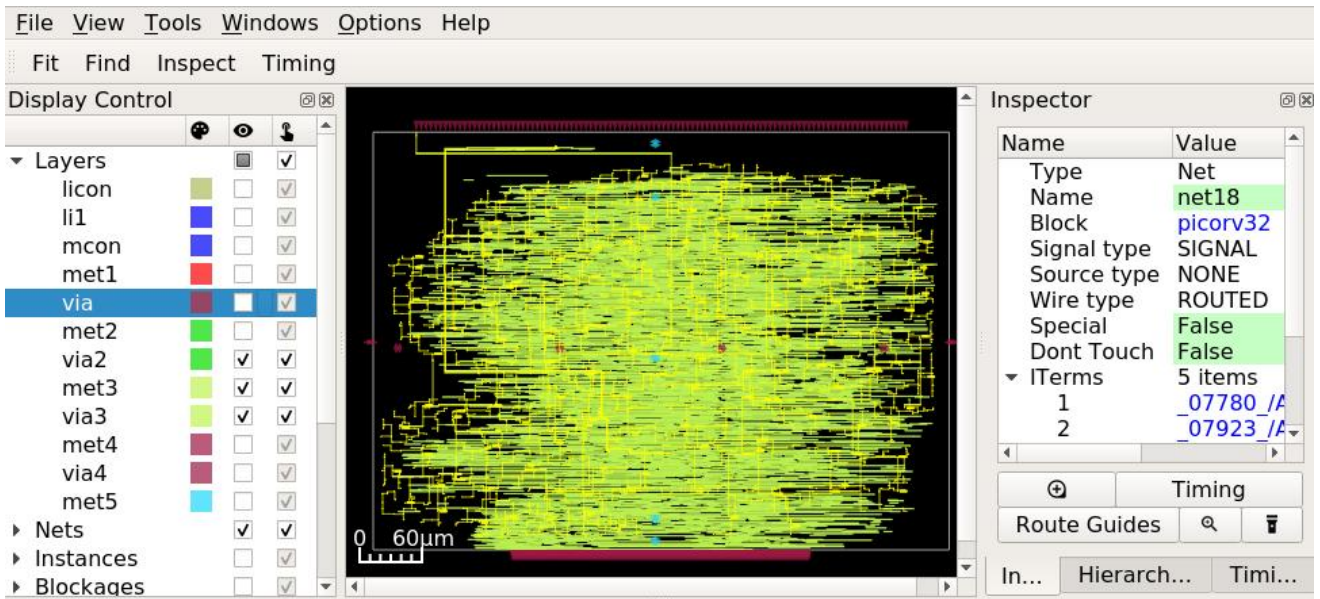
Clock Inverter: -



Clock routing and its metal layer usage: -

```
“foreach net [$block getNets] {  
    if {[regexp -nocase "clk" [$net getName]]} {  
        gui::highlight_net [$net getName] 1  
    }  
}”
```



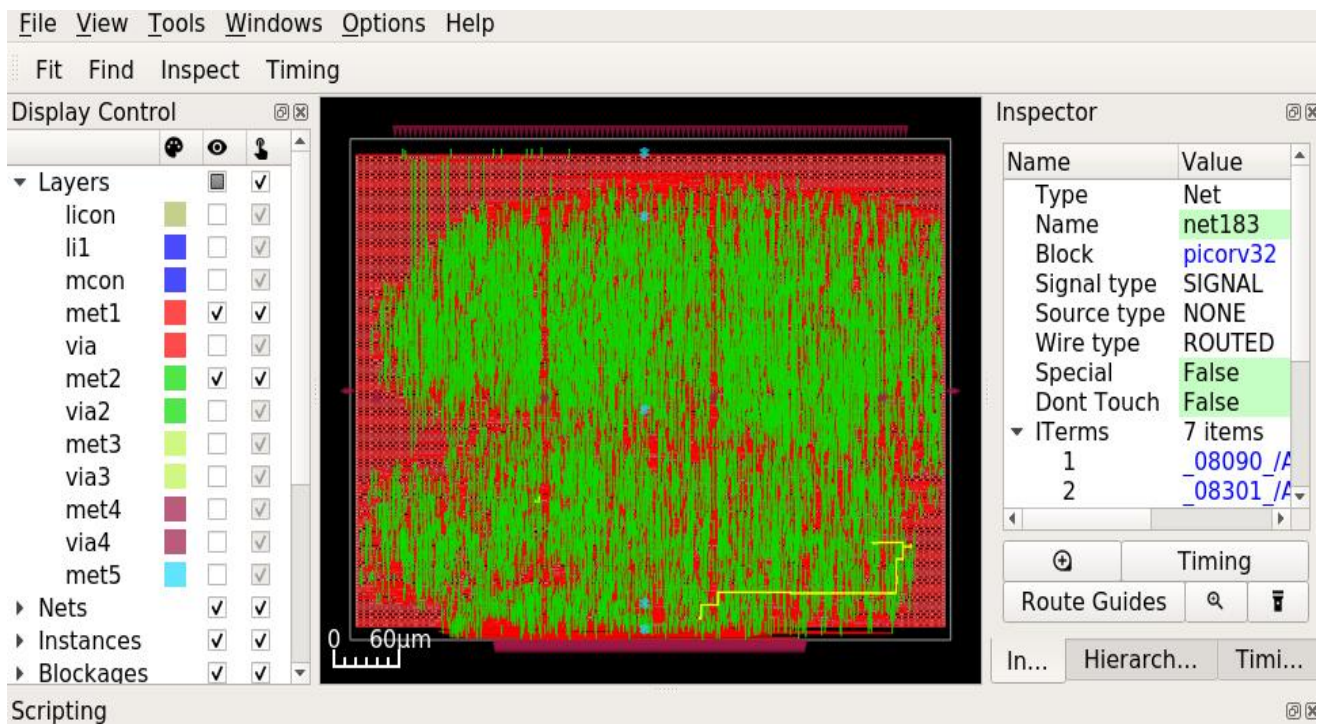


- *Clock routing primarily uses Metal 3 (met3).*

4. Routing

Opening data base in the gui using command “read_db /openlane/designs/picorv32/runs/RUN_2026.07.04_18.52.21/results/routing/picorv32.odb”.

Signal routing and its metal layer usage: -



- *Signal routing primarily uses met1 and met2.*



Newly added buffer and inverter in routing stage: -

- Loading the Routing Database - read_db
/openlane/designs/picorv32/runs/RUN_2026.07.04_18.52.21/results/routing/picorv32.odb
- Extract Buffers and Inverters - set fp [open "/tmp/routing_bufinv.txt" w]

```
foreach inst [$block getInsts] {  
    set master [[[$inst getMaster] getName]  
    if {[regexp {buf|inv} $master]} {  
        puts $fp "[$inst getName] $master"  
    }  
}  
}close $fp
```
- Loading the CTS Database - read_db
/openlane/designs/picorv32/runs/RUN_2026.07.04_18.52.21/results/cts/picorv32.odb
- Comparing Both Files - comm -13 <(sort /tmp/cts_bufinv.txt) <(sort /tmp/routing_bufinv.txt)

```
shams_vlsi@VLSI: ~/Desktop/OpenLane x shams_vlsi@VLSI: ~/Desktop/OpenLane x v
OpenLane Container (1155091): /openlane$ cat /tmp/routing_added.txt
_07613_ sky130_fd_sc_hd__inv_4
_07617_ sky130_fd_sc_hd__buf_4
_07618_ sky130_fd_sc_hd__buf_4
_07620_ sky130_fd_sc_hd__clkbuf_4
_07659_ sky130_fd_sc_hd__clkbuf_8
_07664_ sky130_fd_sc_hd__buf_4
_07671_ sky130_fd_sc_hd__clkbuf_4
_07673_ sky130_fd_sc_hd__clkbuf_4
_07679_ sky130_fd_sc_hd__buf_4
_07680_ sky130_fd_sc_hd__clkbuf_4
_07681_ sky130_fd_sc_hd__clkbuf_8
_07689_ sky130_fd_sc_hd__clkbuf_4
_07696_ sky130_fd_sc_hd__clkinv_4
_07699_ sky130_fd_sc_hd__buf_4
_07702_ sky130_fd_sc_hd__buf_4
_07703_ sky130_fd_sc_hd__clkbuf_4
_07723_ sky130_fd_sc_hd__clkbuf_4
_07733_ sky130_fd_sc_hd__buf_4
_07738_ sky130_fd_sc_hd__clkbuf_4
_07739_ sky130_fd_sc_hd__buf_2
_07757_ sky130_fd_sc_hd__buf_4
_07779_ sky130_fd_sc_hd__clkbuf_4
_07787_ sky130_fd_sc_hd__clkbuf_4
```



```
shams_vlsi@VLSI: ~/Desktop/OpenLane x shams_vlsi@VLSI: ~/Desktop/OpenLane x v
_08929_ sky130_fd_sc_hd__buf_2
_08933_ sky130_fd_sc_hd__buf_2
_08936_ sky130_fd_sc_hd__buf_2
_08951_ sky130_fd_sc_hd__buf_2
_08957_ sky130_fd_sc_hd__buf_2
_08960_ sky130_fd_sc_hd__buf_2
_08964_ sky130_fd_sc_hd__buf_1
_08967_ sky130_fd_sc_hd__buf_4
_08971_ sky130_fd_sc_hd__buf_1
_08975_ sky130_fd_sc_hd__buf_1
_08979_ sky130_fd_sc_hd__buf_1
_08983_ sky130_fd_sc_hd__buf_1
_08993_ sky130_fd_sc_hd__inv_6
_08994_ sky130_fd_sc_hd__buf_4
_08995_ sky130_fd_sc_hd__clkbuf_8
_09004_ sky130_fd_sc_hd__buf_4
_09005_ sky130_fd_sc_hd__buf_6
_09006_ sky130_fd_sc_hd__buf_8
_09008_ sky130_fd_sc_hd__buf_8
_09013_ sky130_fd_sc_hd__buf_6
_09018_ sky130_fd_sc_hd__clkbuf_8
_09024_ sky130_fd_sc_hd__buf_6
_09025_ sky130 fd sc hd clkbuf 8
```

```
output80 sky130_fd_sc_hd__clkbuf_4
output81 sky130_fd_sc_hd__clkbuf_4
output82 sky130_fd_sc_hd__clkbuf_4
output83 sky130_fd_sc_hd__clkbuf_4
output84 sky130_fd_sc_hd__clkbuf_4
output85 sky130_fd_sc_hd__clkbuf_4
output86 sky130_fd_sc_hd__clkbuf_4
output87 sky130_fd_sc_hd__clkbuf_4
output88 sky130_fd_sc_hd__clkbuf_4
output89 sky130_fd_sc_hd__clkbuf_4
output90 sky130_fd_sc_hd__clkbuf_4
output91 sky130_fd_sc_hd__clkbuf_4
output92 sky130_fd_sc_hd__clkbuf_4
output93 sky130_fd_sc_hd__clkbuf_4
output94 sky130_fd_sc_hd__clkbuf_4
output95 sky130_fd_sc_hd__clkbuf_4
output96 sky130_fd_sc_hd__clkbuf_4
output97 sky130_fd_sc_hd__clkbuf_4
output98 sky130_fd_sc_hd__clkbuf_4
output99 sky130_fd_sc_hd__clkbuf_4
wire1 sky130_fd_sc_hd__buf_1
wire237 sky130_fd_sc_hd__clkbuf_2
OpenLane Container (ff5509f):/openlane$
```



Conclusion

The complete Physical Design (PnR) flow of the **picorv32** design was successfully carried out using the OpenLane flow. The design progressed through floorplanning, power distribution network (PDN) generation, placement, clock tree synthesis (CTS), routing, and signoff.

During floorplanning, the core area was generated based on the specified utilization and aspect ratio, and the I/O pins were placed on the designated metal layer. A PDN core ring was created to provide reliable power and ground distribution throughout the design.

Placement arranged the standard cells within the core while minimizing wirelength and avoiding overlaps. During CTS, clock buffers and inverters were automatically inserted to distribute the clock signal efficiently and reduce clock skew. Global and detailed routing then connected all signal nets while satisfying the routing constraints using the available metal layers.

The routing database was analyzed to study the final routed design, and a comparison between the CTS and routing stages was performed to identify any additional buffers and inverters inserted during routing optimization. The design database was also explored using the OpenROAD GUI to visualize the floorplan, core boundary, PDN, placement, clock network, and routed interconnections.

Overall, the experiment demonstrated the complete ASIC physical design flow and provided practical understanding of how OpenLane and OpenROAD automate placement, clock tree synthesis, routing, and post-routing optimization to generate a manufacturable layout.